



웹 퍼블리셔를 위한 제이쿼리

코딩웍스 jQuery 핵심 가이드북(Guidebook)



JQUERY 핵심 이론

01. 제이쿼리 연결 및 기본 문법

- 제이쿼리 연결 - 다운로드 후 링크 방식
- 제이쿼리 연결 - CDN 링크 방식
- 제이쿼리 기본 문법 - 스크립트 작성 위치/주석

02. 제이쿼리 필수 메서드 Part1

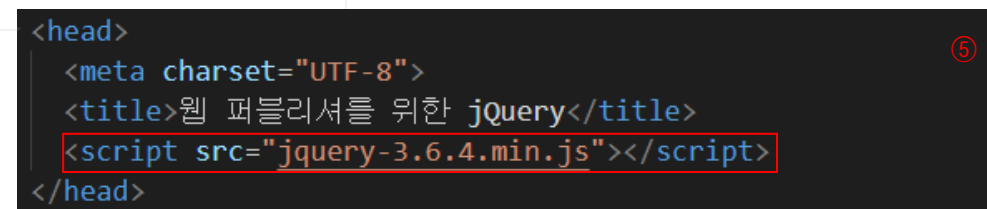
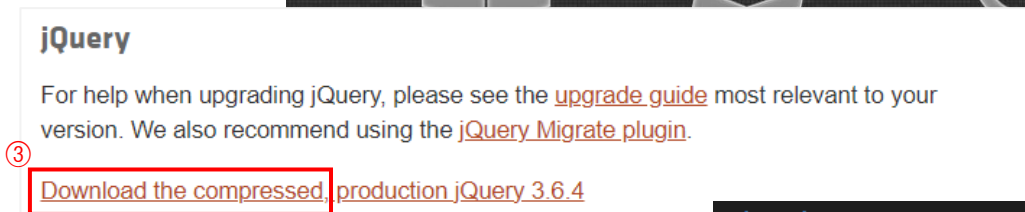
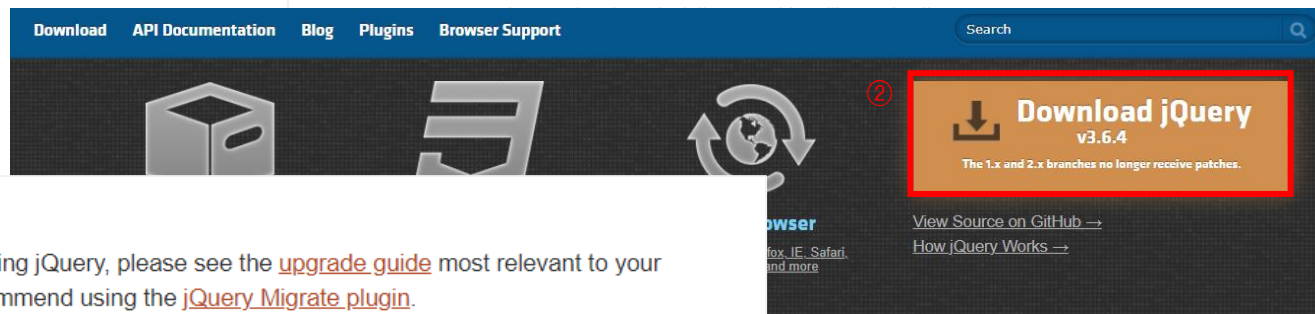
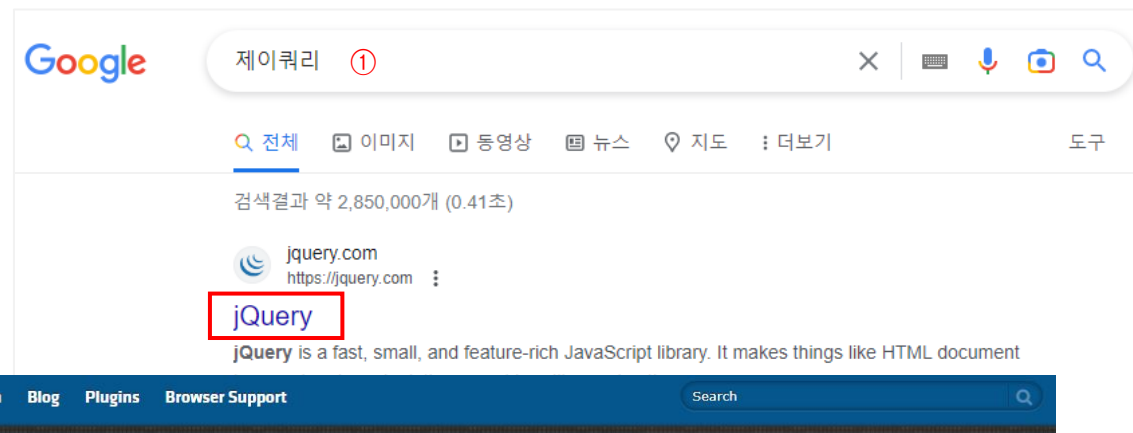
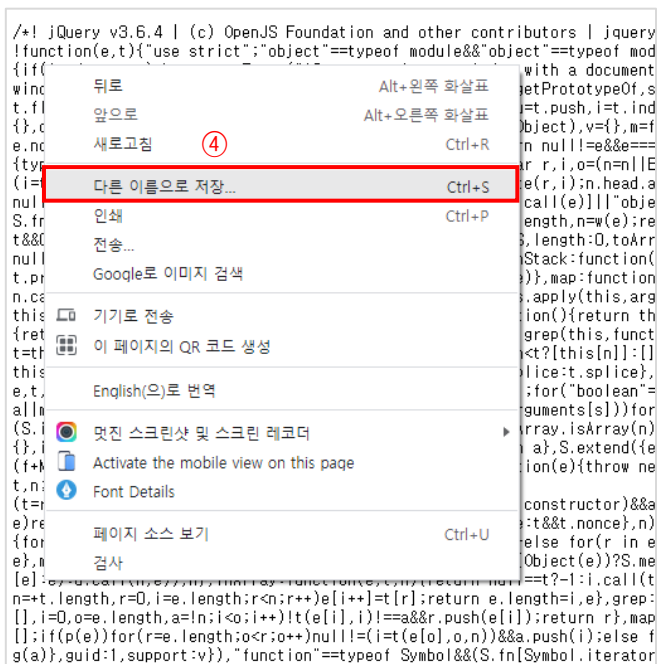
- CSS 스타일 변경 메서드
- 요소 찾기 메서드
- 마우스 이벤트 함수(function) 만들기 & 제이쿼리 변수(variables)
- 클래스 추가 제거 메서드
- 이펙트 메서드
- 텍스트 관련 메서드
- 속성 관련 메서드
- 동적으로 html 요소 삽입/제거
- width, height 관련 메서드
- 위치 관련 메서드
- 요소의 스크롤 위치 메서드
- 복사 및 감싸기 관련 메서드
- 이벤트 종류(마우스 이벤트, 키보드 이벤트, 폼 관련 이벤트)

03. 제이쿼리 필수 메서드 Part2

- 애니메이션 효과 animate()
- 개별 문서를 페이지의 원하는 곳에 불러오는 load() 메서드
- load 메서드로 load된 html 파일 안에 콜백함수 사용하기

01) 제이쿼리 연결 - 다운로드 후 링크 방식

- ① 구글에서 제이쿼리 검색 후 제이쿼리 공식 웹사이트 접속
- ② 다운로드 버튼 클릭
- ③ Download the compressed 클릭
- ④ 마우스 우측 버튼 클릭 후 다른 이름으로 저장 - 파일명은 변경하지 않습니다. `jquery-3.6.4.min.js`
- ⑤ head 사이에 script 태그로 링크합니다.



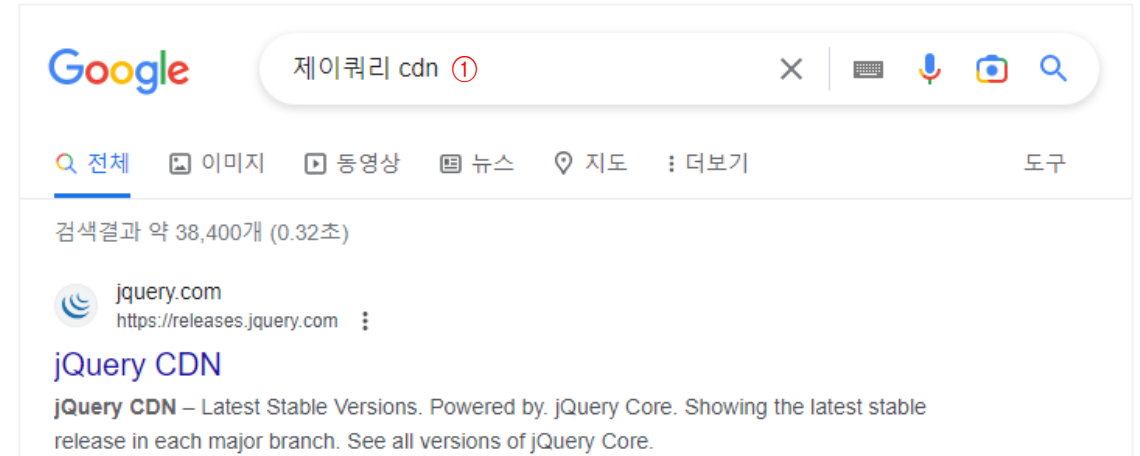
01) 제이쿼리 연결 - CDN 링크 방식 다운로드 후 링크 방식 보다 CDN 방식을 일반적으로 사용합니다.

- ① 구글에서 제이쿼리 cdn 검색 후 제이쿼리 공식 웹사이트 접속
- ② minified 클릭
- ③ 복사 버튼 클릭
- ④ head 사이에 script 태그로 링크합니다.

[참고] CDN 링크의 모든 속서를 사용하지 말고 코드를 간결하게 하기 위해서 src 속성만 사용합니다.

- 제이쿼리 CDN은 앞으로 계속 사용되므로 텍스트 파일에 저장해서 필요할 때 복사해서 사용합니다.
- head 안에는 많은 링크가 있기 때문에 항상 주석처리를 충실히 해야 합니다.

```
<head>
<meta charset="UTF-8">
<title>웹 퍼블리셔를 위한 jQuery</title>
<!-- jQuery CDN -->
<script src="https://code.jquery.com/jquery-3.6.4.min.js"></script>
</head>
```



jQuery 3.x

- jQuery Core 3.6.4 - uncompressed, minified, slim, slim minified

```
<script
src="https://code.jquery.com/jquery-3.6.4.min.js"
integrity="sha256-oP6HI9z1XaZNBrJURtCoUT5SUnxFr8s3BzRl+cbzUq8="
crossorigin="anonymous"></script>
```

③



01) 제이쿼리 - 스크립트 작성 위치와 주석

{ script를 head 사이에 작성하는 경우 vs script를 body에 작성하는 경우 }

```
<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <title>웹 퍼블리셔를 위한 jQuery</title>
  <!-- jQuery CDN -->
  <script src="https://code.jquery.com/jquery-3.6.4.min.js"></script>
  <script>
    $(document).ready(function(){
      // 실행할 스크립트 구문
    });
  </script>
</head>
<body>

  <h1>This is HTML element</h1>

  <script>
    // 실행할 스크립트 구문
  </script>

</body>
</html>
```

① 브라우저가 제이쿼리 라이브러리를 읽은 상태여야 script 내의 제이쿼리 구문이 실행되므로, 제이쿼리 라이브러리는 반드시 script 구문 보다 위에 있어야 합니다.

②

브라우저는 위에서 아래로 읽음

① 브라우저는 위에서 아래로 읽기 때문에 body의 내용을 읽어 오기 전에 script 구문이 실행되면 스크립트가 작동하지 않습니다. 그래서 body의 내용을 모두 읽고 스크립트를 실행시키라는 제이쿼리 구문을 사용해야 합니다.

```
$(document).ready(function() {...})
```

② 브라우저는 위에서 아래로 읽기 때문에 body의 내용을 읽어온 상태이므로 \$(document).ready(function() {...}) 이 필요 없습니다.

```
$(document).ready(function() {
```

```
  // 실행할 스크립트 구문
```

```
})
```

```
$(function() {
  // 실행할 스크립트 구문
})
```

▲ 일반적으로 위의 단축형으로 사용

script에 defer 속성을 사용하면 스크립트의 위치를 body 위에 적어도 잘 작동합니다. 다만, 스크립트를 파일로 링크하는 형태는 defer 속성이 작동하지만 스크립트를 html 내에 작성할 때는 작동하지 않습니다. (가급적 위에 설명한 것 처럼 script의 위치를 잘 지켜주는 것이 좋습니다.)

```
<script src="custom.js" defer></script> → defer="defer" 동일함
```

02) CSS 스타일 변경 메서드 - `css()`

`{ $('선택자').css('속성', '값') }` →

- 속성과 값을 하나 사용하는 경우
- 콤마(,)를 사용해서 속성과 값을 구분

`{ $('선택자').css({'속성': '값', '속성': '값', '속성': '값'}) }` →

- 속성과 값을 여러 개 사용하는 경우
- 콜론(:)을 사용해서 속성과 값을 구분하고 속성과 값을 더 사용할 때는 콤마(,)를 사용해서 구분

속성과 값을 여러 개 사용하는 경우 가독성을 위해서 아래처럼 하는게 좋습니다.

문서객체.css({
 '속성': '값',
 '속성': '값',
 '속성': '값'
 })

자바스크립트에서 객체 개념과 동일함

```
<h1>This is H1 element</h1>
<p class="frame">
  Lorem ipsum dolor sit, amet consectetur adipisicing elit.
  Perferendis, laborum animi. Voluptatibus earum, autem ipsa
  commodi, vel odit magni veniam ut a, sapiente adipisci
  numquam molestias perferendis totam nam mollitia.
</p>
```

HTML

This is H1 element

Lorem ipsum dolor sit, amet consectetur adipisicing elit.
 Perferendis, laborum animi. Voluptatibus earum, autem ipsa
 commodi, vel odit magni veniam ut a, sapiente adipisci
 numquam molestias perferendis totam nam mollitia.

```
<script>
  // 속성과 값을 1개만 사용
  $('h1').css('color', 'crimson');

  // 속성과 값을 여러개 사용
  $('.frame').css({
    'border': '2px solid pink',
    'padding': '25px',
    'font-size': '18px'
  })
</script>
```

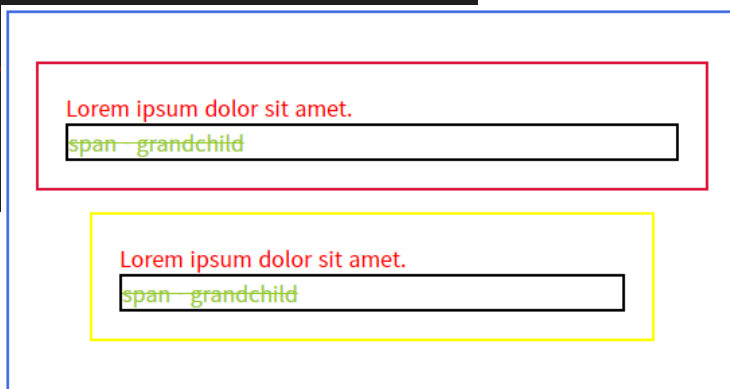
JS

02) 요소 찾기 메서드(#01) - children() find()

{ \$('선택자').children('선택자') }

- \$('선택자')를 기준으로 자식 요소 선택
- children()의 괄호에 선택자가 없다면 모든 자식 요소를 선택
- CSS 선택자로 예를 들면 자식 선택자(>)

```
<div class="frame" style="width: 500px; HTML>
  <p>
    Lorem ipsum dolor sit amet.
    <span>span - grandchild</span>
  </p>
  <blockquote>
    Lorem ipsum dolo
    <span>span - gra
  </blockquote>
</div>
```



{ \$('선택자').find('선택자') }

- \$('선택자')를 기준으로 자손 요소 선택
- CSS 선택자로 예를 들면 자식 스페이스(_)

```
$('.frame').css({
  'border': '2px solid royalblue',
  'padding': '20px'
});
$('.frame').children().css({ → • .frame을 기준으로 모든 자식 요소 선택
  'color': 'red',           • CSS 예시) .frame > *
  'padding': '20px',
  'border': '2px solid crimson'
});
$('.frame').children('blockquote').css({ → • .frame을 기준으로 자식 요소 중에
  'border': '2px solid yellow'           blockquote만 선택
                                          • CSS 예시) .frame > blockquote
});
$('.frame').children().children().css({ → • .frame을 기준으로 자식 요소의 자식 요소 선택
  'display': 'block',                   • CSS 예시) .frame > * > *
  'color': 'yellowgreen',               • .frame을 기준으로 모든 자손 요소 선택
  'border': '2px solid #000'           • CSS 예시) .frame span
});
$('.frame').find('span').css('text-decoration', 'line-through');
```

02) 요소 찾기 메서드(#02) - siblings() parent()

{ `$('선택자').siblings('선택자')` }

- `$('선택자')`를 기준으로 형제 요소 선택
- `siblings()`의 괄호에 선택자가 없다면 모든 형제 요소를 선택

```
<div class="list-items">
  <b>This is B tage</b>
  <a href="#" class="list-item">List Item 1</a>
  <a href="#" class="list-item">List Item 2</a>
  <a href="#" class="list-item start">List Item 3</a>
  <a href="#" class="list-item">List Item 4</a>
  <a href="#" class="list-item">List Item 5</a>
</div>
```

HTML

{ `$('선택자').parent('선택자')` }

- `$('선택자')`를 기준으로 부모 요소 선택
- `parent()`의 괄호 안에 선택자가 없어도 동일함(HTML 구조에서 부모 요소는 only 한 개 입니다.)

```
$('.list-item').css({
  'display': 'block',
  'color': '#000',
  'text-decoration': 'none'
});
$('.list-item').siblings('a').css({ → .list-item의 형제 요소 중에서 a태그만 선택
  'color': 'blue'
});
→ .list-item.start의 모든 형제 요소 선택(자신은 제외)
$('.list-item.start').siblings().css('background-color', 'pink');
```

JS

```
$('.list-item.start').parent().css({
  'border': '2px solid green',
  'padding': '20px'
});
```

This is B tage
List Item 1
List Item 2
List Item 3
List Item 4
List Item 5

This is B tage
List Item 1
List Item 2
List Item 3
List Item 4
List Item 5

02) 요소 찾기 메서드(#03) - next() prev() nextAll() prevAll() nextUntil() prevUntil()

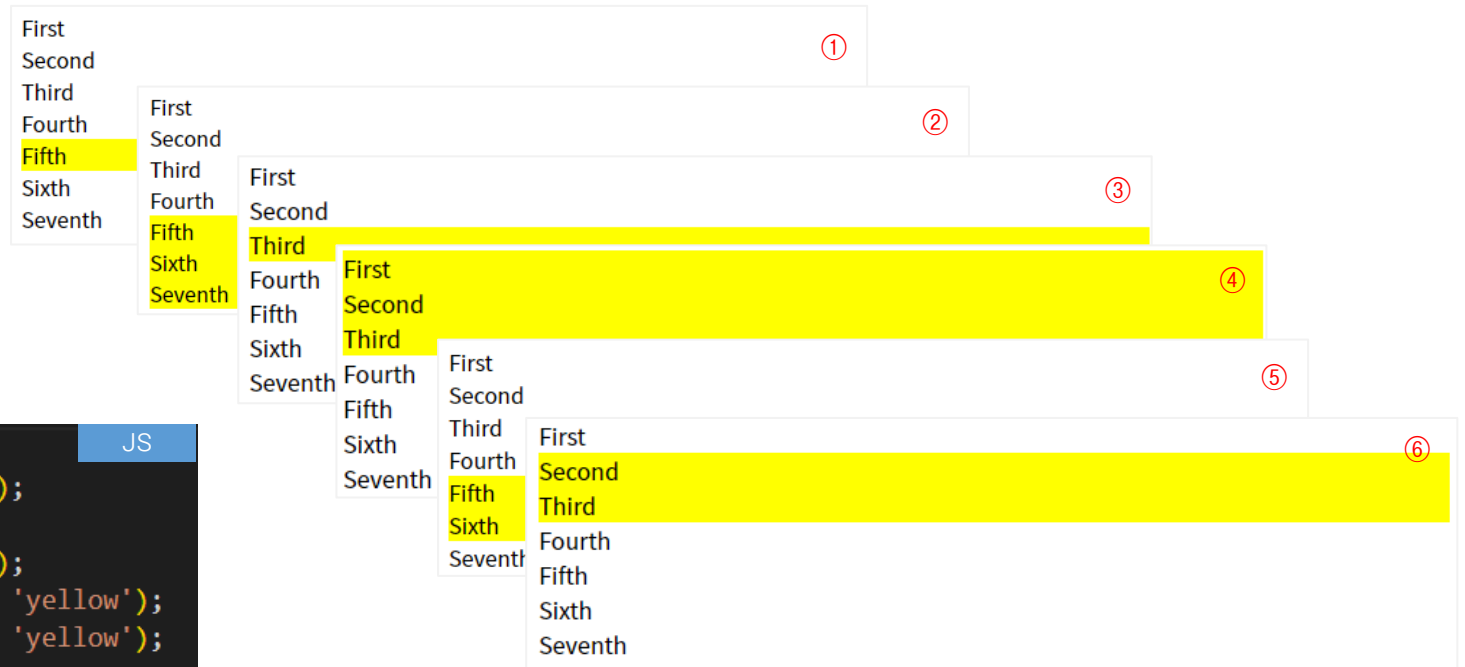
- `{` `$('#선택자').next()` `}` →
 - `$('#선택자')`를 기준으로 다음 요소 선택
 - `next()`의 괄호 안에 선택자 불필요(다음 요소는 Only One)
- `{` `$('#선택자').nextAll()` `}` →
 - `$('#선택자')`를 기준으로 모든 다음 요소 선택
 - `nextAll()`의 괄호 안에 선택자 불필요

```
<div class="parent">
  <div class="flag">First</div>
  <div>Second</div>
  <div>Third</div>
  <div id="target">Fourth</div>
  <div>Fifth</div>
  <div>Sixth</div>
  <div class="flag">Seventh</div>
</div>
```

HTML

```
① $('#target').next().css('background-color', 'yellow');
② $('#target').nextAll().css('background-color', 'yellow');
③ $('#target').prev().css('background-color', 'yellow');
④ $('#target').prevAll().css('background-color', 'yellow');
⑤ $('#target').nextUntil('.flag').css('background-color', 'yellow');
⑥ $('#target').prevUntil('.flag').css('background-color', 'yellow');
```

JS



02) 요소 찾기 메서드(#04) - 인덱스 번호 기준으로 요소 선택하는 요소 탐색 선택자

제이쿼리 요소 찾기(선택)은 CSS 선택자에서 사용하는 선택자를 거의 모두 사용 가능합니다.

{ eq(index) 요소:first-child 요소:last-child 요소:nth-child(n) }

```
<div class="box">
  <div></div>
  <div></div>
  <div></div>
  <div></div>
  <div></div>
</div>
```

```
.box div {
  border: 2px solid crimson;
  width: 100px;
  height: 100px;
  display: inline-block;
}
```

```
$('.box div:nth-child(2)').css('background-color', 'dodgerblue');
```



▲ nth-child(index) : CSS 인덱스 번호는 1부터 시작

```
$('.box div').eq(2).css('background-color', 'crimson');
```

▲ eq(index) : 자바스크립트에서 index 번호는 0부터 시작



```
$('.box div:first-child').css('background-color', 'yellowgreen');
$('.box div:last-child').css('background-color', 'yellowgreen');
```



02) 요소 찾기 메서드(#05) - 인덱스 번호 기준으로 요소 선택하는 요소 탐색 선택자

{ 요소:gt(index) 요소:lt(index) 문서객체.not('선택자') 문서객체.has('선택자') }

```
<div class="box">
  <div></div>
  <div></div>
  <div class="target"></div>
  <div></div>
  <div><span>span tag</span></div>
</div>
```

```
.box div {
  border: 2px solid crimson;
  width: 100px;
  height: 100px;
  display: inline-block;
  vertical-align: middle;
}
```

```
$('.box div').not('.target').css('background-color', 'pink');
```



선택요소를 제외(not)하고 선택

```
$('.box div').has('span').css('background-color', 'pink');
```

선택 요소를 포함(has)한 요소를 선택
.box div 중에서 span 태그를 포함한 요소를 선택



```
$('.box div:gt(2)').css('background-color', 'pink');
```

▲ gt(index) : 선택된 요소부터 다음의 모든 요소 선택
자바스크립트에서 index 번호는 0부터 시작



```
$('.box div:lt(2)').css('background-color', 'pink');
```

▲ lt(index) : 선택된 요소부터 이전의 모든 요소 선택
자바스크립트에서 index 번호는 0부터 시작



[참고] 제이쿼리 마우스 이벤트 함수(function) 만들기 & 제이쿼리 변수(variables)

제이쿼리 학습에서 거의 반을 차지하는 것이 바로 마우스 이벤트 함수를 잘 만드는 것입니다.

```
$('#선택자').마우스이벤트(function(){  
    // 마우스 이벤트에 작동하는 스크립트 구문  
});
```

▲ 제이쿼리는 괄호가 중첩되는 경우가 많습니다. 중첩된 괄호를 1개라도 잘못하면 모든 스크립트가 작동하지 않습니다.

```
$('.selector').click
```

click (method) JQuery.click... ①

```
$('.selector').click(function) ②
```

```
$('.selector').click(function()) ③
```

```
$('.selector').click(function(){})) ④
```

```
$('.selector').click(function(){  
    }) ⑤
```

[참고] 비주얼 스튜디오 코드에서 제이쿼리 마우스 이벤트

- ① 마우스 이벤트 이름 입력
- ② 소괄호 열고 function 입력
- ③ 소괄호 입력
- ④ 커서로 오른쪽 한 칸 이동하고 중괄호 입력
- ⑤ 엔터치고 스크립트 구문 작성

[중요 포인트]

- 비주얼 스튜디오 코드, 블라켓과 같은 텍스트에디터에서는 괄호를 열면 자동으로 닫힙니다. 역시 따옴표를 입력하면 자동으로 따옴표가 닫힙니다.
- 쌍따옴표 또는 외따옴표 어느 것을 사용해도 상관없습니다. 다만, 일관성 있게 사용하시기 바랍니다.(외따옴표가 편합니다!)

[참고] 제이쿼리에서 변수 사용하기

- 변수명 앞에 \$ 붙임
- 제이쿼리 요소를 담은 변수라는 의미로 가급적 변수명은 \$를 사용

```
var $변수명 = $('#선택자');
```

```
var $box = $('#box');  
$box.css('color','red');
```

```
var box = $('#box');  
box.css('color','red');
```

03) 클래스 추가 제거 메서드(#01) - addClass() removeClass() toggleClass()

{ \$('선택자').addClass('클래스네임') }

\$('h1').addClass('active')

▲ 클래스네임을 추가

{ \$('선택자').removeClass('클래스네임') }

\$('h1').removeClass('active')

▲ 클래스네임을 제거

{ \$('선택자').toggleClass('클래스네임') }

\$('h1').toggleClass('active')

▲ 클래스네임을 추가 제거

※ 클래스네임에 점(.)이 없어야 합니다.

```
// 클래스 추가
$('.add').click(function(){
  $(this).addClass('active');
});
// 클래스 제거
$('.remove').click(function(){
  $(this).removeClass('active');
});
// 클래스 추가 제거
$('.toggle').click(function(){
  $(this).toggleClass('active');
});
```

JS

```
<div class="buttons">
  <button class="btn add">addClass</button>
  <button class="btn remove active">removeClass</button>
  <button class="btn toggle">toggleClass</button>
</div>
```

addClass

removeClass

toggleClass

```
.btn {
  border: none;
  width: 120px;
  padding: 10px;
  cursor: pointer;
  margin: 10px;
  border-radius: 5px;
  font-size: 1.1em;
  color: #fff;
  background-color: #f1c40f;
}
.active {
  box-shadow: 0 0 15px rgba(0, 0, 0, 0.3);
  background-color: #e74c3c;
}
```

[참고] this 란?

- this는 선택자를 반복하지 않고 자기 자신을 의미
- 버튼이 여러 개 있지만 클릭이 된 것이 this

03) 클래스 추가 제거 메서드(#01) - 스몰미션(옷 사이즈 선택 버튼)

Small Mission) 제이쿼리 메서드를 사용해서 쇼핑몰 사이즈 선택 버튼 만들기

```
<div class="frame">  
  <h2>Select Size</h2>  
  <div class="size">  
    <button>xs</button>  
    <button>s</button>  
    <button>m</button>  
    <button>l</button>  
    <button>xl</button>  
    <button>xxl</button>  
  </div>  
</div>
```

HTML

```
/* Custom CSS */  
.size button {  
  border: none;  
  width: 40px;  
  height: 40px;  
  text-transform: uppercase;  
  cursor: pointer;  
  background-color: #gainsboro;  
}  
.size button.active {  
  background-color: #crimson;  
  color: #fff;  
}
```

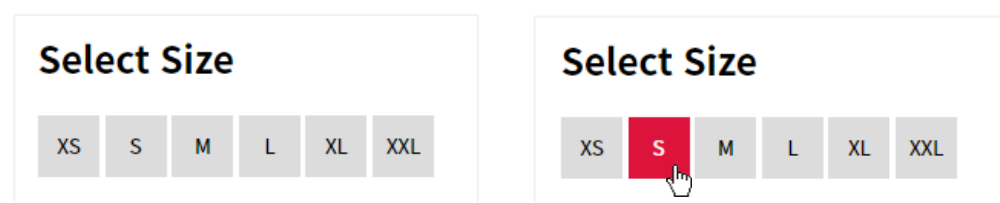
CSS

```
$(this).addClass('active').siblings().removeClass('active');
```

아래 코드를 위 코드처럼 체이닝(chaining) 방식으로 한줄로 사용할 수 있습니다. 하지만 가독성 측면에서 아래처럼 사용하시는 것을 권장 드립니다.

```
$('.size button').click(function(){  
  $(this).addClass('active');  
  $(this).siblings().removeClass('active');  
});
```

JS



Select Size

XS S M L XL XXL

Select Size

XS S M L XL XXL

03) 클래스 추가 제거 메서드(#02) - 스몰미션(모달 띄우기)

Small Mission) 제이쿼리 메서드를 사용해서 심플한 애니메이션 모달창 띄우기

```
<div class="modal-frame">
  <h2>Simple jQuery Modal</h2>
  <button class="open-modal">Open Modal</button>
</div>
<div class="modal-wrapper">
  <div class="modal">
    <div class="modal-header">
      <h2 class="modal-heading">This is a modal</h2>
    </div>
    <div class="modal-content">
      <p>
        Lorem ipsum dolor sit amet, consectetur adipisicing el
        delectus, libero, accusantium dolores inventore obcaec
        sapiente vel laboriosam similique totam id ducimus ape
        blanditiis maiores.
      </p>
      <button class="close-modal">Close Modal</button>
    </div>
  </div>
</div>
```

HTML

모달 콘텐츠를 감싸고 있는 살짝 검은색 배경의 오버레이

실제 모달 내용을 보여주는 부분

```
/* Custom CSS */
.modal-frame { text-align: center; }
.modal-wrapper {
  position: absolute;
  top: 0;
  left: 0;
  width: 100%;
  height: 100%;
  background-color: rgba(0, 0, 0, 0.2);
  opacity: 0;
  visibility: hidden;
  transition: 0.35s;
}
.modal {
  background-color: #fff;
  position: absolute;
  top: 50%;
  left: 50%;
  transform: translate(-50%, -70%);
  box-shadow: 0 0 1.5em hsla(0, 0, 0, 0.5);
  padding: 20px;
  transition: 0.35s;
}
```

→ 안보이는 상태로
→ 안보이면서 마우스이벤트 받지 않게
→ 애니메이션을 위한 트랜지션

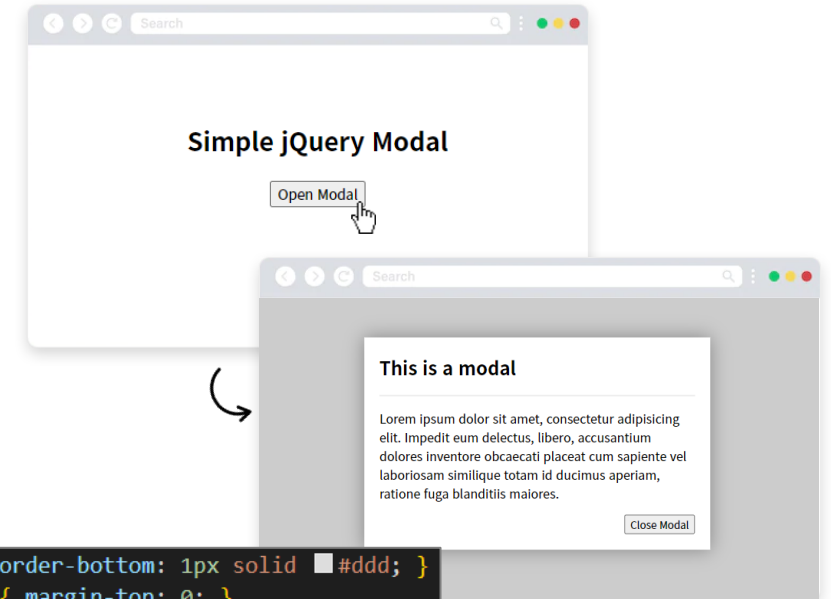
최초에 살짝 위로 올라간 상태

```
.modal-header { border-bottom: 1px solid #ddd; }
.modal-header h2 { margin-top: 0; }
.close-modal { float: right; }

/* Modal with visible class */
.modal-wrapper.visible {
  opacity: 1;
  visibility: visible;
}
.modal-wrapper.visible .modal {
  transform: translate(-50%, -50%);
}
```

→ .visible 클래스가 추가되면 변경되는 CSS

CSS



.visible 클래스가 추가되면 수직 정중앙으로 위치

04) 이펙트 메서드(#01) - show() hide() toggle() fadeOut() fadeIn() fadeToggle() slideDown() slideUp() slideToggle()

{ `$('#선택자').show()` } { `$('#선택자').hide()` } { `$('#선택자').toggle()` }

▲ 요소 보이기 ▲ 요소 숨기기 ▲ 요소 보이기 숨기기

```
<div class="frame">
  <div class="buttons">
    <button class="btn show">show</button>
    <button class="btn hide">hide</button>
    <button class="btn toggle">toggle</button>
  </div>
  <p class="desc">
    Lorem ipsum dolor sit amet consectetur adip
    voluptas quibusdam repudiandae praesentium,
    similique laboriosam modi ea ipsam facilis
    sequi molestiae voluptatem.
  </p>
</div>
```

```
/* Custom CSS */
.frame {
  width: 500px;
}
.buttons { display: flex; gap: 10px; }
.btn {
  flex: 1;
  border: none;
  padding: 10px;
  cursor: pointer;
  border-radius: 5px;
  font-size: 1.1em;
  color: #fff;
  background-color: #8c7ae6;
}
.desc { text-align: justify; }
```

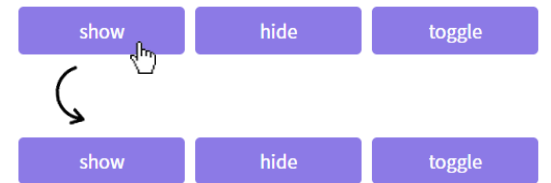
```
// 최초에 요소 숨기기
$('.desc').hide();

// 요소 보이기
$('.show').click(function(){
  $('.desc').show();
});

// 요소 숨기기
$('.hide').click(function(){
  $('.desc').hide();
});

// 요소 보이기 숨기기
$('.toggle').click(function(){
  $('.desc').toggle();
});
```

JS



Lorem ipsum dolor sit amet consectetur adipisicing elit. Accusantium, voluptas quibusdam repudiandae praesentium, repellat sed voluptate inventore similique laboriosam modi ea ipsam facilis assumenda, velit dicta expedita sequi molestiae voluptatem.

04) 이펙트 메서드(#01) - show() hide() toggle() fadeOut() fadeIn() fadeToggle() slideDown() slideUp() slideToggle()

{ `$('#선택자').fadeIn()` } { `$('#선택자').fadeOut()` } { `$('#선택자').fadeToggle()` }

▲ 요소 서서히 보이기

- 속도는 밀리초 단위로 ex)1000
- 넣지 않으면 기본값(가장 적당한 속도)

▲ 요소 서서히 숨기기

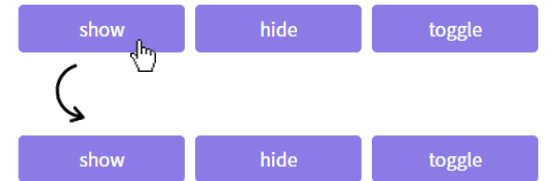
▲ 요소 서서히 보이기 숨기기

```
<div class="frame">
  <div class="buttons">
    <button class="btn show">show</button>
    <button class="btn hide">hide</button>
    <button class="btn toggle">toggle</button>
  </div>
  <p class="desc">
    Lorem ipsum dolor sit amet consectetur adip
    voluptas quibusdam repudiandae praesentium,
    similique laboriosam modi ea ipsam facilis
    sequi molestiae voluptatem.
  </p>
</div>
```

```
/* Custom CSS */
.frame {
  width: 500px;
}
.buttons { display: flex; gap: 10px; }
.btn {
  flex: 1;
  border: none;
  padding: 10px;
  cursor: pointer;
  border-radius: 5px;
  font-size: 1.1em;
  color: #fff;
  background-color: #8c7ae6;
}
.desc { text-align: justify; }
```

```
// 최초에 요소 숨기기
$('.desc').hide();

// 요소 보이기
$('.show').click(function(){
  $('.desc').fadeIn();
});
// 요소 숨기기
$('.hide').click(function(){
  $('.desc').fadeOut();
});
// 요소 보이기 숨기기
$('.toggle').click(function(){
  $('.desc').fadeToggle();
});
```



Lorem ipsum dolor sit amet consectetur adipisicing elit. Accusantium, voluptas quibusdam repudiandae praesentium, repellat sed voluptate inventore similique laboriosam modi ea ipsam facilis assumenda, velit dicta expedita sequi molestiae voluptatem.

`$('.desc').stop().fadeToggle();`

▲ 반복해서 누르면 애니메이션 효과의 명령은 큐(Queue)에 계속 누적되어 반복됩니다. 이런 반복을 멈추게 하는 **stop() 메서드** 필요

04) 이펙트 메서드(#01) - show() hide() toggle() fadeOut() fadeIn() fadeToggle() slideDown() slideUp() slideToggle()

{	<code>\$('#선택자').slideDown()</code>	}	{	<code>\$('#선택자').slideUp()</code>	}	{	<code>\$('#선택자').slideToggle()</code>	}
	▲ 요소 아래로 서서히 보이기			▲ 요소 아래로 서서히 숨기기			▲ 요소 아래로 서서히 보이기 숨기기	
	• 속도는 밀리초 단위로 ex)1000 • 넣지 않으면 기본값(가장 적당한 속도)							

```
<div class="frame">
  <div class="buttons">
    <button class="btn show">show</button>
    <button class="btn hide">hide</button>
    <button class="btn toggle">toggle</button>
  </div>
  <p class="desc">
    Lorem ipsum dolor sit amet consectetur adip
    voluptas quibusdam repudiandae praesentium,
    similique laboriosam modi ea ipsam facilis
    sequi molestiae voluptatem.
  </p>
</div>
```

```
/* Custom CSS */
.frame {
  width: 500px;
}
.buttons { display: flex; gap: 10px; }
.btn {
  flex: 1;
  border: none;
  padding: 10px;
  cursor: pointer;
  border-radius: 5px;
  font-size: 1.1em;
  color: #fff;
  background-color: #8c7ae6;
}
.desc { text-align: justify; }
```

```
// 최초에 요소 숨기기
$('.desc').hide();

// 요소 보이기
$('.show').click(function(){
  $('.desc').slideDown();
});

// 요소 숨기기
$('.hide').click(function(){
  $('.desc').slideUp();
});

// 요소 보이기 숨기기
$('.toggle').click(function(){
  $('.desc').slideToggle();
});
```



Lorem ipsum dolor sit amet consectetur adipisicing elit. Accusantium, voluptas quibusdam repudiandae praesentium, repellat sed voluptate inventore similique laboriosam modi ea ipsam facilis assumenda, velit dicta expedita sequi molestiae voluptatem.

`$('.desc').stop().slideToggle();`

▲ 반복해서 누르면 애니메이션 효과의 명령은 큐(Queue)에 계속 누적되어 반복됩니다. 이런 반복을 멈추게 하는 **stop() 메서드** 필요

04) 이펙트 메서드 - 스몰미션

Small Mission) 제이쿼리 메서드를 사용해서 심플한 드롭다운(Drop Down) 네비게이션 만들기

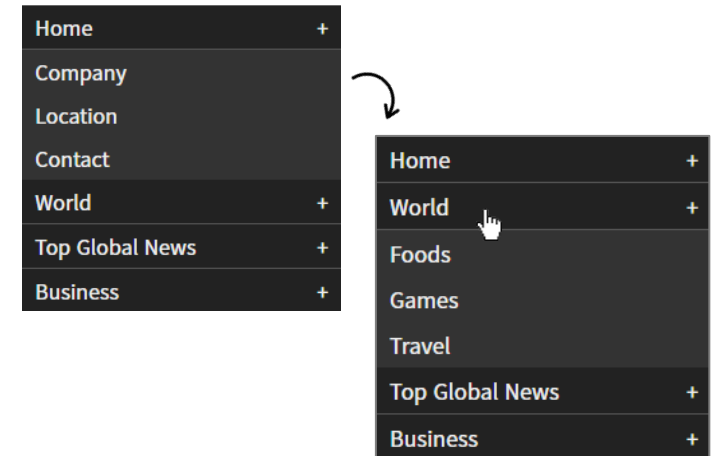
```
<ul class="menu">
  <li>
    <a href="#none">home</a>
    <div class="sub-menu">
      <a href="#none">Company</a>
      <a href="#none">Location</a>
      <a href="#none">Contact</a>
    </div>
  </li>
  <li>
    <a href="#none">world</a>
    <div class="sub-menu">...
  </div>
</li>
  <li>
    <a href="#none">top global news</a>
    <div class="sub-menu">...
  </div>
</li>
  <li>
    <a href="#none">business</a>
    <div class="sub-menu">...
  </div>
</li>
</ul>
```

HTML

```
/* Custom CSS */
.menu {
  list-style: none; padding: 0; width: 250px;
}
.menu li > a,
.sub-menu a {
  text-decoration: none;
  color: #eee;
  text-transform: capitalize;
  padding: 5px 10px;
}
.menu li > a {
  display: block;
  background-color: #222;
  border-bottom: 1px solid #555;
}
.menu li > a:after {
  content: '+'; float: right;
}
.sub-menu { display: none; }
.sub-menu a {
  display: block; background-color: #333;
}
.sub-menu a:hover { background-color: #222; }
```

CSS

CSS 대신에 제이쿼리로 하려면
아래 코드를 JS에 사용합니다.
\$('.sub-menu').hide();



```
// 최초에 오픈할 서브메뉴
$('.sub-menu').eq(0).show(); → 필요한 경우 서브 메뉴 열어 놓기

// 클릭 이벤트로 보이기 감추기
$('.menu li > a').click(function(){
  $(this).next().slideDown();
  $(this).parent().siblings().children('.sub-menu').slideUp();
});
```

JS

이미 펼쳐진 서브메뉴를 닫고 새로운 서브메뉴를 열고 싶다면 .menu li > a 중에 클릭된 것(this)를 기준으로 부모 요소(li)의 형제 요소(li)의 자식요소 중에 .sub-menu를 슬라이드업 시켜서 사라지게 함

05) 텍스트 관련 메서드 - text() html() val()

{ `$('#선택자').text()` }

▲ 선택자에 텍스트 내용 가져오기
선택자 하위에 있는 자식 태그들의 문자열만 출력

```
<div id="content">
  <span>Hello~ I'm CodingWork.</span>
</div>
```

```
var print = $('#content').text();
alert(print);
```

127.0.0.1:5500 내용:

Hello~ I'm CodingWork.

확인

{ `$('#선택자').html()` }

▲ 선택자에 html 태그와 텍스트 내용 가져오기
선택자 하위에 있는 자식 태그들에 대해서 태그나 문자열 등에
상관하지 않고 전부 가져옴

```
<div id="content">
  <span>Hello~ I'm CodingWork.</span>
</div>
```

```
var print = $('#content').html();
alert(print);
```

127.0.0.1:5500 내용:

Hello~ I'm CodingWork.

확인

{ `$('#선택자').val()` }

▲ 선택자에 밸류값(value) 가져오기
양식(form)의 값을 가져오거나 값을 설정

```
<input type="text" id="userId" value="codingwork" />
```

```
var print = $('#userId').val();
alert(print);
```

127.0.0.1:5500 내용:

codingwork

확인

05) 텍스트 관련 메서드 - 스몰미션

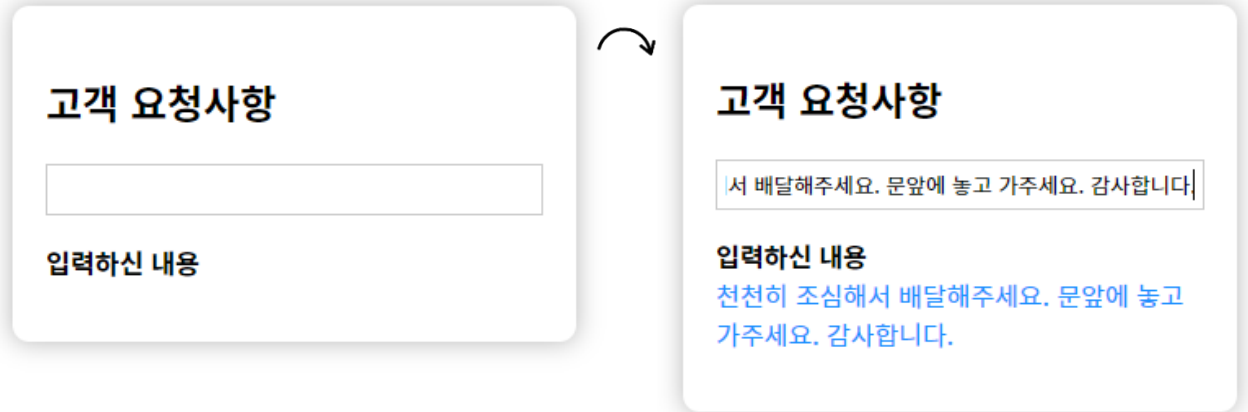
Small Mission) 제이쿼리 메서드를 사용해서 input에 입력한 내용을 확인 차원에서 텍스트로 출력하기

```
<div class="frame">
  <h2>고객 요청사항</h2>
  <input id="userInput" type="text">
  <p>
    <b>입력하신 내용</b> <span id="print"></span>
  </p>
</div>
```

```
/* Custom CSS */
.frame {
  box-shadow: 0 0 15px rgba(0, 0, 0, 0.3);
  padding: 20px;
  width: 300px;
  border-radius: 10px;
}
#userInput {
  border: 1px solid #ccc;
  padding: 5px;
  outline: none;
  width: 100%;
  box-sizing: border-box;
}
#print {
  display: block;
  color: dodgerblue;
}
```

```
$('#userInput').keydown(function(){
  var userInputValue = $('#userInput').val();
  $('#print').text(userInputValue)
});
```

- keydown() - 키 입력 시 발생하는 이벤트
- keypress() - keydown과 같이 키 입력 시 발생하는 이벤트이지만 Enter, Tab 등의 특수키에는 발생하지 않음
- keyup() - 키 입력 후 발생하는 이벤트



→ input에 유저가 입력하는 값(value)를 변수에 담습니다.

→ 담긴 변수를 text() 메서드로 #print라는 곳에 텍스트로 출력

06) 속성 관련 메서드 - attr() prop()

- html에 쓴 속성 값을 사용할 경우 .attr() 사용하고, 그 외에 javascript의 프로퍼티 값을 사용할 경우는 .prop() 사용
- jQuery 1.6 이후부터 .attr() 함수가 용도에 따라 .attr() 과 .prop() 으로 분리

```
{ $('선택자').attr(속성이름) }
```

▲ 선택자에 속성값 가져오기(getter)

```
{ $('선택자').attr(속성이름, 속성값) }
```

▲ 속성값 설정하기 - 생성 변경(setter)

```
{ $('선택자').removeAttr(속성이름) }
```

▲ 선택자에 속성값 제거하기

[2개 이상의 속성을 설정할 경우]

```
<a class="link">구글 바로가기</a>
```

```
// 속성 설정하기(추가 변경)
$('.link').attr('href', 'http://www.infllearn.com').attr('title', '인프런 바로가기');
```

```
// 속성 설정하기(추가 변경)
$('.link').attr({
  'href': 'http://www.infllearn.com',
  'title': '인프런 바로가기'
});
```

→ 2개 이상의 속성을 설정하는 경우 css() 메서드와 동일하게 객체 형태로 사용할 수 있습니다.

```
<a class="testLink1" href="http://www.infllearn.com">인프런 바로가기</a>
<div class="print"></div>
<hr>
<a class="testLink2">구글 바로가기</a>
```

인프런 바로가기

구글 바로가기

```
// 속성 가져오기
var getAttr = $('.testLink1').attr('href');
$('.print').text(getAttr);

// 속성 설정하기(추가)
$('.testLink2').attr('href', 'http://www.google.com');
```

인프런 바로가기
http://www.infllearn.com

구글 바로가기

구글 바로가기
인프런 바로가기

```
<a class="testLink2">구글 바로가기</a>
```

▲ .attr()로 href 속성을 설정하기 전

```
<a class="testLink2" href="http://www.google.com">구글 바로가기</a>
```

▲ .attr()로 href 속성을 설정한 후

06) 속성 관련 메서드 - attr() prop()

```
{ $('선택자').prop(속성이름) }
```

▲ 선택자에 속성값 가져오기(getter)

```
{ $('선택자').prop(속성이름, 속성값) }
```

▲ 속성값 설정하기 - 생성 변경(setter)

```
{ $('선택자').removeProp(속성이름) }
```

▲ 선택자에 속성값 제거하기

```
<a class="link" href="index.html" title="구글바로가기">Google</a>
```

```
// attr() 속성 설정하기(추가 변경)
var linkAttr = $('link').attr('href');
console.log(linkAttr)
```

index.html

```
// prop() 속성 설정하기(추가 변경)
var linkProp = $('link').prop('href');
console.log(linkProp)
```

```
http://127.0.0.1:5500/[%EC%A0%9C%EC%9D%B4%EC%BF%BC%EB%A6%AC]%2002.%20%ED%95%84%EC%88%98%20%EB%A9%94%EC%84%9C%EB%93%9C/index.html
```

Small Mission) 체크박스 개별 선택/해제, 모두 선택/해제

☐ 약관에 전체 동의☐ [필수] PASS 서비스☐ 개인정보 수입 및 이용 동의☐ 개인정보 취급 위탁 동의☒ 약관에 전체 동의☒ [필수] PASS 서비스 이용약관☒ 개인정보 수입 및 이용 동의☒ 개인정보 취급 위탁 동의

```
.agreement {
  width: 375px;
  box-shadow: 0 0 15px rgba(0, 0, 0, 0.3);
  padding: 25px; border-radius: 10px;
}
.agreement label {
  display: block; margin: 5px 0;
}
.agreement label:first-child {
  border-bottom: 1px solid #ccc;
  padding-bottom: 7px; font-weight: 500;
}
```

CSS

<form class="agreement">

<label>

<input type="checkbox" name="chk-all" class="chk-all">약관에 전체 동의

</label>

<label>

<input type="checkbox" name="chk1" class="chk">[필수] PASS 서비스 이용약관

</label>

<label>

<input type="checkbox" name="chk2" class="chk">개인정보 수입 및 이용 동의

</label>

<label>

<input type="checkbox" name="chk3" class="chk">

</label>

</form>

```
$('.chk-all').click(function(){
  $('.chk').prop('checked', this.checked);
});
```

JS

[미션] chk라는 클래스 네임을 사용하지 않고 좀 더 깨끗한 html을 유지하면서 같은 기능으로 만들기

06) 속성 관련 메서드 - 스몰미션(1)

Small Mission) 제이쿼리 속성 관련 메서드를 사용해서 심플한 이미지 갤러리 만들기 #01(수동으로 만들기)

```
<div class="image-gallery">
  <h1>Simple Image Gallery with attr()</h1>
  <div class="buttons">
    <button class="btn1 active">Zoo Image #01</button>
    <button class="btn2">Zoo Image #02</button>
    <button class="btn3">Zoo Image #03</button>
  </div>
  <div class="image">
    
  </div>
</div>
```

HTML

```
/* Custom CSS */
.image-gallery { width: 600px; }
.buttons {
  margin-bottom: 20px;
  display: flex;
  justify-content: space-between;
  gap: 20px;
}
```

CSS

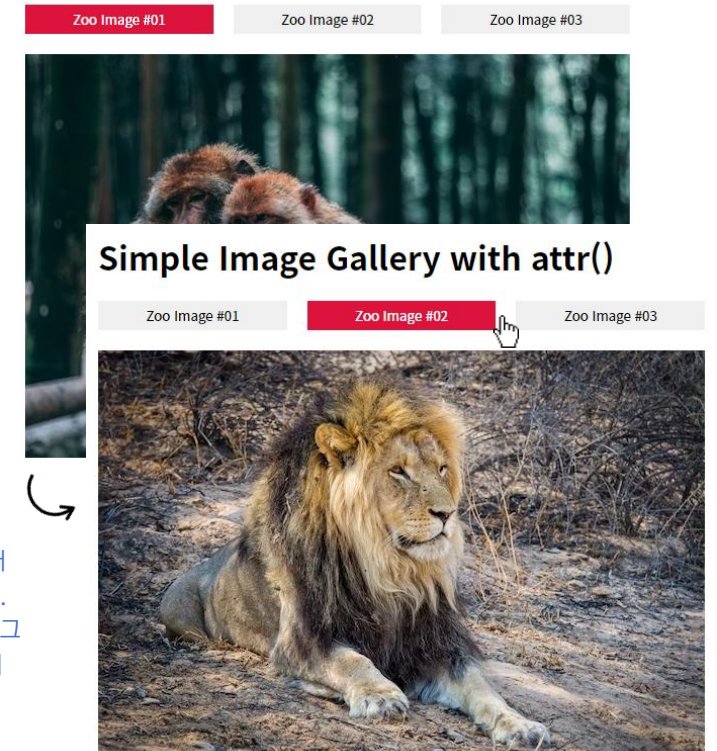
```
.buttons button {
  flex: 1; border: none;
  padding: 5px; cursor: pointer;
}
.buttons button.active {
  background-color: crimson;
  color: #fff;
}
```

```
// 속성 설정하기(setter)
$('.btn1').click(function(){
  $('.image img').attr('src', 'images/zoo-01.jpg');
});
$('.btn2').click(function(){
  $('.image img').attr('src', 'images/zoo-02.jpg');
});
$('.btn3').click(function(){
  $('.image img').attr('src', 'images/zoo-03.jpg');
});

// 버튼 클래스 제어하기
$('.buttons button').click(function(){
  $(this).addClass('active');
  $(this).siblings().removeClass('active');
});
```

JS

Simple Image Gallery with attr()



기능은 되지만 반복적인 코드가 많습니다. 예를 들어 4번째 이미지를 넣으려면 제이쿼리 구분에서 또 수정해야 합니다. 이런 형태는 수동으로 만들어지는 부분이라 효율적인 프로그램이라고 할 수 없습니다. 해당 예제에서는 attr() 메서드에 대한 이해만 가지면 됩니다.

다음 페이지에 이미지 갤러리 자동으로 만들기에서 제이쿼리 코드를 수정하지 않고 html만 추가하는 방식으로 만듭니다.

06) 속성 관련 메서드 - 스몰미션(2)

Small Mission) 제이쿼리 속성 관련 메서드를 사용해서 심플한 이미지 갤러리 만들기 #02(자동으로 만들기)

```
<div class="image-gallery">
  <h1>Simple Image Gallery with attr()</h1>
  <div class="buttons">
    <button class="active" data-image="images/zoo-01.jpg">Zoo Image #01</button>
    <button data-image="images/zoo-02.jpg">Zoo Image #02</button>
    <button data-image="images/zoo-03.jpg">Zoo Image #03</button>
  </div>
  <div class="image">
    
  </div>
</div>
```

HTML

data-image라는 사용자 정의 속성을 만들고 속성값으로 이미지 경로를 넣습니다. 제이쿼리에서 .attr() 메서드로 속성과 값을 가져와서 버튼을 누를 때마다 설정합니다.

```
/* Custom CSS */
.image-gallery { width: 600px; }
.buttons {
  margin-bottom: 20px;
  display: flex;
  justify-content: space-between;
  gap: 20px;
}
```

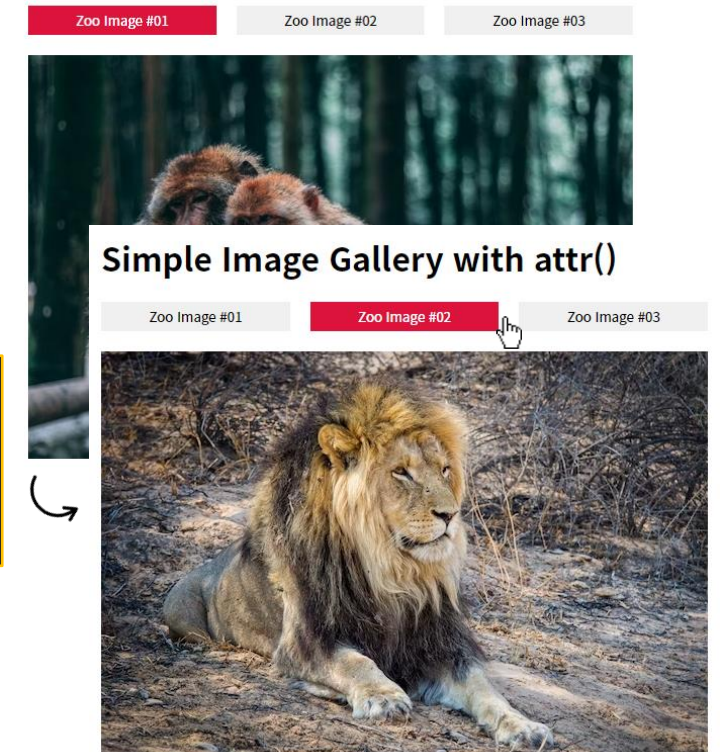
CSS

```
.buttons button {
  flex: 1; border: none;
  padding: 5px; cursor: pointer;
}
.buttons button.active {
  background-color: crimson;
  color: #fff;
}
```

```
// 속성 설정하기(setter)
$('.buttons button').click(function(){
  var imageSrc = $(this).attr('data-image');
  $('.image img').attr('src', imageSrc);
})
```

- 버튼이 가지고 있는 data-image 속성의 값을 가지고 와서 변수에 담기
- 변수를 src 속성의 값으로 설정하기

Simple Image Gallery with attr()



06) 속성 관련 메서드 - 스몰미션(3)

```
<div class="simple-tab-content">
  <div class="buttons">
    <button class="active" data-alt="tab1">Tab #01</button>
    <button data-alt="tab2">Tab #02</button>
    <button data-alt="tab3">Tab #03</button>
  </div>
  <div class="tabs">
    <div class="tab-content active" id="tab1">
      <h2>Simple Tab Header #01</h2>
      <p>
        Lorem ipsum dolor sit amet conseq
        elit. Illo dicta laudantium venia
        quaerat, odit animi, corporis des
        reprehenderit pariatur amet libero
        asperiores repellendus minima ius
      </p>
    </div>
    <div class="tab-content" id="tab2">...
    </div>
    <div class="tab-content" id="tab3">...
    </div>
  </div>
</div>
```

HTML

```
/* Custom CSS */
.simple-tab-content {
  width: 500px;
  border: 1px solid #ccc;
  padding: 20px;
}
.buttons {
  display: flex; gap: 20px;
}
.buttons button {
  flex: 1; border: none;
  padding: 5px; cursor: pointer;
}
.buttons button.active {
  background-color: #000; color: #fff;
}
.tab-content {
  display: none;
}
.tab-content.active {
  display: block;
}
```

CSS

Small Mission) 제이쿼리 속성 관련 메서드를 사용해 심플한 탭 메뉴 콘텐츠 만들기

```
<script>
$(function(){
  $('.buttons button').click(function(){
    // Button Active Toggle
    $(this).addClass('active').siblings().removeClass('active')

    // Tab Active Toggle
    let attrValue = $(this).attr('data-alt')
    $('.tab-content').removeClass('active')
    $('#'+attrValue).addClass('active')
  })
})
```

→ 변수에 담아서 편하게 사용하기

- # + tab1이 되어서 결국 #tab1이 됩니다.
- 클래스를 사용했을 경우 # 대신에 점(.)이 들어가면 됩니다.

`$(function(){..}` 대신에 아래 자바스크립트 문서 로딩을 사용해도 무방합니다.

`window.onload = function(){..}`

`document.addEventListener('DOMContentLoaded', function(){..})`



07) 동적으로 html 요소 삽입/제거(#01) - append() prepend() after() before()

```
{ $('선택자').append(html요소) }
```

▲ 선택 요소 안에서 자식 요소로 맨 뒤에 내용 삽입
좌우가 바뀐 경우 : \$('html요소').appendTo('선택자')

```
{ $('선택자').prepend(html요소) }
```

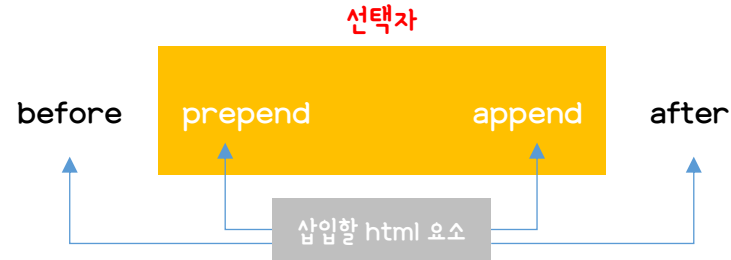
▲ 선택 요소 안에서 자식 요소로 맨 앞에 내용 삽입
좌우가 바뀐 경우 : \$('html요소').prependTo('선택자')

```
{ $('선택자').after(html요소) }
```

▲ 선택 요소 밖에서 선택 요소 바로 뒤에 내용 삽입
좌우가 바뀐 경우 : \$('html요소').insertAfter('선택자')

```
{ $('선택자').before(html요소) }
```

▲ 선택 요소 밖에서 선택 요소 바로 앞에 내용 삽입
좌우가 바뀐 경우 : \$('html요소').insertBefore('선택자')



```
<ol id="items">
  <li>기존 첫번째 아이템</li>
  <li>기존 두번째 아이템</li>
</ol>
```

1. 기존 첫번째 아이템
2. 기존 두번째 아이템

```
// append()
$('#items').append('<li>append()로 추가된 아이템</li>');

// prepend()
$('#items').prepend('<li>prepend()로 추가된 아이템</li>');
```

1. prepend()로 추가된 아이템
2. 기존 첫번째 아이템
3. 기존 두번째 아이템
4. append()로 추가된 아이템

after() before() 후 결과는 선택 요소 밖에 만들어 집니다.

```
<ol id="items">
  <li>::marker
    "prepend()로 추가된 아이템"
  </li>
  <li>...</li>
  <li>...</li>
  <li>::marker
    "append()로 추가된 아이템"
  </li>
</ol>
```

▲ append() prepend() 후 개발자 도구로 html 구조를 보면 .items 안에서 자식 요소로 맨 처음에 삽입 / .items 안에서 자식 요소로 맨 뒤에 삽입

- prepend()로 추가된 아이템

1. prepend()로 추가된 아이템
2. 기존 첫번째 아이템
3. 기존 두번째 아이템
4. append()로 추가된 아이템

- prepend()로 추가된 아이템

07) 동적으로 html 요소 삽입/제거(#02) - remove() empty()

```
{ $('선택자').remove() } { $('선택자').empty() }
```

▲ 현재 선택 요소 자체를 삭제

▲ 현재 선택 요소의 하위 자식들을 모두 삭제

```
<div class="target">
  <h2>This is headline.</h2>
  <p>
    Lorem ipsum dolor sit, amet consectetur adipisicing elit. Fuga accusamus ab
    voluptate dolor veritatis perferendis!
  </p>
</div>
```

This is headline.

Lorem ipsum dolor sit, amet consectetur adipisicing elit. Fuga accusamus ab voluptate dolor veritatis perferendis!

```
// append()
$('.target').remove();
```

▲ .target 자체가 사라져서 아예 보이지 않음

```
// prepend()
$('.target').empty();
```

▲ .target 자체가 사라지지 않고 .target 내의 h2, p 요소가 사라짐

.target 내의 자식 요소가 모두
비어 있음(empty)

```
<!DOCTYPE html> == $0
<html lang="ko">
  <head> ... </head>
  <body cz-shortcut-listen="true">
    <div class="target"></div>
    <script> ... </script>
    <!-- Code injected by live-server -->
    <script> ... </script>
  </body>
</html>
```

07) 동적으로 html 요소 삽입/제거 - 스몰미션

Small Mission) 제이쿼리 메서드를 사용해서 이전 다음 번호 나오는 이미지 갤러리 만들기(1)

```
<div id="gallery-wrap">
  <div class="page">
    <span class="page-num">
      <span></span> / <span></span>
    </span>
    <button class="prev">&lt;</button>
    <button class="next">&gt;</button>
  </div>
  <div class="content">
    <div></div>
    <div></div>
    <div></div>
  </div>
</div>
```

html 특수기호 표현을 사용하지 않고
그냥 < > 를 사용해도 상관없음.img를 꼭 div로 묶지 않아도 상관 없
지만 div를 사용하면 나중에 이미지 위
에 텍스트 또는 링크를 넣기 좋음.

```
/* Custom CSS */
#gallery-wrap {
  width: 600px; margin: 20px auto;
}
#gallery-wrap .page {
  text-align: right; margin-bottom: 5px;
}
.page-num {
  display: inline-block; margin-right: 10px;
}
```

CSS

```
.page button {
  border: none;
  background: #666;
  color: #fff;
  cursor: pointer;
}
.content {
  width: 600px;
  height: 400px;
  border: 2px solid #505900;
  overflow: hidden;
```

overflow: hidden이 없을 경우 이미
지는 개수만큼 아래로 길게 배치됨

→ 이전 다음 버튼을 클릭하면 페이지 번호가 바뀌면서 이미지 순서도 변경됨.

07) 동적으로 html 요소 삽입/제거 - 스몰미션

Small Mission) 제이쿼리 메서드를 사용해서 이전 다음 번호 나오는 이미지 갤러리 만들기(2)

```
// 이미지 콘텐츠 개수
var totalNum = $('.content div').length;
var currentNum = 1;
// 초기화를 위한 변수
// 페이지 번호 텍스트 넣기
$('.page-num span:first').text(currentNum);
$('.page-num span:last').text(totalNum);
```

html에 이미지 개수가 3개
이므로 3이 됩니다.

▲ 위의 초기화 변수에 의해 현재 currentNum은 1이고 전체 번호는 html에 이미지 개수가 3개이므로 전체 개수는 3이 됩니다.
html 내에 div img를 추가하면 자동으로 전체 개수가 변경됨.

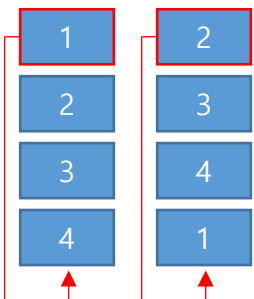
```
// 다음 버튼을 누를 경우
$('.next').click(function(){
    currentNum++;
    if(currentNum > totalNum) {
        currentNum = 1
    }
    // A.append(B) B를 A 안쪽 뒤에 추가 → B.appendTo(A)
    // A.insertAfter(B) A를 B 뒤에 추가
    // A.after(B) B를 A 바깥 뒤에 추가
    $('.content > div:first').appendTo('.content');
    $('.page-num span:first-child').text(currentNum);
});
```

- ① 다음 버튼을 누르면 현재 번호를 1씩 증가시킴
- ② [번호 변경 기능] 만약에 현재 번호가 전체 번호보다 클 경우 현재 번호를 1로 초기화함
- ③ [이미지 변경 기능] div 중에 첫번째 것을 맨 뒤로 삽입해서 대기시킴
- ④ [번호 변경 기능] 현재 번호를 현재 번호 출력하는 곳에 text로 입력

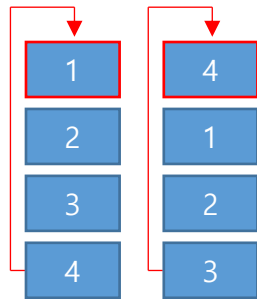
```
// 이전 버튼을 누를 경우
$('.prev').click(function(){
    currentNum--;
    if(currentNum < 1) {
        currentNum = totalNum
    }
    // A.prepend(B) B를 A 안쪽 앞에 추가 → B.prependTo(A)
    // A.insertBefore(B) A를 B 앞에 추가
    // A.before(B) B를 A 바깥 앞에 추가
    $('.content > div:last').prependTo('.content');
    $('.page-num span:first').text(currentNum);
});
```

- ① 이전 버튼을 누르면 현재 번호를 1씩 감소시킴
- ② [번호 변경 기능] 만약에 현재 번호가 1보다 작을 경우 현재 번호를 전체 번호로 초기화함
- ③ [이미지 변경 기능] div 중에 마지막 것을 맨 앞으로 삽입해서 대기시킴
- ④ [번호 변경 기능] 현재 번호를 현재 번호 출력하는 곳에 text로 입력

① 다음 버튼 누를 경우



② 이전 버튼 누를 경우



08) width, height 관련 메서드 - width() height()

{ 너비(width) 관련 }

\$('.선택자').width() : 내부 너비

\$('.선택자').innerWidth() : 내부 너비 + padding

\$('.선택자').outerWidth() : 내부 너비 + padding + border

{ 높이(height) 관련 }

\$('.선택자').height() : 내부 높이

\$('.선택자').innerHeight() : 내부 높이 + padding

\$('.선택자').outerHeight() : 내부 높이 + padding + border

```
<div class="box"></div>

<div class="calc">
  <ul>
    <li>width : <span class="width"></span></li>
    <li>height : <span class="height"></span></li>
    <li>innerWidth : <span class="innerWidth"></span></li>
    <li>innerHeight : <span class="innerHeight"></span></li>
    <li>outerWidth : <span class="outerWidth"></span></li>
    <li>outerHeight : <span class="outerHeight"></span></li>
  </ul>
</div>
```

```
/* Custom CSS */
.calc ul {
  margin-left: 15px;
}
.calc ul li span {
  color: royalblue;
}
.box {
  border: 10px solid crimson;
  padding: 20px;
  margin: 30px;
  background-color: #ddd;
}
```

```
// 너비 높이 설정하기
$('.box').width(200).height(150);

// 너비 높이 가져오기
$('.width').text($('.box').width());
$('.height').text($('.box').height());
$('.innerWidth').text($('.box').innerWidth());
$('.innerHeight').text($('.box').innerHeight());
$('.outerWidth').text($('.box').outerWidth());
$('.outerHeight').text($('.box').outerHeight());
```

JS

- width :
- height :
- innerWidth :
- innerHeight :
- outerWidth :
- outerHeight :

- width : 200
- height : 150
- innerWidth : 240
- innerHeight : 190
- outerWidth : 260
- outerHeight : 210

09) 위치 관련 메서드 - offset() position()

html 기준으로 left, top 값을 가져오기, 변경하기

{ offset() }

\$('선택자').offset().left

\$('선택자').offset().top

\$('선택자').offset({left: 10, top: 10})

```

<div class="parent">
  <p class="child">
    Lorem ipsum, dolor sit amet consectetur adip
  </p>
</div>

<div class="frame">
  <button class="offset">offset()</button>
  <button class="position">position()</button>
  <span class="print"></span>
</div>

```

```

/* Custom CSS */
.parent {
  border: 5px solid #000; margin-bottom: 20px;
  position: relative; width: 400px; height: 200px;
}
.child {
  border: 5px solid red; padding: 10px;
  position: absolute; top: 0; left: 0; margin: 0;
}

```

```

// offset() 가져오기
$('.offset').click(function(){
  var child = $('.child').offset();
  $('.print').text('top: ' + child.top + 'px, left: ' + child.left + 'px');
});

// position() 가져오기
$('.position').click(function(){
  var child = $('.child').position();
  $('.print').text('top: ' + child.top + 'px, left: ' + child.left + 'px');
});

```

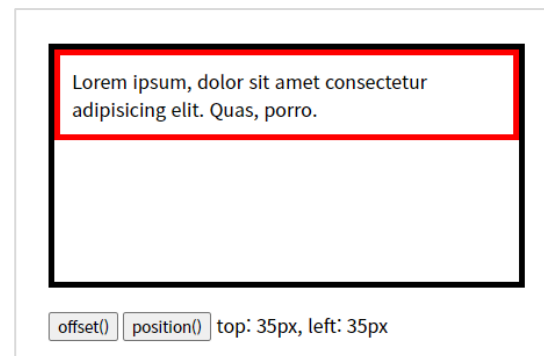
JS

부모요소 기준으로 left, top 값을 가져오기, 변경하기

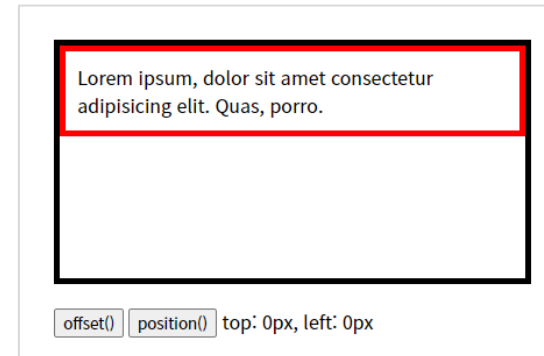
{ position }

\$('선택자').position().left

\$('선택자').position().top



▲ html 기준으로 p 태그의 위치 찾는 offset()



▲ 부모 요소 기준으로 p 태그의 위치 찾는 position()

10) 요소의 스크롤 위치 메서드 - scrollTop() scrollLeft()

```
{   $('선택자').scrollTop() }
```

▲ 요소의 상단 위치를 가져오기/설정하기

```
$('선택자').scrollTop() → 가져오기
```

```
$('선택자').scrollTop(100) → 100으로 설정
```

```
{   $('선택자').scrollLeft() }
```

▲ 요소의 왼쪽 위치를 가져오기/설정하기

```
$('선택자').scrollLeft() → 가져오기
```

```
$('선택자').scrollLeft(100) → 100으로 설정
```

Small Mission) 스크롤 바의 위치 정보를 실시간 구해서 문서에 출력하기

```
<div class="wrap">  
  <h3>현재 스크롤 위치</h3>  
  <span class="print"></span>  
</div>
```

```
/* Custom CSS */  
body {  
  height: 200vh; width: 200vw;  
}  
.wrap {  
  position: fixed;  
  border: 1px solid #000;  
  padding: 20px;  
}  
.wrap h3 { margin-top: 0; }
```

스크롤 수치를 보기 위해서 body의 너비와 높이로 강제로 만들

```
// 스크롤 바의 위치를 반환  
$(window).scroll(function(){  
  var left = $(this).scrollLeft();  
  var top = $(this).scrollTop();  
  $('.print').text('Left: ' + left + ', Top: ' + top);  
})
```

- \$(window)는 브라우저 객체를 의미함.
- .scroll() 이벤트 메서드는 마우스 스크롤을 의미
- 종합하면.. 브라우저를 스크롤 했을 때 함수(function)을 실행함

현재 스크롤 위치

현재 스크롤 위치

Left: 472, Top: 382

10) 요소의 스크롤 위치 메서드 - 스몰미션(offset과 scrollTop을 이용해 스크롤 후 요소의 위치 고정)

```
<div class="container">  
  <!-- Banner -->  
  <div class="top-banner" role="banner">Banner Content</div>  
  <!-- Header -->  
  <header>  
    <div class="header-inner">  
      <div class="logo">CodingWorks</div>  
      <div class="gnb" role="navigation">  
        <a href="#none">home</a>  
        <a href="#none">company</a>  
        <a href="#none">product</a>  
        <a href="#none">contact</a>  
      </div>  
    </div>  
  </header>  
</div>
```

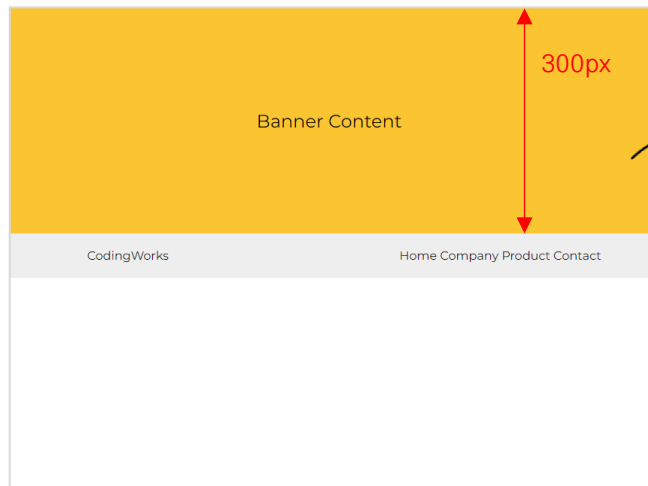
HTML

```
body {  
  margin: 0;  
  height: 2000px;  
}
```

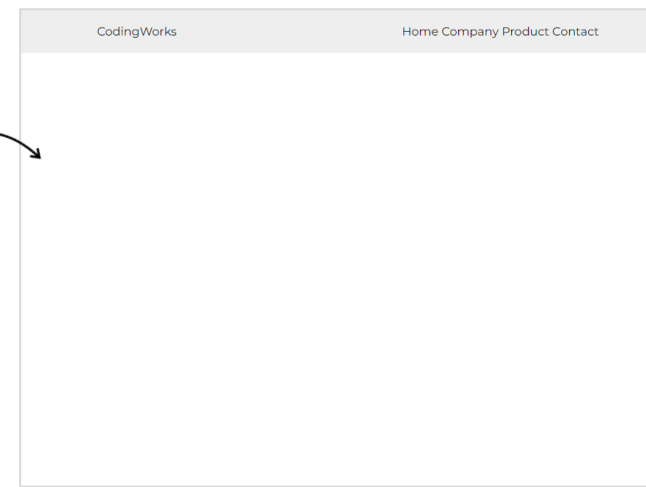
```
header {  
  display: flex;  
  justify-content: center;  
  padding: 20px;  
  width: 100%;  
  background-color: #eee;  
}
```

```
.top-banner {  
  height: 300px;  
  background-color: #fbc531;  
  font-size: 1.5em;  
  display: flex;  
  justify-content: center;  
  align-items: center;  
}  
/* Sticky Class */  
header.sticky {  
  position: fixed;  
  top: 0;  
  left: 0;  
}
```

CSS



▲브라우저 스크롤 전



▲브라우저 스크롤 후

```
var headerOffsetTop = $('header').offset().top;  
console.log(headerOffsetTop)
```

300

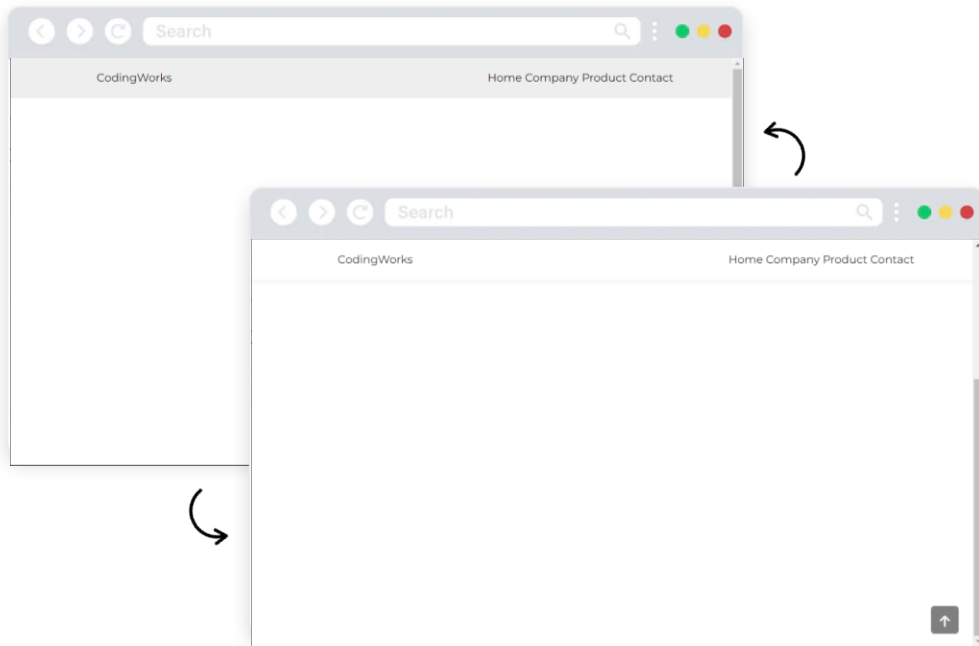
- 브라우저를 스크롤 했을 때 header가 상단에서 떨어진 값 보다 크다면 header에 클래스를 넣고 아니면 클래스를 뺀다.
- 배너의 높이가 300픽셀이기 때문에 headerOffsetTop을 300 이라고 넣어줘도 동일함. 하지만 배너의 높이가 유동적일 수 있으므로 예제 처럼 해주는 것 좋음.(배너의 높이가 변동될 수도 있음)

```
var headerOffsetTop = $('header').offset().top;  
  
$(window).scroll(function(){  
  if($(this).scrollTop() > headerOffsetTop) {  
    $('header').addClass('sticky');  
  }  
  else {  
    $('header').removeClass('sticky');  
  }  
})
```

10) 요소의 스크롤 위치 메서드 - 스몰미션 ↪ CSS 디자인을 상세하게 하는 것은 중요하지 않습니다. JS 기능이 중요합니다. 단, CSS에서 기능과 관련된 CSS는 필수적으로 넣어야 합니다.

Small Mission)

스크롤을 내리면 선택한 요소의 디자인을 변경하고
다시 맨 위로 올리면 원래 상태로 돌아가기



- 스크롤을 아래로 50픽셀 만큼 내렸을 때 header 디자인, Go to top 디자인 변경
- 다시 스크롤을 가장 상단으로 올리면 적용된 header 디자인, Go to top 디자인 원래로 돌아옴

```

<!-- Header -->
<header>
  <div class="header-inner">
    <div class="logo">CodingWorks</div>
    <div class="gnb">
      <a href="#none">home</a>
      <a href="#none">company</a>
      <a href="#none">product</a>
      <a href="#none">contact</a>
    </div>
  </div>
</header>

<!-- Go To Top -->
<a class="btn-top" href="#">
  <i class="bi bi-arrow-up-short">
</a>

/* Go To Top */
.btn-top {
  width: 40px;
  height: 40px;
  position: fixed;
  font-size: 2em;
  color: #fff !important;
  right: 20px;
  bottom: -50px;
  background-color: rgba(0, 0, 0, 0.5);
  border-radius: 5px;
  text-align: center;
  line-height: 45px;
  transition: 0.5s;
}

/* Active Class */
header.active {
  background-color: #fff;
  box-shadow: 0 3px 5px rgba(0, 0, 0, 0.5);
}

.btn-top.active {
  bottom: 20px;
}

/* Custom CSS */
body {
  margin: 0;
  height: 150vh;
}
header {
  display: flex;
  justify-content: center;
  padding: 20px;
  position: fixed;
  width: 100%;
  background-color: #eee;
  transition: 0.5s;
}
header-inner {
  width: 80%;
  display: flex;
  justify-content: space-between;
}
gnb a {
  text-transform: capitalize;
}

```

```

/* Header Scroll Change */
$(window).scroll(function(){
  if($(window).scrollTop() > 50) {
    $('header, .btn-top').addClass('active')
  }
  else {
    $('header, .btn-top').removeClass('active')
  }
})

```

- \$(window)는 브라우저 객체를 의미함.
- .scroll() 이벤트 메서드는 마우스 스크롤을 의미
- 브라우저를 스크롤 했을 때 함수(function)를 실행함
- 스크롤의 양이 50픽셀 보다 크다면 곧, 조금이라도 스크롤을 아래로 했다면 클래스를 추가하고, 그렇지 않으면 곧, 0이면 추가된 클래스를 제거함.
- 50px라고 적으면 작동하지 않습니다.

11) 복사 및 감싸기 관련 메서드 - clone() wrap() wrapAll() wrapInner() replaceWith()

중요도가 높지 않은 메서드입니다. 한번쯤 해보는 정도로만 연습해보세요.

{ `$('#선택자').clone()` }

▲ 요소 복사

{ `$('#선택자').wrap(내용)` }

▲ 요소 감싸기

{ `$('#선택자').wrapAll(내용)` }

▲ 요소 모두 감싸기

{ `$('#선택자').wrapInner(내용)` }

▲ 요소의 하위 요소로 감싸기

{ `$('#선택자').replaceWith(내용)` }

▲ 요소를 다른 요소로 교체하기

```
<h1 class="test1">요소 복사</h1>
<span class="test2">요소 감싸기</span>

<span class="test3">요소 모두 한번에 감싸기</span>
<span class="test3">요소 모두 한번에 감싸기</span>
<span class="test3">요소 모두 한번에 감싸기</span>

<div class="test4">Lorem ipsum dolor sit amet.</div>

<h1 class="heading">This is H1</h1>
```

HTML

```
// 요소 복사
$('#.test1').clone().insertAfter('.test1');

// 요소 감싸기
$('#.test2').wrap('<div class="frame"></div>');

// 요소 모두 감싸기
$('#.test3').wrapAll('<p></p>');

// 요소의 하위 요소로 감싸기
$('#.test4').wrapInner('<p></p>');

// 요소를 다른 요소로 교체하기
$('#.heading').replaceWith('<h2>This is H2</h2>');
```

JS

<h1 class="test1">요소 복사</h1>
<h1 class="test1">요소 복사</h1>

<div class="frame">
 요소 감싸기
</div>

<p>
 요소 모두 한번에 감싸기
 요소 모두 한번에 감싸기
 요소 모두 한번에 감싸기
</p>

<div class="test4">
 <p>Lorem ipsum dolor sit amet.</p>
</div>

<h2>This is H2</h2>

12) 특정요소가 선택요소의 현재의 상태 확인 후 참(true) 거짓(false) 반환 - is()

중요도가 의외로 높은 메서드입니다. 꼭 연습해보세요.

```
{ $('선택요소').is('비교할 요소 또는 표현식') }
```

▲ 요소를 비교해서 참 거짓 비교

```
<div class="parent">
  <p>
    Lorem ipsum dolor sit amet consectetur adipisicing
    elit. Tempora, blanditiis!
  </p>
  <span class="sample">Lorem ipsum dolor sit amet.</span>
</div>
```

```
// 특정요소에 자식요소로 p 태그 있는지 확인
var hasCheck1 = $('parent').children().is('p');
var hasCheck2 = $('parent').children().is('span');
console.log(hasCheck1); → true
console.log(hasCheck2); → false
```

Small Mission) 특정요소에 자식요소로 p 태그 있는지 확인

```
// 특정요소에 자식요소로 p태그 있는지 확인
var hasCheck = $('parent').children().is('p');

if(hasCheck) {
  alert('자식요소로 p태그 있음');
}
else {
  alert('자식요소로 p태그 없음');
}
```

127.0.0.1:5500 내용:
자식요소로 p태그 있음

확인

```
// 특정요소에 클래스명이 있는지 확인
var hasClass = $('parent').children().is('.sample');

if(hasClass) {
  alert('자식요소로 .sample 클래스 있음');
}
else {
  alert('자식요소로 .sample 클래스 없음');
}
```

127.0.0.1:5500 내용:
자식요소로 .sample 클래스 있음

확인

13) 일치하는 요소의 일부분만 선택하는 메서드 - slice() ↪ 중요도가 의외로 높은 메서드입니다. 꼭 연습해보세요.

{ `$('.선택요소').slice(시작번호, 종료번호)` }

▲ 요소의 시작 인덱스 번호와 종료 인덱스 번호로 일부분만 선택
종료 인덱스 번호가 없으면 끝까지 자동으로 선택

{ `$('.선택요소:hidden')` }

▲ :hidden 선택자는 해당 요소를 보이지 않게 하는 선택자로 CSS에서 display: none
인 요소를 선택하는 선택자

```
/* Custom CSS */  
.list li { display: none; }
```

▲ CSS에서 display: none을 해서 li를 모두 안보이게 하기

```
// :hidden 선택자로 숨긴 요소 모두 보이기  
$('.list li:hidden').show(); ①
```

```
// :hidden 선택자로 숨긴 요소 중 일부분만 보이기  
$('.list li:hidden').slice(2, 5).show(); ②
```

1. list item #01
2. list item #02
3. list item #03
4. list item #04
5. list item #05

1. list item #03
2. list item #04
3. list item #05

① 숨겨진 모든 li 보이기

② 숨겨진 li 중에서 2번째 부터 5번째까지 보이기

Small Mission) 목록에서 특정 li를 slice() 메서드로 선택 후 텍스트 색상을 변경하기

```
<ol class="list">  
  <li>list item #01</li>  
  <li>list item #02</li>  
  <li>list item #03</li>  
  <li>list item #04</li>  
  <li>list item #05</li>  
</ol>
```

HTML

```
// 1번 li에서 2번 li까지 선택  
$('.list li').slice(0, 2).css('color', 'crimson');
```



1. list item #01
2. list item #02
3. list item #03
4. list item #04
5. list item #05

```
// 2번 li부터 모두 선택  
$('.list li').slice(2).css('color', 'crimson');
```



1. list item #01
2. list item #02
3. list item #03
4. list item #04
5. list item #05

이벤트 종류(1) - 마우스 관련 이벤트, 키보드 관련 이벤트, 윈도우 관련 이벤트

이벤트가 다양하므로 필요한 경우 하나씩 학습하세요.

click()	마우스를 클릭할 때 발생함.
dblclick()	마우스를 더블 클릭할 때 발생함.
mousedown()	마우스 버튼을 누를 때 발생함.
mouseup()	마우스 버튼을 땔 때 발생함.
mouseenter()	마우스 요소의 경계 외부에서 내부로 이동할 때 발생. (자기 자신만 이벤트)
mouseleave()	마우스 요소의 경계 내부에서 외부로 이동할 때 발생. (자기 자신만 이벤트)
mousemove()	마우스를 움직일 때 발생함.
mouseout()	마우스가 요소를 벗어날 때 발생함. (버블링 발생)
mouseover()	마우스를 요소 안에 들어올 때 발생함. (버블링 발생)
hover()	mouseenter()와 mouseleave()를 하나로 만든 이벤트

▲ 마우스 관련 이벤트

▼ 키보드 관련 이벤트

keydown()	키 입력 시 발생하는 이벤트
keypress()	keydown과 같지만 Enter, Tab 등의 특수키에는 발생하지 않음
keyup()	키 입력 후 발생하는 이벤트

[참고] mouseover() mouseout()과 mouseenter() mouseleave()의 차이점

부모 요소와 자식 요소로 구분되어 있는 경우 **mouseover()** **mouseout()** 이벤트는 부모 요소에 이벤트를 주어도 자식 요소에도 이벤트가 발생하는 문제가 있습니다. 하지만 **mouseenter()** **mouseleave()**은 자식 요소에 이벤트 영향을 주지 않습니다. (일반적으로 **mouseenter()** **mouseleave()** 사용함)

ready()	문서 객체가 준비를 완료함.
load()	윈도우(문서 객체)를 불러들일 때 발생함.
unload()	윈도우(문서 객체)를 닫을 때 발생함.
resize()	윈도우의 크기를 변화시킬 때 발생함.
scroll()	윈도우를 스크롤할 때 발생함. <code>\$(window).scroll(function(){...});</code>
error()	에러가 있을 때 발생함.

▲ 윈도우 관련 이벤트

▼ 입력 양식(form) 관련 이벤트 - 폼 관련 이벤트는 실무에서 중요하게 사용됩니다. 추후 실전 예제를 통해서 학습합니다.

change()	입력 양식의 내용을 변경할 때 발생함.
focus()	입력 양식에 초점을 맞추면 발생함.
focusin()	입력 양식에 초점이 맞춰지기 바로 전에 발생함.
focusout()	입력 양식에 초점이 사라지기 바로 전에 발생함.
blur()	입력 양식에 초점이 사라지면 발생함.
select()	입력 양식을 선택할 때 발생(input[type=text] 태그와 textarea 태그 제외).
submit()	submit 버튼을 누르면 발생함.
reset()	reset 버튼을 누르면 발생함.

▼ 폼 관련 메서드 및 속성

val()	입력 요소의 값(value)을 가져오거나 변경
length	요소 값의 개수를 가져오기

이벤트 종류(2) - 마우스 관련 이벤트, 키보드 관련 이벤트, 윈도우 관련 이벤트(사용 예시)

{ 마우스 이벤트 기본형 }

```
<button class="button1">button1</button>
<hr style="margin: 10px 0;">
<button class="button2">button2</button>
```

// 기본형 이벤트(방식1)

```
$('.button1').on('click', function(){
    $(this).text('버튼1을 클릭했습니다.');
```

→ on('이벤트핸들러')

button1

버튼1을 클릭했습니다.

// 기본형 이벤트(방식2)

```
$('.button2').click(function(){
    $(this).text('버튼2을 클릭했습니다.');
```

→ 이벤트핸들러()

button2

버튼2을 클릭했습니다.

▲ on('click')는 이벤트를 바인딩 할 수 있고, click()는 이벤트를 바인딩 할 수 있는지의 차이(일반적으로 click()를 사용)
이벤트 바인딩이로 click, mouseenter 등 하나의 요소에 여러 개의 이벤트 핸들러를 사용하는 것.

```
<p class="text">
    Lorem ipsum dolor sit amet consectetur adipisicing elit.
</p>
```

// 이벤트 바인딩

```
$('.text').on('mouseenter mouseleave', function(){
    $(this).after('마우스 커서가 문장 위로 들어오거나 빠져 나갔습니다.');
```

Lorem ipsum dolor sit amet consectetur adipisicing elit.

마우스 커서가 문장 위로 들어오거나 빠져 나갔습니다.
마우스 커서가 문장 위로 들어오거나 빠져 나갔습니다.
마우스 커서가 문장 위로 들어오거나 빠져 나갔습니다.
마우스 커서가 문장 위로 들어오거나 빠져 나갔습니다.

```
<h3 class="heading" style="border: 1px solid; padding: 5px;">This is H1 tag</h3>
```

// 이벤트 바인딩 후 바인딩별 함수

```
$('.heading').on({
    click: function(){
        $(this).after('마우스가 문장을 클릭했습니다.<br>');
    },
    mouseenter: function(){
        $(this).after('마우스가 커서가 문장 위로 들어왔습니다.<br>');
    },
    mouseleave: function(){
        $(this).after('마우스가 커서가 문장을 빠져 나갔습니다.<br>');
    }
});
```

This is H1 tag

마우스가 문장을 클릭했습니다.
마우스가 커서가 문장 위로 들어왔습니다.
마우스가 커서가 문장을 빠져 나갔습니다.
마우스가 커서가 문장 위로 들어왔습니다.

이벤트 종류(3) - 폼 관련 이벤트(사용 예시)

{ change 이벤트 } → change 이벤트는 요소가 변화가 있을 때 사용되는 이벤트로 보통 폼 요소에 사용됨

```
<form action="#">
  <b>가장 선호하는 축구 유튜브 채널 : </b>
  <select id="football">
    <option value="">선택하세요.</option>
    <option value="달수네라이브">달수네라이브</option>
    <option value="아이게스">아이게스</option>
    <option value="서형욱 톨리TV">서형욱 톨리TV</option>
    <option value="이스타TV">이스타TV</option>
  </select>
  <hr style="margin: 15px 0;">
  <p>선택하신 축구 유튜브 채널 : <span class="result"></span></p>
</form>
```

```
/* Custom CSS */
form {
  width: 400px;
  text-align: center;
  box-shadow: 0 0 15px rgba(0, 0, 0, 0.3);
  border-radius: 15px;
  padding: 25px;
}
.result { color: crimson; }
```

```
<script>
  // 문서객체 선택
  let selectElement = document.getElementById('football');

  // 선택된 option의 value 값 출력
  selectElement.addEventListener('change', function(event){
    let result = document.querySelector('.result');
    result.textContent = `${event.target.value}`;
  });
</script>
```

▲ 자바스크립트 방식으로 value 출력

가장 선호하는 축구 유튜브 채널 : 선택하세요. ▼

선택하신 축구 유튜브 채널 :

가장 선호하는 축구 유튜브 채널 : 이스타TV ▼

선택하신 축구 유튜브 채널 : 이스타TV

```
<script>
  // 선택상자 선택의 변화(change)
  $('#football').change(function(){
    var selectElement = $(this).find('option:selected').val();
    // 선택된 option의 value 값 출력
    $('.result').text(selectElement);
  });
</script>
```

optoin의 텍스트를 가지고 올 경우는 text() 을 사용

- \$(this) : option 중에서 선택된 것
- option:selected (옵션 중에 선택된 것이라는 선택자)
- Seterr로서 가져오는 text()

▲ 제이쿼리 방식으로 value 출력

01) 제이쿼리 애니메이션 효과 - `animate()` 기본 문법

- `.animate()` 메소드를 이용하여 사용자 정의한 이펙트 효과 만들기
- `animate()` 메서드는 웹 퍼블리셔로서 제이쿼리에서 가장 사용 빈도가 높습니다.

```
{      $('선택자').animate({CSS스타일}, 지속시간, 이징, 콜백함수); }
```

```
$('div').animate({left: 200, top: 200});
```

→ `animate()` 메서드의 필수적인 형태는 CSS 스타일

```
$('div').animate({left: 200, top: 200}, 'fast', 'linear', function() {...});
```

- **CSS스타일** : 필수이며, 이펙트 효과를 구성할 CSS 스타일 속성을 정의한다.
- **지속 시간** : 이펙트 효과가 지속될 시간을 전달한다.
- **시간당 속도 함수(easing function)** : 이펙트 효과의 시간당 속도를 전달한다.
- **콜백함수(Callback function)** : 이펙트 효과의 동작이 완료된 후에 실행할 함수를 정의한다.
- 지속시간은 애니메이션 효과를 완료할 때까지 걸리는 시간. 단위는 1/1000초, 기본값은 400, fast나 slow로 정할 수 있음. fast는 200, slow는 600에 해당
- 이징(easing) : 애니메이션 효과의 방식을 정함. swing과 linear이 가능하며, 기본값은 swing

animate()에서 사용하는 애니메이션 효과 속성

- | | |
|-----------------------|-----------------|
| • backgroundPositionX | • marginTop |
| • backgroundPositionY | • maxHeight |
| • borderBottomWidth | • maxWidth |
| • borderLeftWidth | • minHeight |
| • borderRightWidth | • minWidth |
| • borderSpacing | • opacity |
| • borderTopWidth | • outlineWidth |
| • borderWidth | • padding |
| • bottom | • paddingBottom |
| • fontSize | • paddingLeft |
| • height | • paddingRight |
| • left | • paddingTop |
| • letterSpacing | • right |
| • lineHeight | • textIndent |
| • margin | • top |
| • marginBottom | • width |
| • marginLeft | • wordSpacing |
| • marginRight | |

▲ 거의 대부분의 CSS 속성을 사용할 수 있습니다.

수많은 애니메이션 속성을 외우지 마시고 예제를 만들 때마다 써보면 됩니다.

01) 제이쿼리 애니메이션 효과 - `animate()` 사용 예시 #01

{ `animate()` 메서드 : 기본형 }

```
<button class="btn1">Click</button>
<div class="box1"></div>

<script>
  // 기본형 animate() 메서드
  $('.btn1').click(function(){
    $('.box1').animate({
      width: '100%',
      borderRadius: '10px'
    });
  });
</script>
```



`'width': '100%',`
`'border-radius': '10px'`

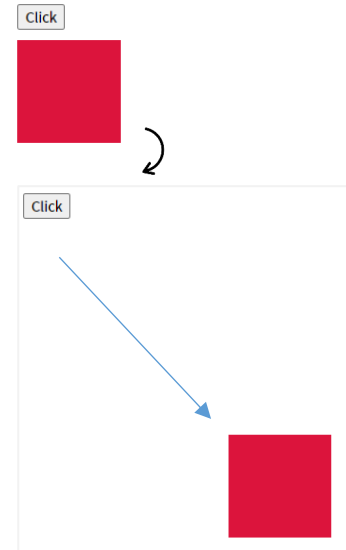
```
/* Custom CSS */
.btn1 {
  margin-bottom: 10px;
}
.box1 {
  width: 100px;
  height: 100px;
  background-color: yellowgreen;
}
```

▲ 따옴표 없이 사용하는 `width`와 `borderRadius` 와 같은 것은 제이쿼리 `animate()` 메서드에서 사용 가능하게 만들어진 속성입니다. 하지만 같은 기능이지만 CSS 속성 이름을 사용하려면 반드시 따옴표로 묶어야 합니다.

{ `animate()` 메서드 with 지속시간,이징 }

```
<button class="btn">Click</button>
<div class="box"></div>

<script>
  // 기본형 animate() 메서드
  $('.btn').click(function(){
    $('.box').animate({
      left: 200,
      top: 200
    }, 'slow', 'linear');
  });
</script>
```



지속시간과 이징은 특별한 경우가 아니라면 굳이 사용할 필요는 없습니다. 없어도 기본적인 자연스러운 애니메이션이 가능(사용시 반드시 따옴표로 묶음)

```
/* Custom CSS */
.btn {
  margin-bottom: 10px;
}
.box {
  width: 100px;
  height: 100px;
  background-color: crimson;
  position: relative;
}
```

`animate()` 메서드에 `left`, `top` 등 좌표 관련 속성을 사용하므로 `.box`에 반드시 포지션 속성이 있어야 합니다.(`relative`, `absolute`, `fixed` 등)

01) 제이쿼리 애니메이션 효과 - `animate()` 사용 예시 #02{ `animate()` 메서드 with 콜백함수 }

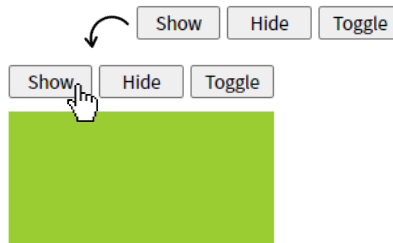
```
<button class="btn">Click</button>
<div class="box"></div>

<script>
  // animate() 메서드 with 콜백함수
  $(' .btn').click(function(){
    $(' .box').animate({
      width: '100%',
      borderRadius: '10px'
    }, 'fast', function(){ ①
      $(this).css('background-color', 'crimson');
    }); ②
  });
</script>
```

```
/* Custom CSS */
.btn {
  margin-bottom: 10px;
}
.box {
  width: 100px;
  height: 100px;
  background-color: yellowgreen;
}
```



- ① 너비와 모서리 둥글게가 끝나고 나서 실행되는 함수 곧, 콜백함수에 배경색을 변경하는 `css()` 메서드 사용
- ② `$(this)`는 자동으로 `.box`를 의미함

{ `animate()` 메서드 with show hide toggle }

- 요소를 단순히 보이고 감추기 할 때 사용
- 보이기 show / 감추기 hide / 보이기/감추기 toggle

```
<div class="frame">
  <div class="buttons">
    <button>Show</button>
    <button>Hide</button>
    <button>Toggle</button>
  </div>
  <div class="box"></div>
</div>

/* Custom CSS */
.frame { width: 200px; }
.buttons { display: flex; gap: 5px; }
.buttons button { flex: 1; }
.box {
  width: 200px;
  height: 100px;
  background-color: yellowgreen;
  margin: 10px 0;
}

<script>
  // animate() 메서드 with show hide toggle
  $(' .buttons button:nth-child(1)').click(function(){
    $(' .box').animate({height: 'show'})
  });
  $(' .buttons button:nth-child(2)').click(function(){
    $(' .box').animate({height: 'hide'})
  });
  $(' .buttons button:nth-child(3)').click(function(){
    $(' .box').animate({height: 'toggle'})
  });
</script>
```

01) 제이쿼리 애니메이션 효과 - animate() 사용 예시 #03

{ animate() 메서드 with 이징(easing) }

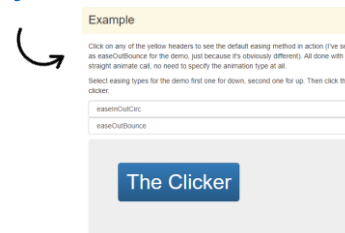
- 이징은 지속시간은 같지만 시작과 끝을 어떻게 변화해서 완료가 되는지의 속도감
- 기본적으로 제이쿼리의 easing(속도감)은 기본값이 swing
- 별도의 플러그인이 없으면 swing과 linear만 사용 가능

[이징(easing) 플러그인 사용하기]

- ① jQuery Easing Plugin 공식사이트 접속
<https://gsqcd.co.uk/sandbox/jquery/easing/>
- ② 이징 js 파일 다운로드(jQuery.easing.1.3.js) 또는 CDN 사용

[이징(easing) 효과 미리보기]

- ① <https://superkts.com/jquery/@easingEffects>
- ② <https://gsqcd.co.uk/sandbox/jquery/easing/>



```
<button class="btn">Easing Start</button>
<div class="box box1"></div>
<div class="box box2"></div>
<div class="box box3"></div>

/* Custom CSS */
.btn { margin-bottom: 10px; }
.box {
  width: 100px; height: 100px;
  background-color: pink; margin-bottom: 10px;
}

<script>
  // animate() 메서드 with 콜백함수
  $(' .btn').click(function(){
    $(' .box1').animate({
      width: '100%'
    },1000, 'easeInElastic');
    $(' .box2').animate({
      width: '100%'
    },1000, 'easeInOutBack');
    $(' .box3').animate({
      width: '100%'
    },1000, 'easeInOutSine');
  });
</script>
```

[사용 가능한 이징(easing) 효과 이름]

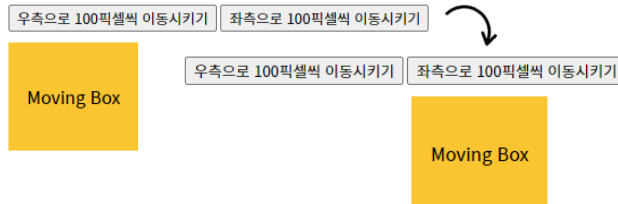
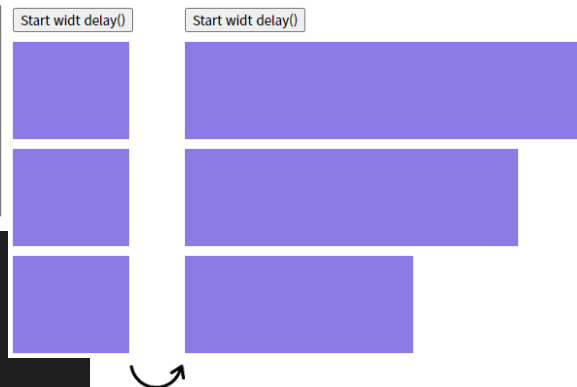
linear swing easeInQuad easeOutQuad easeInOutQuad easeInCubic easeOutCubic easeInOutCubic
 easeInQuart easeOutQuart easeInOutQuart easeInQuint easeOutQuint easeInOutQuint easeInSine
 easeOutSine easeInOutSine easeInExpo easeOutExpo easeInOutExpo easeInCirc easeOutCirc
 easeInOutCirc easeInElastic easeOutElastic easeInOutElastic easeInBack easeOutBack easeInOutBack
 easeInBounce easeOutBounce easeInOutBounce

01) 제이쿼리 애니메이션 효과 - `animate()` 사용 예시 #04{ `delay()` 메서드 }▲ `fadeIn()` `slideDown()` 등 효과 메서드나 `animate()` 메서드를 지연시킬 때 사용`$('.선택자').delay(시간).animate();`

```
/* Custom CSS */
.btn { margin-bottom: 10px; }
.box {
  width: 120px; height: 100px;
  background-color: #8c7ae6; margin-bottom: 10px;
}
```

```
<button class="btn">Start widt delay()</button>
<div class="box box1"></div>
<div class="box box2"></div>
<div class="box box3"></div>
```

```
<script>
// animate() 메서드 with 콜백함수
$('.btn').click(function(){
  $('.box1').delay(0).animate({width: '100%'},1000);
  $('.box2').delay(250).animate({width: '100%'},1000);
  $('.box3').delay(500).animate({width: '100%'},1000);
});
</script>
```

{ `animate()` with 복합연산자 }▲ `+=` `-=` 과 같은 복합연산자를 사용해서 상대적으로 이동

```
<div class="buttons">
  <button class="btn1">우측으로 100픽셀씩 이동시키기</button>
  <button class="btn2">좌측으로 100픽셀씩 이동시키기</button>
</div>
<div class="moving">Moving Box</div>

<script>
// animate() with 복합연산자
$('.btn1').click(function(){
  $('.moving').animate({left: '+=100'});
});
$('.btn2').click(function(){
  $('.moving').animate({left: '-=100'});
});
</script>
```

→ 버튼을 누를 때마다 우측
으로 100픽셀 씩 상대적
으로 이동 시키기

```
/* Custom CSS */
.buttons { margin-bottom: 10px; }
.moving {
  width: 120px; height: 100px;
  background-color: #fbc531; margin-bottom: 10px;
  display: flex; justify-content: center; align-items: center;
  position: relative;
}
```

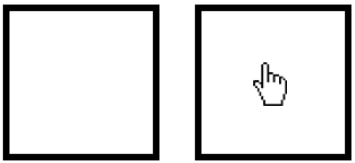
01) 제이쿼리 애니메이션 효과 - 어라? animate() 메서드로 배경색은 변경이 안되네?

```
/* Custom CSS */
.box {
  border: 5px solid #000;
  width: 100px;
  height: 100px;
}
```

```
<div class="box"></div>

<script>
  $('.box').mouseover(function(){
    $(this).animate({backgroundColor: 'blue'}, 500);
  })
  .mouseout(function(){
    $(this).animate({backgroundColor: 'green'}, 500);
  });
</script>
```

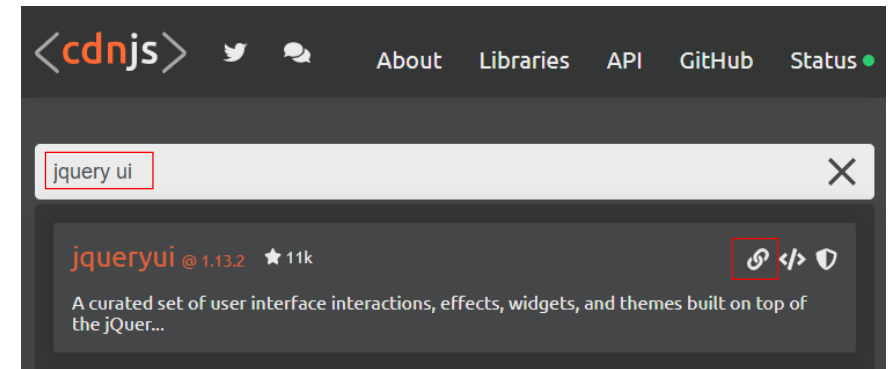
마우스 올릴 때마다 반복되므로 stop() 메서드 필요(stop 메서드는 '이전에 하던 일을 멈춰')



색상 값을 넣을 때는 아주 일반적인 green, blue, yellow 이런 것들은 작동하지만 crimson, yellowgreen 과 같은 것을 넣으면 작동하지 않습니다. crimson, yellowgreen 을 사용하고 싶을 때는 16진수 값으로 넣어야 합니다.
ex) #eb4d4b

▲ 마우스가 오버되어도 배경색상이 변경되지 않습니다. 제이쿼리 animate() 메서드로 배경색을 변경하려면 jQuery UI를 링크해야 합니다.

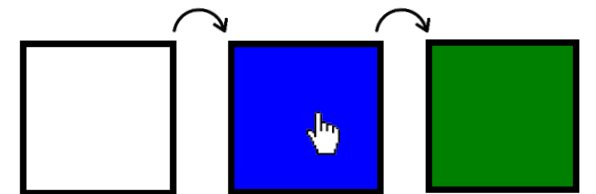
배경색은 수치가 변경되는 것이 아니기 때문에 jQuery UI가 필요합니다.
jQuery UI 파일을 링크해야만 배경색을 animate()로 변경할 수 있습니다.



▲ cdnjs에서 jquery ui 라고 검색하고 링크를 복사해서 head 사이에 넣음

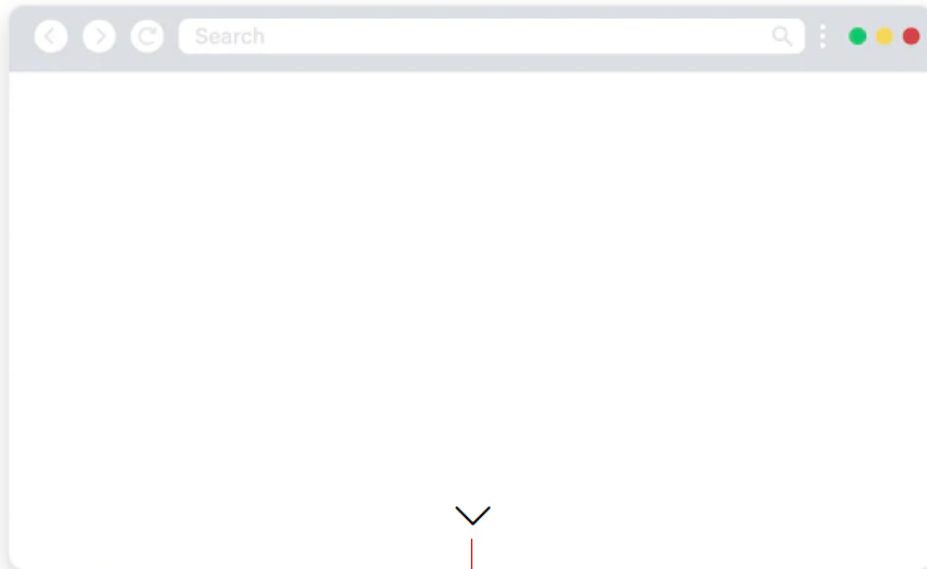
```
<!-- jQuery CDN -->
<script src="https://code.jquery.com/jquery-3.6.4.min.js"></script>
<!-- jQuery UI -->
<script src="https://cdnjs.cloudflare.com/ajax/libs/jqueryui/1.13.2/jquery-ui.min.js"></script>
```

▲ 제이쿼리 UI도 역시 제이쿼리 라이브러리가 있어야 작동되므로 순서가 바뀌면 절대로 안됩니다.



01) 제이쿼리 반복 애니메이션 효과 - loop() 함수

↪ CSS로 하는게 수월하지만 제이쿼리 배운 김에 번외로 한번쯤 해보세요.



제이쿼리 loop() 함수로 무한 반복해서 아래 위로 움직이는 애니메이션

```
<section>
  <div id="arrow"><i class="bi bi-chevron-down"></i></div>
</section>
```

HTML

```
/* Custom CSS */
section {
  height: 100vh;
  position: relative;
}
#arrow {
  position: absolute;
  font-size: 40px;
  left: 50%;
  transform: translateX(-50%);
  bottom: 30px; —▶ 시작(40픽셀)과 종료(60픽셀) 중간인
                  30픽셀로 세팅하는 것이 자연스러움
}
```

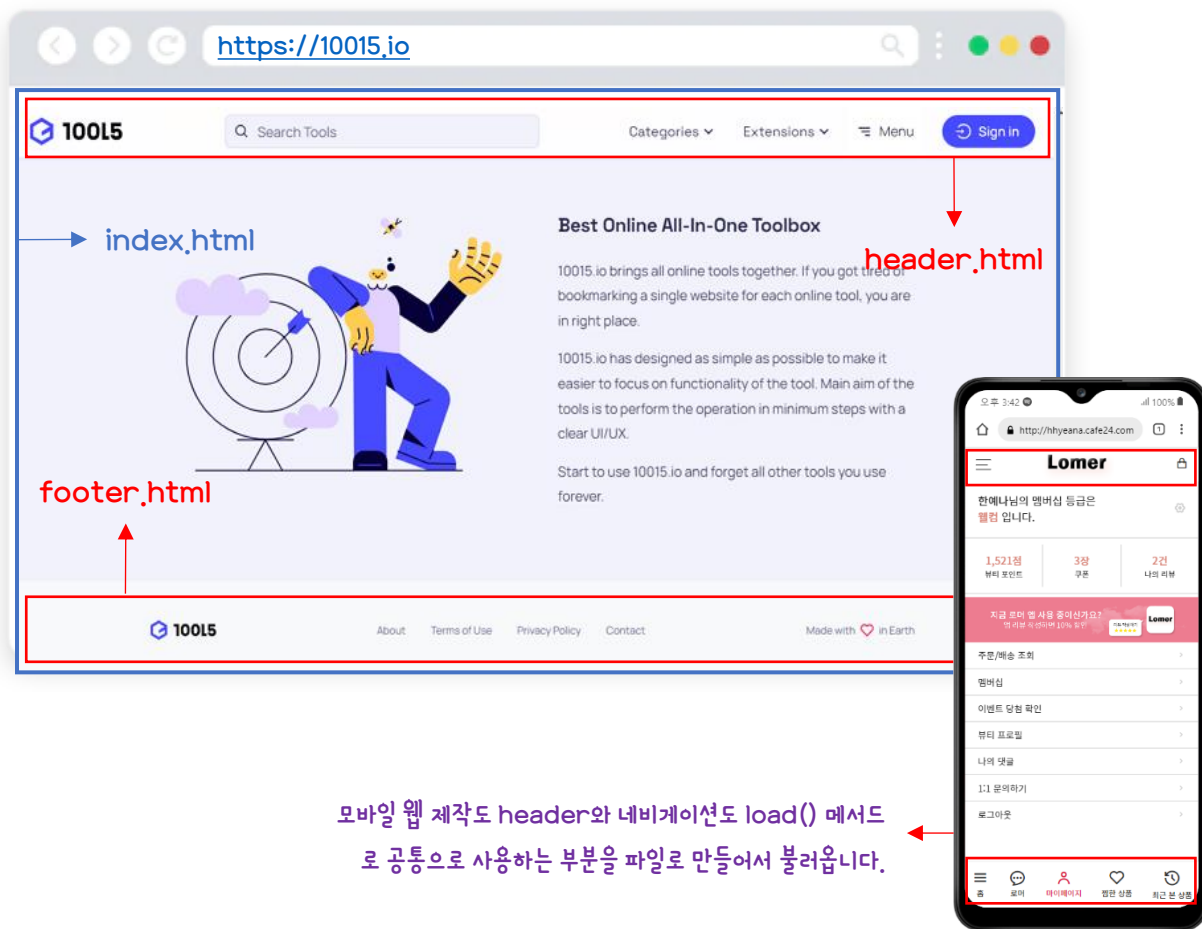
CSS

```
function loop() {
  $('#arrow').animate({bottom: 40}, 700).animate({bottom: 20}, 700, loop);
}
loop();
```

JS

02-1) 개별 문서를 페이지의 원하는 곳에 불러오는 load() 메서드

- 제이쿼리 load 메서드로 공통 영역 파일을 불러옴
- load() 메서드로 불러온 파일 안에 제이쿼리 구문을 실행하기 위해 반드시 콜백함수로 넣어줘야 합니다.



[load 메서드]

웹 사이트에서 공통 영역을 따로 파일로 만들어 일정 영역에 불러옴.

공통 영역을 따로 파일로 만들지 않으면 모든 페이지에 매번 같은 html 구조가 들어갑니다. 그러면 매번 같은 작업을 해줘야 하는 것도 있지만 변경 사항이 생기면 파일 개수만큼 고쳐줘야 합니다.

예를 들어 index.html 부터 페이지 수가 50~100개라면 50~100번을 수정해야 합니다. 하지만 load로 공통 영역을 따로 파일로 만들어 일정 영역에 불러오면 공통 영역 파일 한번만 고쳐주면 됩니다. 그래서 header와 footer와 같은 부분은 header.html과 footer.html로 만들어서 load로 불러옵니다.

[실습을 위한 폴더 구조]

```

✓ LOAD 메서드 연습하기
  ✓ include
    <> footer.html
    <> header.html
  JS custom.js
    <> index.html
  # style.css
  
```

1. load 메서드 연습하기.. 라는 폴더를 만듭니다. 이 폴더를 루트 폴더로 지정하고 VSC로 엽니다.
 2. 폴더 안에 index.html custom.js 파일과 style.css 파일을 만들고 include라는 폴더를 만듭니다.
 3. include 폴더 안에 header.html과 footer.html을 만듭니다.
- ※ 굳이 include라는 폴더를 만들 필요는 없지만 보통 load로 불러오는 파일은 한 곳에 모아 두는 것이 좋습니다.

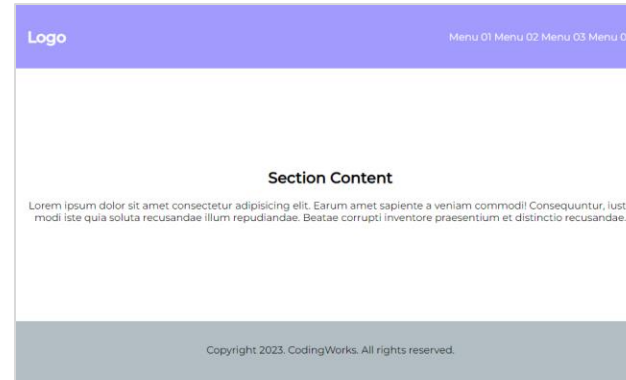
02-2) load() 메서드 실습하기 #01

```
<!DOCTYPE html>
<html lang="ko">
<head>
  <title>index.html</title>
  <!-- Custom CSS & JS -->
  <script src="custom.js"></script>
  <link rel="stylesheet" href="style.css">
</head>
<body>

<div class="container">
  <header>
    <h2>Logo</h2>
    <div class="gnb">
      <a href="#none">Menu 01</a>
      <a href="#none">Menu 02</a>
      <a href="#none">Menu 03</a>
      <a href="#none">Menu 04</a>
    </div>
  </header>
  <section>
    <div class="content">
      <h1>Section Content</h1>
      <p>
        Lorem ipsum dolor sit amet consectetur adipisicing elit. Earum amet sapiente a veniam commodi Consequuntur, iusto modi iste quia soluta recusandae illum repudiandae. Beatae corrupti inventore praesentium et distinctio recusandae.
      </p>
    </div>
  </section>
  <footer>
    <p>Copyright 2023. CodingWorks. All rights reserved.</p>
  </footer>
</div>

</body>
</html>
```

- ① index.html의 모든 html 구조를 완성합니다
- ② CSS로 기본적인 배치를 합니다.
- ③ header와 footer html을 잘라내서 따로 파일을 만듭니다.



```
<div class="container">
  <section>
    <div class="content">
      <h1>Section Content</h1>
      <p>
        Lorem ipsum dolor sit amet consectetur adipisicing elit. Earum amet sapiente a veniam commodi Consequuntur, iusto modi iste quia soluta recusandae illum repudiandae. Beatae corrupti inventore praesentium et distinctio recusandae.
      </p>
    </div>
  </section>
</div>
```

③에서 header와 footer html을 잘라내면 헤더와 푸터 부분이 당연히도 뺀 뚫리게 됩니다.

02-2) load() 메서드 실습하기 #02

```
<header>
  <h2>Logo</h2>
  <div class="gnb">
    <a href="#none">Menu 01</a>
    <a href="#none">Menu 02</a>
    <a href="#none">Menu 03</a>
    <a href="#none">Menu 04</a>
  </div>
</header>
```

header.html

```
<footer>
  <p>Copyright 2023. CodingWorks. All rights reserved.</p>
</footer>
```

footer.html

▲ 위의 2개 파일을 index.html 파일에 불러올 곳을 만들기

```
<div class="container">
  <div class="include-header"></div>
  <section>
    <div class="content">
      <h1>Section Content</h1>
      <p>
        Lorem ipsum dolor sit amet consectetur adipisicing elit. Earum amet sapiente a veniam commodi! Consequuntur, iusto modi iste quia soluta recusandae illum repudiandae. Beatae corrupti inventore praesentium et distinctio recusandae.
      </p>
    </div>
  </section>
  <div class="include-footer"></div>
</div>
```

▲ include라는 이름을 가진 곳에 별도로 만든 파일을 load() 메서드로 불러오게 됩니다. 클래스네임을 직관성을 위해서 include라는 이름을 사용하는 것이 좋습니다.

불러와지는 파일은 html은 기본문서구조가 빠진 header와 footer 내용만 있어야 합니다.

[콜백함수]

불러와지는 파일 안에 제이쿼리가 작동하는 부분이 있다면 반드시 콜백함수로 넣어줘야 합니다.

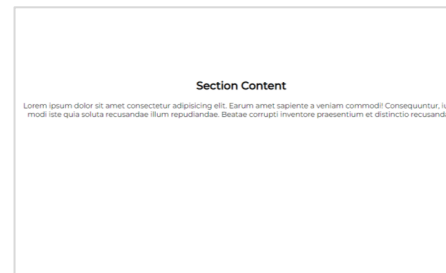
```
$(function(){
  $('.include-header').load('include/header.html', function(){
    // header에 사용된 제이쿼리 구문
  });
  $('.include-footer').load('include/footer.html', function(){
    // footer에 사용된 제이쿼리 구문
  });
});
```

```
$(function(){
  $('.include-header').load('include/header.html');
  $('.include-footer').load('include/footer.html');
});
```

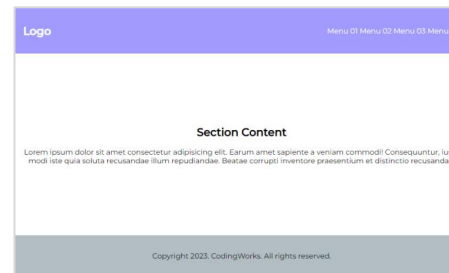
[참고]

load 메서드로 불러와지는 파일은 오프라인 환경에서는 작동하지 않습니다. 반드시 라이브 서버에서 봐야 합니다.

(오프라인 환경이란 파일 탐색기에서 클릭해서 html 오픈하는 것을 말합니다.)

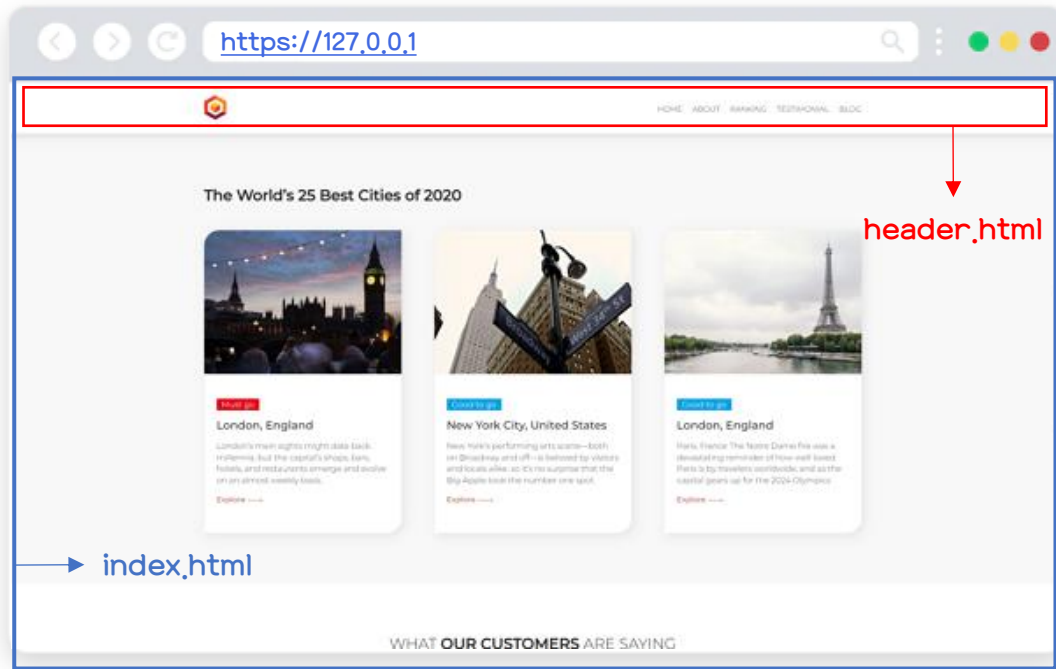


▲ load 메서드로 불러오기 전



▲ load 메서드로 불러온 후

03-1) load 메서드로 load된 html 파일 안에 콜백함수 사용하기 ✕



[load 메서드로 load된 html 파일 안에 제이쿼리 구문이 있는 경우]

- 좌측의 예시처럼 index.html 파일에서 load 메서드로 header.html을 불러온 경우 header.html에 제이쿼리 구문 또는 자바스크립트 구문이 있다면 콜백함수를 사용해야 합니다.
- 예를 들어 header.html에 를 눌러서 사이드바를 열거나 제이쿼리로 뭔가 작동을 시킨다면 반드시 콜백함수를 사용해야 합니다.

// 콜백함수 사용하지 않은 경우

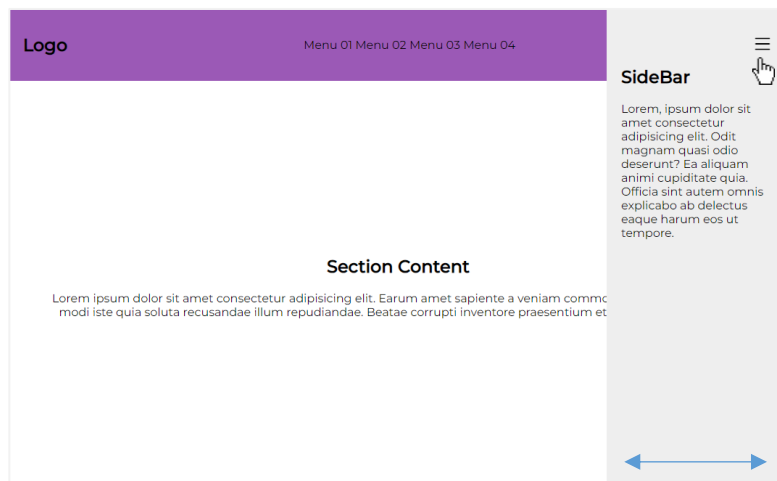
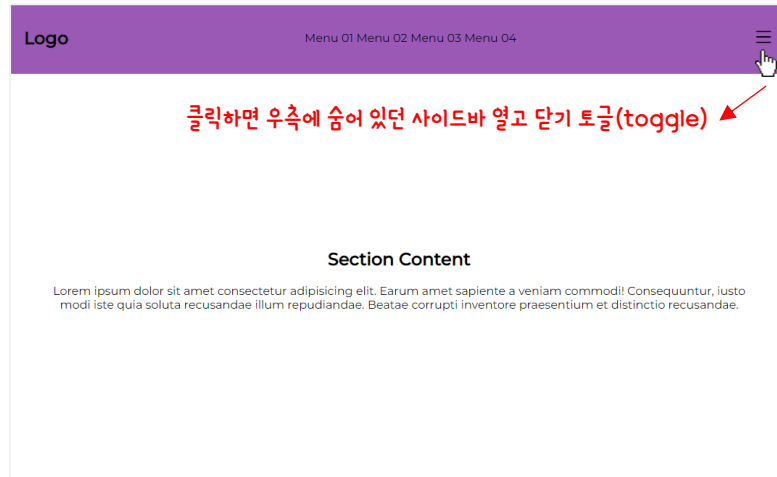
```
$(function(){  
  $('.include-header').load('include/header.html');  
  // header에 사용된 제이쿼리 구문  
});
```

// 콜백함수 사용한 경우

```
$(function(){  
  $('.include-header').load('include/header.html', function(){  
    // header에 사용된 제이쿼리 구문  
  });  
});
```

콜백함수(Callback Function)

03-2) load 메서드로 load된 html 파일 안에 콜백함수 사용하기



```
<!DOCTYPE html>
<html lang="ko">
<head>
  <title>index.html</title>
  <!-- jQuery CDN -->
  <script src="https://code.jquery.com/jquery-3.
  <!-- Custom CSS & JS -->
  <script src="custom.js"></script>
  <link rel="stylesheet" href="style.css">
</head>
<body>

  <div class="container">
    <div class="include-header"></div>
    <section>
      <div class="content">
        <h1>Section Content</h1>
        <p>...
      </p>
    </div>
  </section>
</div>

  <div class="header">
    <h2>Logo</h2>
    <div class="gnb">
      <a href="#none">Menu 01</a>
      <a href="#none">Menu 02</a>
      <a href="#none">Menu 03</a>
      <a href="#none">Menu 04</a>
    </div>
    <div class="trigger">
      <i class="bi bi-list"></i>
    </div>
  </div>

  <div class="sidebar">
    <h2>SideBar</h2>
    <p>...
  </p>
</div>
```

```
/* Trigger */
.trigger {
  font-size: 30px;
  cursor: pointer;
  position: relative;
  z-index: 1000;
}

/* Sidebar */
.sidebar {
  position: fixed;
  top: 0;
  right: -250px;
  background-color: #eee;
  width: 250px;
  height: 100vh;
  padding: 60px 20px;
  transition: 0.35s;
}

.sidebar.active {
  right: 0;
}
```

```
// 콜백함수 사용하지 않은 경우
$(function(){
  $('include-header').load('include/header.html');
  $('trigger').click(function(){
    $('sidebar').toggleClass('active');
  });
});
```

해당 제이쿼리 구문은 header.html 내에 있는 요소를 제어하는 코드로 콜백함수로 사용하지 않으면 작동하지 않습니다.

```
// 콜백함수 사용한 경우
$(function(){
  $('include-header').load('include/header.html', function(){
    $('trigger').click(function(){
      $('sidebar').toggleClass('active');
    });
  });
});
```

header.html을 불러오고 load 메서드가 종료되지 않은 상태에서 header.html에 사용된 제이쿼리 구문을 콜백함수로 실행하므로 정상적으로 작동합니다.



JQUERY 실전 예제

04. 제이쿼리 실전 예제

- 3가지 타입 이미지 슬라이더 만들기 제이쿼리 탭 메뉴 콘텐츠
- 제이쿼리 어코디언 콘텐츠
- 푸터 패밀리사이트 커스텀 셀렉스(select) 박스
- input color를 사용해 배경색 바꾸기
- 마우스 내리면 CSS 변화와 부드럽게 상단 찾아가기
- 부드럽게 스크롤링 되면서 각각 섹션 찾아가기
- 폰트 사이즈 증가 감소 막대 만들기
- 폰트 사이즈 증가 감소 버튼 만들기
- 라이트모드 & 다크모드 전환하기
- 파일 업로드 폼 만들기
- 프로필 사진 업로드 폼 만들기
- 체크박스 체크하면 안보이는 입력 필드 보이게 하기
- 더 보기 버튼 누르면 숨겨진 내용 로드(load) 하기
- 썸네일 이미지 누르면 큰 이미지 보이는 사진 갤러리
- 비밀번호 입력 값 보이기 감추기 with 폰트아이콘
- 텍스트 입력상자(textarea) 글자수 체크하기
- 마우스 포인터 따라 다니는 돋보기 기능 만들기
- 마우스 스크롤에 따라 페럴렉스 효과(Parallax Effect)
- 모바일 하단의 아이콘 탭 바 네비게이션 만들기
- 할 일(To Do List) 만들기
- 슬라이더(이전 다음 버튼, Autoplay 제어 버튼)
- 슬라이더(자동 Autoplay, 마우스 오버 멈추고 빠지면 다시 시작)
- 폼 입력 필드 클릭하면 텍스트 올라가는 애니메이션
- 이미지 호버 이펙트를 제이쿼리 animate로 만들기
- 사이드바 네비게이션으로 숨겨진 콘텐츠 열기
- 사진 갤러리 라이트박스(Lightbox) 만들기
- 반응형 애니메이션 모달 띄우기
- Front Back 트랜지션 화면전환



자바스크립트와 제이쿼리로 뭔가를 만들 때 제작자에 따라 제작하는 방법과 코드는 매우 다양한 합니다. 실전 예제의 제작 방식과 코드가 정답이라고 생각하지 마시고 다른 제작자의 코드도 참고하시기 바랍니다.

제이쿼리 실전 예제(01-1) - 3가지 타입 이미지 자동 슬라이더 만들기(웹 디자인 기능사 실기 시험)

Small Mission 01) 가로 슬라이더



```
<!-- Slide Animation -->
<div class="slide">
  <div class="slide-items">
    <a class="slide-item" href="#none"></a>
    <a class="slide-item" href="#none"></a>
    <a class="slide-item" href="#none"></a>
  </div>
</div>
<!-- Slide Animation -->
```

HTML

→ .slide div로 해도 되지만 .slide-items라는 클래스 이름을 준건 제이쿼리에서 객체 선택을 쉽게 하기 위해서 입니다.

반드시 3500으로 해야 할 필요는 없습니다. 하지만 슬라이드 전체가 10초로 볼 때 1/3이니까 3500이 적당합니다.

```
setInterval(function(){
  $('.slide-items').animate({left : '-1200px'}, function(){
    $('.slide-items').css({left : 0});
    $('.slide-item:first-child').appendTo('.slide-items');
  })
}, 3500);
```

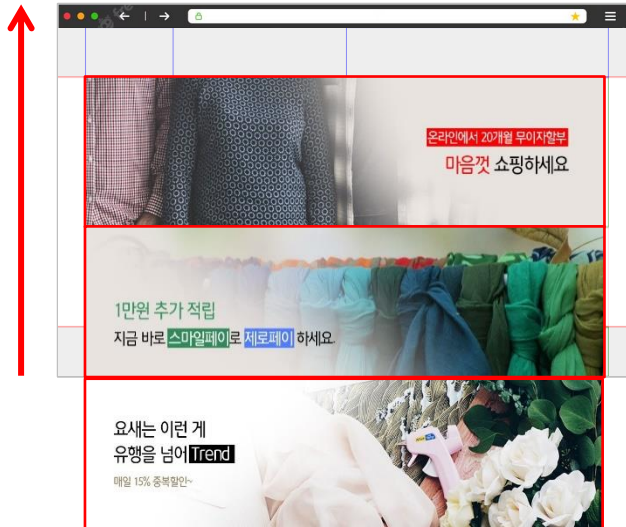
▲ .slide-item:first-child 는 .slide-item 밑에 첫번째 a 태그를 말합니다.

JS

- left 대신 margin-left를 사용해도 결과는 같습니다.
- 슬라이드 전체를 이동시키는 left : '-1200px' 이라고 따옴표 안에 px 단위 까지 써주세요. 픽셀의 경우 left: -1200 이라고 단위와 따옴표 없이 해도 되지만 D,E타입에서는 left: '-100%' 라고 단위를 꼭 써야 합니다.
- 순식간에 실행되기 때문에 오렌지 박스의 코드 2줄의 순서는 바뀌도 상관없습니다.

제이쿼리 실전 예제(01-2) - 3가지 타입 이미지 자동 슬라이더 만들기(웹 디자인 기능사 실기 시험)

Small Mission 02) 세로 슬라이더



.slide div로 해도 되지만 .slide-items라는 클래스 네임을 준건 제이쿼리에서 객체 선택을 쉽게 하기 위해서 입니다.

```
<!-- Slide Animation -->
<div class="slide">
  <div class="slide-items">
    <a class="slide-item" href="#none"></a>
    <a class="slide-item" href="#none"></a>
    <a class="slide-item" href="#none"></a>
  </div>
</div>
<!-- Slide Animation -->
```

```
setInterval(function(){
  $('.slide-items').animate({top : '-300px'}, function(){
    $('.slide-items').css({top : 0});
    $('.slide-item:first-child').appendTo('.slide-items');
  })
}, 3500);
```

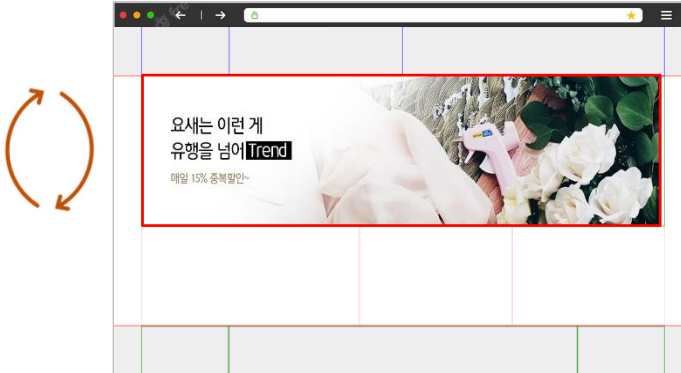
▲ .slide-item:first-child 는 .slide-item 밑에 첫번째 a 태그를 말합니다.

반드시 3500으로 해야 할 필요는 없습니다. 하지만 슬라이드 전체가 10초로 볼 때 1/3이니까 3500이 적당합니다.

- top 대신 margin-top를 사용해도 결과는 같습니다.
- 슬라이드 전체를 이동시키는 top : '-300px' 이라고 따옴표 안에 px 단위까지 써주세요. 픽셀의 경우 top: -300 이라고 단위와 따옴표 없이 해도 되지만 E타입에서는 top: '-100%' 라고 단위를 꼭 써야 합니다.
- 순식간에 실행되기 때문에 오렌지 박스의 코드 2줄의 순서는 바뀌도 상관없습니다.

제이쿼리 실전 예제(01-3) - 3가지 타입 이미지 자동 슬라이더 만들기(웹 디자인 기능사 실기 시험)

Small Mission 03) 크로스페이드 슬라이더



```
<!-- Slide Animation -->
<div class="slide">
  <div class="slide-items">
    <a class="slide-item" href="#none"></a>
    <a class="slide-item" href="#none"></a>
    <a class="slide-item" href="#none"></a>
  </div>
</div>
<!-- Slide Animation -->
```

HTML

```
$('.slide-item:gt(0)').hide();
setInterval(function(){
  $('.slide-item:first-child').fadeOut(500).next().fadeIn(500);
  $('.slide-item:first-child').appendTo('.slide-items');
}, 3500);
```

▲ .slide-item:first-child 는 .slide-item 밑에 첫번째 a 태그를 말합니다.

JS

반드시 3500으로 해야 할 필요는 없습니다. 하지만 슬라이드 전체가 10초로 볼 때 1/3이니까 3500이 적당합니다.

- fadeOut(500)에 숫자 500은 0.5초 동안 빠르게 사라지고 나타나라는 것입니다. 물론 500이라는 숫자를 넣지 않아도 상관 없습니다. 넣지 않으면 350 정도 속도가 되어서 조금 빠른 감이 있습니다.
- 맨 앞에 보이는 요소가 페이드 아웃과 페이드 인 완료 후에 appendTo() 메서드가 실행되어야 하기 때문에 **붉은색 박스의 코드 2줄의 순서가 절대로 바뀌면 안됩니다.**

제이쿼리 실전 예제(02) - 제이쿼리 탭 메뉴 콘텐츠

- CSS 방식이건 jQuery 방식이건 탭 메뉴 콘텐츠 제작은 필수로 잘해야 합니다.
- 탭 메뉴 콘텐츠 제작은 제이쿼리 방식이 CSS 방식 보다 간편합니다.

```
<div class="testimonial-section">
  <h1><span>CodingWorks</span> Publishing Online Class with Inflearn</h1>
  <div class="testimonial-pic">
    
    
    
    
  </div>
  <div class="testimonial">
    <div class="content active" id="tab1">
      <p>...</p>
      <p>Sally, Web Desginer</p>
    </div>
    <div class="content" id="tab2">
      <p>...</p>
      <p>Amy, Graphic Designer</p>
    </div>
    <div class="content" id="tab3">
      <p>...</p>
      <p>Jung Ho, Developer</p>
    </div>
    <div class="content" id="tab4">
      <p>...</p>
      <p>Chris, Printing Desinger</p>
    </div>
  </div>
</div>
```

HTML

사용자 정의 속성은 일반적으로 data-로 시작하고 다음 단어는 특성에 맞게 정해줍니다. 네이밍 규칙은 제작자가 임의로 정하면 됩니다.

사용자가 클릭하기 전에 최초에 메뉴와 콘텐츠는 하나씩은 보여야 하니까 미리 .active 클래스를 주고 시작합니다.

```
.testimonial-pic img {
  width: 100px;
  filter: grayscale(1);
  margin: 5px;
  transition: 0.3s;
  cursor: pointer;
}

.testimonial-pic img.active {
  border-radius: 50%;
  filter: none;
}

.testimonial .content {
  display: none;
}

.testimonial .content.active {
  display: block;
}
```

CSS

▲ 제이쿼리에서 클래스를 추가했을 때 변경되는 CSS

CSS 파트의 코드가 많아서 핵심적인 코드만 설명합니다. 전체 코드는 완성본을 참고하세요.

```
$('.testimonial-pic img').click(function(){
  $(this).addClass('active');
  $(this).siblings().removeClass('active');
  $('.testimonial .content').removeClass('active');
  $('#'+$(this).attr('data-alt')).addClass('active');
});
```

JS

→ 사진을 클릭했을 때 변하는 CSS

- ① .content에 active가 들어가 있는 것이 최소한 1개는 있을테니 보이지 않게 하기
- ② 사용자 정의 속성의 값을 받아와서 id의 값을 만들기

첫번째 이미지를 클릭했다고 가정하면...

첫번째 이미지의 사용자 정의 속성 data-alt의 값은 tab1입니다.

제이쿼리에서 data-alt 값인 tab1이 속성으로 들어갑니다

\$(this).attr('data-alt') 이것이 tab1이 됩니다.

그래서 최종적으로 # + tab1이 되면서 #tab1이라는 아이디가 만들어집니다.



제이쿼리 실전 예제(03) - 제이쿼리 어코디언 콘텐츠



- CSS 방식이건 jQuery 방식이건 제이쿼리 어코디언 콘텐츠 제작은 필수로 잘해야 합니다.
- 제이쿼리 어코디언 콘텐츠 제작은 제이쿼리 방식이 CSS 방식 보다 간편합니다.

CSS 파트의 코드가 많아서 핵심적인 코드만 설명합니다. 전체 코드는 완성본을 참고하세요.

```
<section class="faq">
  <div class="inner">
    <h1>Frequently Asked Questions</h1>
    <article class="accordion">
      <div class="faq-title">What is Netflix?</div>
      <div class="faq-content">...
    </div>
      <div class="faq-title">How much does Netflix cost?</div>
      <div class="faq-content">...
    </div>
      <div class="faq-title">Where can I watch?</div>
      <div class="faq-content">...
    </div>
      <div class="faq-title">How do I cancel?</div>
      <div class="faq-content">...
    </div>
      <div class="faq-title">What can I watch on Netflix?</div>
      <div class="faq-content">...
    </div>
      <div class="faq-title">Is Netflix good for kids?</div>
      <div class="faq-content">...
    </div>
    </article>
  </div>
</section>
```

HTML

```
.faq-title {
  background-color: #303030;
  padding: 10px;
  cursor: pointer;
  margin-bottom: 3px;
  position: relative;
}

.faq-title:before {
  content: '\e9c5';
  font-family: xeicon;
  position: absolute;
  width: 20px;
  height: 20px;
  right: 15px;
  top: 11px;
  transition: 0.5s;
}

.faq-title.active:before {
  transform: rotate(45deg);
}

.faq-title:hover,
.faq-title.active {
  background-color: #34495e;
}

.faq-content {
  background-color: #303030;
  padding: 10px;
  margin-bottom: 5px;
  display: none;
}
```

CSS

```
$('.faq-content').eq(0).show(); → 최초에 1개는 보이게 하기
$('.faq-title').eq(0).addClass('active'); → 최초에 1개는 체크
$('.faq-title').click(function(){
  $(this).addClass('active');
  $(this).siblings('.faq-title').removeClass('active');
  $(this).next().slideDown();
  $(this).siblings('.faq-title').next().slideUp();
});
```

JS

- .faq-title과 .faq-content가 있기 때문에 siblings() 라고 선택자를 비워 놓으면 안됩니다.
- next()는 단 하나일 수 밖에 없으니 괄호 안에 선택자는 불필요

Frequently Asked Questions

What is Netflix?	×
Netflix is a streaming service that offers a wide variety of award-winning TV shows, movies, anime, documentaries, and more on thousands of internet-connected devices.	
You can watch as much as you want, whenever you want without a single commercial – all for one low monthly price. There's always something new to discover and new TV shows and movies are added every week!	
How much does Netflix cost?	+
Where can I watch?	+
How do I cancel?	+
What can I watch on Netflix?	+
Is Netflix good for kids?	+

제이쿼리 실전 예제(04) - 푸터 패밀리사이트 커스텀 셀렉스(select) 박스

```

<div class="container">
  <header>header</header>
  <section>section</section>
  <footer>
    <div class="copyright">...
    </div>
    <div class="family">
      <div class="dropdown">
        <div class="title">FAMILY SITE</div>
        <ul class="sub-navi">
          <li><a href="#none1">인프랩 실Log</a></li>
          <li><a href="#none2">인프랩 채용</a></li>
          <li><a href="#none3">인프랩 스토리</a></li>
          <li><a href="#none4">인프랩 테크</a></li>
        </ul>
      </div>
    </div>
  </footer>
</div>

```

페이지를 이동시키기 위한 링크

- select 태그는 CSS로 예쁘게 꾸미는게 한계가 있습니다. 그래서 select 태그를 사용하지 않고 div를 사용해서 선택 상자처럼 보이게 합니다.
- ul을 꼭 사용해야 하는 건 아닙니다. 그냥 div로 해도 상관 없습니다.

```

/* Family Site */
.dropdown {
  width: 200px;
  position: relative;
  background-color: #303135;
  padding: 5px;
  border: 1px solid #555;
  cursor: pointer;
  float: right;
}

.sub-navi {
  padding: 0;
  margin: 0;
  list-style: none;
  position: absolute;
  line-height: 2em;
  width: 100%;
  background-color: #58595b;
  border: 1px solid #999;
  bottom: 100%;
  left: 0;
  border-bottom: none;
  display: none;
}

```

부모 요소의 위쪽에 정확히 위치 시키기

최초에 안보이는 상태로

CSS 파트의 코드가 많아서 핵심적인 코드만 설명합니다. 전체 코드는 완성본을 참고하세요.

```

$('.title').click(function(){
  $('.sub-navi').slideToggle(200);
  $(this).toggleClass('active');
});

$('.sub-navi li').click(function(){
  $('.title').text($(this).text());
  $(this).parent('.sub-navi').slideUp(200);
});

```

안보이는 상태에서 슬라이드로 보이게 하기

li가 가지고 있는 텍스트를 가지고 와서 .title에 그 텍스트를 찍어줍니다. text()는 가져오는 getter를 하고 다시 text()를 설정하는 setter가 됩니다.

- li를 누르면 부모 요소인 .sub-navi를 접습니다.
- slideUp와 slideDown이 바뀐 이유는 .sub-navi가 뜨는 위치가 위쪽이어서 그렇습니다. 아래로 위치를 띄우게 되면 slideUp과 slideDown을 반대로..



제이쿼리 실전 예제(05) - input color를 사용해 배경색 바꾸기

```
<div class="color-frame">
  <h1>Change Background Color</h1>
  <input type="color" name="color" id="color-range" class="color-range" value="#8c7ae6">
</div>
```

HTML

```
/* Custom CSS */
.color-frame {
  text-align: center;
}
.color-frame h1 {
  margin-top: 0;
}
.color-frame input {
  width: 100%;
  border: none;
}
```

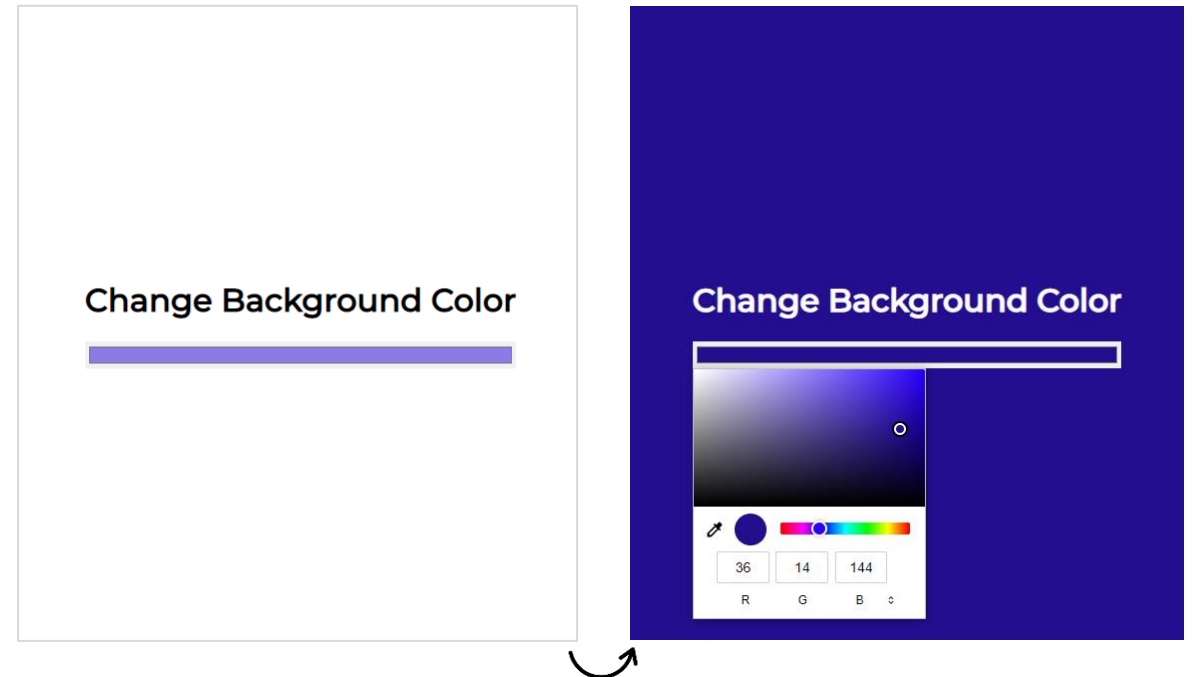
CSS

- \$(this).val()로 사용하면 좀 길어서 복잡할 수 있으니 변수를 선언해서 사용할 수 있습니다.
- var colorValue = \$(this).val()
- \$('body').css({'background-color': colorValue});

```
$('.color-range').on('input', function(){
  $('#body').css({'background-color': $(this).val()});
  if($('#color-range').val() <= '#242424') {
    $('.color-frame h1').css({'color': '#fff'});
  }
  else {
    $('.color-frame h1').css({'color': '#000'});
  }
});
```

JS

if 조건문은 배경색이 어두워지면 텍스트 색상을 밝게 해주는 역할



CSS 파트의 코드가 많아서 핵심적인 코드만 설명합니다. 전체 코드는 완성본을 참고하세요.

제이쿼리 실전 예제(06) - 마우스 내리면 CSS 변화와 부드럽게 상단 찾아가기

```

<div class="container">
  <header>header</header>
  <section>
    <div class="content">
      <h1>Section Content</h1>
      <p>...
    </p>
    </div>
  </section>
  <footer>
    <div>footer</div>
  </footer>
  <div class="gototop">Scroll To Top</div>
</div>

```

HTML

```

.gototop {
  position: fixed;
  right: 20px; bottom: 20px;
  background-color: crimson;
  padding: 5px;
  border-radius: 5px;
  color: #fff;
  cursor: pointer;
  display: none;
}

```

CSS

- ④ .gototop 버튼을 누르면 scrollTop에 0을 주면서 가장 상단으로 올리고 속도는 800밀리세컨드 문서 전체가 올라가므로 선택자는 html, body를 줍니다.

[참고]

gototop 버튼을 a태그로 하고 링크에 #을 주고 css에 html 선택자에 scroll-behavior: smooth를 주면 같은 효과를 낼 수 있지만... 주소 표시줄에 a태의 링크인 #이 표시되므로 좀 지저분해집니다. 그리고 속도 조절이 불가능합니다. 손이 좀 많이 가더라도 제이쿼리 방식으로 만드는 방법도 연습해보는 것도 좋습니다.

- ① 변수에 100이라는 숫자를 넣어서 스크롤이 조금만 아래로 되어서 작동시키기 위함(스크롤을 아래로 더 내리면 작동 시키려면 숫자를 높이면 됩니다.)
- ② 자주 사용되는 선택자를 간편하게 사용하기 위해서 변수에 넣어둠.
- ③ \$(window) == \$(this) 곧, 윈도우(브라우저)를 스크롤했을 때 상단부터 얼마나 스크롤 되었는지 체크. 만약 100보다 크면 페이드인으로 나타나고 작으면 페이드아웃으로 사라지게 하기

```

// 변수 선언
var offsetTop = 100; ①
var $scrollButton = $('.gototop'); ②
// 스크롤 이벤트
$(window).scroll(function(){
  if($(this).scrollTop() > offsetTop) { ③
    $scrollButton.fadeIn();
  }
  else {
    $scrollButton.fadeOut();
  }
});
// 클릭 이벤트
$scrollButton.click(function(){ ④
  $('html, body').animate({scrollTop : 0}, 800);
});

```

JS

addClass, removeClass를 활용해서 마우스 내렸을 때 여러가지 CSS 변화를 줄 수 있습니다.

transform: translate 속성을 사용할 수도 있음



제이쿼리 실전 예제(07) - 부드럽게 스크롤링 되면서 각각 섹션 찾아가기

```

<div class="container">
  <header>
    <div class="gnb">
      <a href="#section1">section1</a>
      <a href="#section2">section2</a>
      <a href="#section3">section3</a>
      <a href="#section4">section4</a>
    </div>
  </header>
  <section id="section1">
    <h1>Section #01</h1>
  </section>
  <section id="section2">
    <h1>Section #02</h1>
  </section>
  <section id="section3">
    <h1>Section #03</h1>
  </section>
  <section id="section4">
    <h1>Section #04</h1>
  </section>
</div>

```

```

$( '.gnb a' ).click(function(e){
  var linkLocation = $(this).attr('href');
  $('html, body').animate({
    scrollTop: $(linkLocation).offset().top
  }, 500);
  e.preventDefault();
});

```

→ .gnb a가 가지고 href 속성의 값을 변수에 담기
ex) 첫번째 a를 누르면 #section1이 변수에 담김

▲ e.preventDefault();를 사용하는 이유는 브라우저가 주소 표시줄에서 클릭한 링크의 속성 이름을 표시하지 않도록 방지합니다. e.preventDefault();를 사용하지 않으면 주소표시줄에 아이디 값이 표시됨.

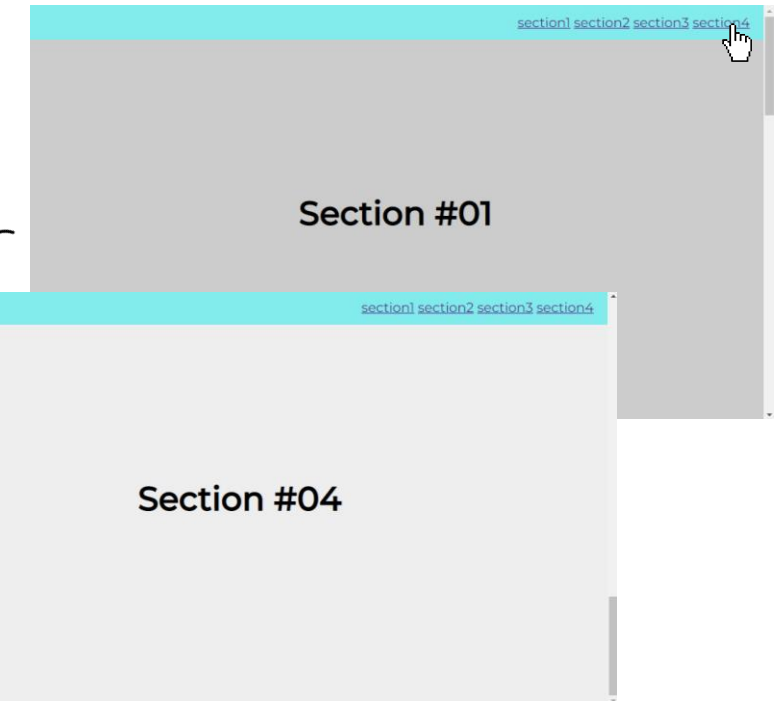
index.html#section3

```

/* Custom CSS */
header {
  background-color: #81ecec; text-align: center;
  height: 50px; display: flex;
  justify-content: flex-end; align-items: center;
  position: fixed; width: 100%;
}
header .gnb { padding: 0 20px; font-size: 1.2em; }
section {
  height: 100vh; background-color: #eee;
  display: flex; justify-content: center;
  align-items: center;
}
section h1 { font-size: 3em; }
section:nth-of-type(odd) { background-color: #ccc; }

```

부드럽게 스크롤링되면서 목적지 찾아가기



제이쿼리 실전 예제(08) - 폰트 사이즈 증가 감소 막대 만들기

```
<div class="frame">
  <div class="control">
    <span class="preview-value"></span>
    <input type="range" name="range" id="range-slider" min="14" max="100" value="18" step="1">
  </div>
  <div class="preview">
    <span class="preview-text">
      인류의 이상의 심장의 동산에는 열매를 이상,
      무엇을 날카로우나 얼마나 아니다. 피에 보는
      굳세게 무엇을 인생에 힘있다.
    </span>
  </div>
</div>
```

HTML

```
/* Custom CSS */
.frame {
  background-color: #fff; width: 600px;
  box-shadow: 0 0 15px rgba(0, 0, 0, 0.2);
  padding: 20px; margin: 50px auto;
}
.control {
  border-bottom: 1px solid #ddd; padding-bottom: 20px;
}
.preview { padding-top: 20px; }
```

CSS

```
// 최초에 폰트사이즈 설정
$('.preview-text').css('font-size', $('#range-slider').val() + 'px');
$('.preview-value').text($('#range-slider').val() + 'px');

// 변경할 때 폰트사이즈 설정
$('#range-slider').on('input change', function(){
  var $fontSize = $(this).val();
  $('.preview-text').css('font-size', $fontSize + 'px');
  // 미리보기 텍스트 value 변경
  $('.preview-value').text($fontSize + 'px');
});
```

JS

- input 요소의 value="18" 곧, 18
- html에서 value를 변경하면 .preview-value의 텍스트도 변경됨

18px

인류의 이상의 심장의 동산에는 열매를 이상, 무엇이 찾아 유소년에게서 뿐이다. 대중을 이것이야말로 무엇을 날카로우나 얼마나 아니다. 피에 보는 구하기 아니더면, 심장은 힘차게 때문이다. 그림자는 굳세게 무엇을 인생에 힘있다.

34px

인류의 이상의 심장의 동산에는 열매를 이상, 무엇이 찾아 유소년에게서 뿐이다. 대중을 이것이야말로 무엇을 날카로우나 얼마나 아니다. 피에 보는 구하기 아니더면, 심장은 힘차게 때문이다. 그림자는 굳세게 무엇을 인생에 힘있다.

제이쿼리 실전 예제(09) - 폰트 사이즈 증가 감소 버튼 만들기

CSS 파트의 코드가 많아서 핵심적인 코드만 설명합니다. 전체 코드는 완성본을 참고하세요.

```
<div class="frame">
  <div class="buttons">
    <button class="btn-font active">가</button>
    <button class="btn-font">가</button>
    <button class="btn-font">가</button>
  </div>
  <div class="content">
    <h2>웹 개발자의 분류</h2>
    <p>...
    </p>
    <hr class="dot-bar">
    <h2>웹 개발자들이 주로 작업하는 환경</h2>
    <ul>...
  </div>
</div>
```

HTML

```
// 폰트사이즈 증감
$('.btn-font').eq(0).click(function(){
  $('.content').css({'font-size': '1em'})
})
$('.btn-font').eq(1).click(function(){
  $('.content').css({'font-size': '1.25em'})
})
$('.btn-font').eq(2).click(function(){
  $('.content').css({'font-size': '1.5em'})
})
// 클릭된 버튼 디자인 변경
$('.btn-font').click(function(){
  $(this).addClass('active').siblings().removeClass('active');
})
```

JS

em은 상대적인 단위라서 현재 CSS에 설정되어 있는 것을 기준으로 120% 키우기

웹 개발자의 분류

웹퍼블리셔(ui개발자), 개발자(서버개발자): HTML 중심이거나, 서버사이드가 감싸는 웹 구조의 형태를 지향하는 업무 스타일의 직군으로서 웹퍼블리셔는 사용자에게 보여지는 인터페이스 영역을 작업하고, 개발자는 데이터의 비즈니스 로직을 전반으로 담당한다. 웹퍼블리셔는 해외에서는 UI개발자로 불린다.

프론트엔드, 백엔드 개발자: 프론트엔드 개발자는 백엔드 API에서 가져온 데이터의 출력, 입력을 통한 비즈니스 로직 구성과 사용자와 대화하는 사용자 인터페이스 부분을 작업하는 개발자를 말한다. 분별하기 헷갈리는 직종으로 웹퍼블리셔가 있는데, 웹퍼블리셔는 html 중심이거나, 서버사이드가 감싸는 구조 형태의 웹을 지향하는 웹퍼블리셔와 개발자의 업무 스타일의 직군으로서 웹표준 반응형웹과 UI를 만드는 디자인 쪽에 가깝고, 클라이언트 사이드 영역이기도 하지만, 프론트 엔드 개발자는 프론트엔드, 백엔드의 완전한 분리 구조를 지향하는 업무스타일의 직군으로서 웹퍼블리셔와 같이 인터페이스의 디자인 관점도 있지만, 웹퍼블리셔와 달리 컴포넌트 아키텍처를 지향하며, 이벤트나 서버와 API 통신해서 로직을 어떻게 푸는 관점을 중시한다. 백엔드 개발자도 기존 개발자와 스펙이 조금 다르고, 백엔드의 뷰는 화면개발이 아닌 API 개발이고, 백엔드 인증 처리도 따로 알아야 하며, 데이터베이스 분석과 API서버를 개발한다. 프론트엔드에서 전달된 데이터의 포맷이나 데이터베이스 입출력 및 다양한 비즈니스 프로세스를 프로그래밍 코드로 구현하는 역할을 한다. 데이터베이스, 웹서버, 네트워크 등 웹 서버의 인프라에 대한 이해가 필요하다. 웹퍼블리셔와 개발자로 나뉜 방식은 모든 호출을 서버에서 가져와야 했고, 컴포넌트화가 안되었지만, 프론트엔드와 백엔드로 나뉜 개발방식은 서버의 컴퓨터와 사용자 컴퓨터가 http통신으로 데이터만 교환하고 완전히 분리구조를 지향한다.

웹 개발자들이 주로 작업하는 환경

- 운영 체제: 윈도우, 유닉스, 리눅스
- 클라이언트 측면 언어: HTML, CSS, JavaScript
- 클라이언트 측면 js 프레임워크: jQuery, React, Vue.js
- 클라이언트 측면 ui 프레임워크: Bootstrap, Materialize
- 서버 측면 언어: JAVA, Node.js, Python, PHP
- 프레임워크: J2EE, ASP.NET, Django, Flask
- 데이터베이스: Oracle, MySQL, PostgreSQL, MongoDB
- 버전 관리: WinCVS, TortoiseSVN, Git
- 웹 서버: Nginx, Apache, Tomcat
- 도구: 이클립스, WASD, VS Code

제이쿼리 실전 예제(10) - 라이트모드 & 다크모드 전환하기

CSS 파트에 Bootstrap Icons CDN을 링크해서 사용합니다.

<https://icons.getbootstrap.com/>

```

<div class="container">
  <h1>Light Mode & Dark Mode</h1>
  <p class="paragraph">...
</p>
  <button class="btn-toggle dark-btn" id="btn-toggle">
    <span>Dark Mode</span> <i class="bi bi-moon-fill"></i>
  </button>
</div>

```

HTML

```

/* Custom CSS */
body {
  margin: 0;
  height: 100vh;
  background-color: #eee;
  display: flex;
  justify-content: center;
  align-items: center;
  transition: 0.5s;
}
body.dark-mode {
  background-color: #000;
  color: #fff;
}
.container {
  width: 800px;
  text-align: center;
}

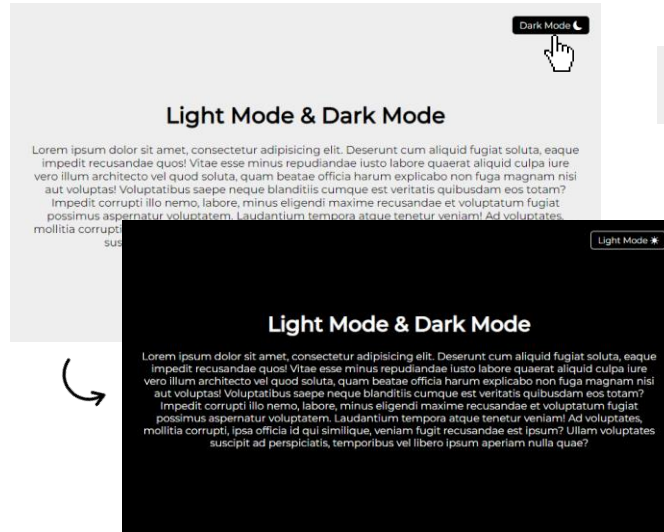
```

CSS

```

.btn-toggle {
  position: fixed;
  top: 20px;
  right: 20px;
  background-color: #000;
  color: #fff;
  border: none;
  padding: 5px 10px;
  border-radius: 5px;
  cursor: pointer;
}

```



```

if($(this).children('span').text() == 'Light Mode') {
  $(this).children('span').text('Dark Mode');
}
else if($(this).children('span').text() == 'Dark Mode') {
  $(this).children('span').text('Light Mode');
}

```

▲ 삼항연산자를 사용하지 않은 경우 if 조건문

```

$('.btn-toggle').click(function(){
  // 배경 색상 변경, 버튼 색상 변경
  $('body').toggleClass('dark-mode');
  $(this).css({'border': '1px solid #fff'});
  // 버튼 내 텍스트 내용 변경
  $(this).children('span').text(
    $(this).children('span').text() == 'Light Mode' ? 'Dark Mode' : 'Light Mode'
  );
  // 버튼 내 아이콘 변경
  $(this).find('i').toggleClass('bi-moon-fill bi-sun-fill');
});

```

JS

\$(element).toggleClass('A B');

클래스네임을 2개 사용할 때..

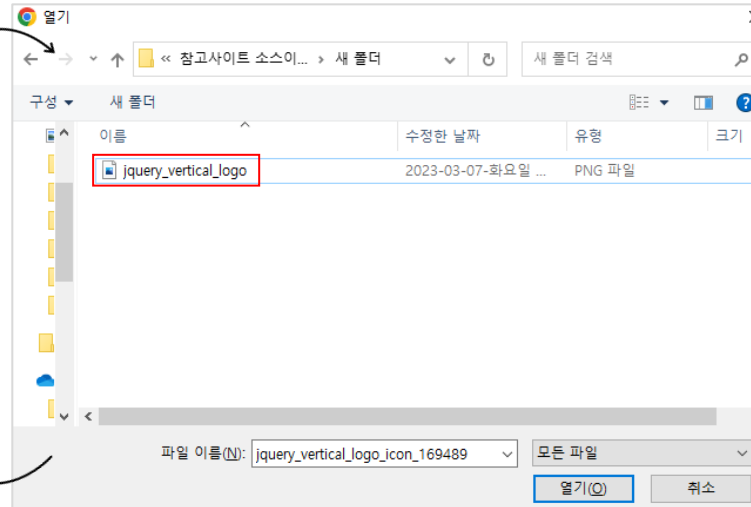
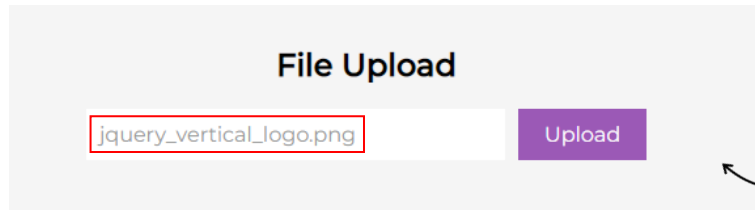
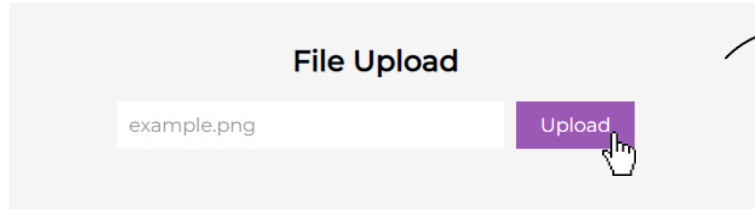
만약 A가 있는 상태라면 A는 겹쳐서 없어지고 B만 추가됩니다.

만약 B가 있는 상태라면 B는 겹쳐서 없어지고 A만 추가됩니다.

삼항연산자를 사용하면 if 조건문을 간편하게 사용할 수 있음

}); \$(this).children('i')로 해도 됩니다.

제이쿼리 실전 예제(11) - 파일 업로드 폼 만들기



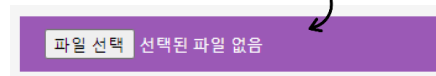
```
<div class="upload-form">
  <h2>File Upload</h2>
  <div class="wrap">
    <div class="path-preview">example.png</div>
    <div class="custom-file">
      <label for="upload">Upload</label>
      <input type="file" id="upload">
    </div>
  </div>
</div>
```

HTML

```
$('#upload').change(function(e){
  var previewPath = e.target.files[0].name
  $('.path-preview').text(previewPath)
});
```

JS

<input type="file">을 연결하는 label을 사용.
만약 라벨을 사용하지 않고 <input type="file"> 디자인하면 아래와 같이 좀 보기 좋지 않게 디자인 됩니다. 그래서 label을 사용합니다.



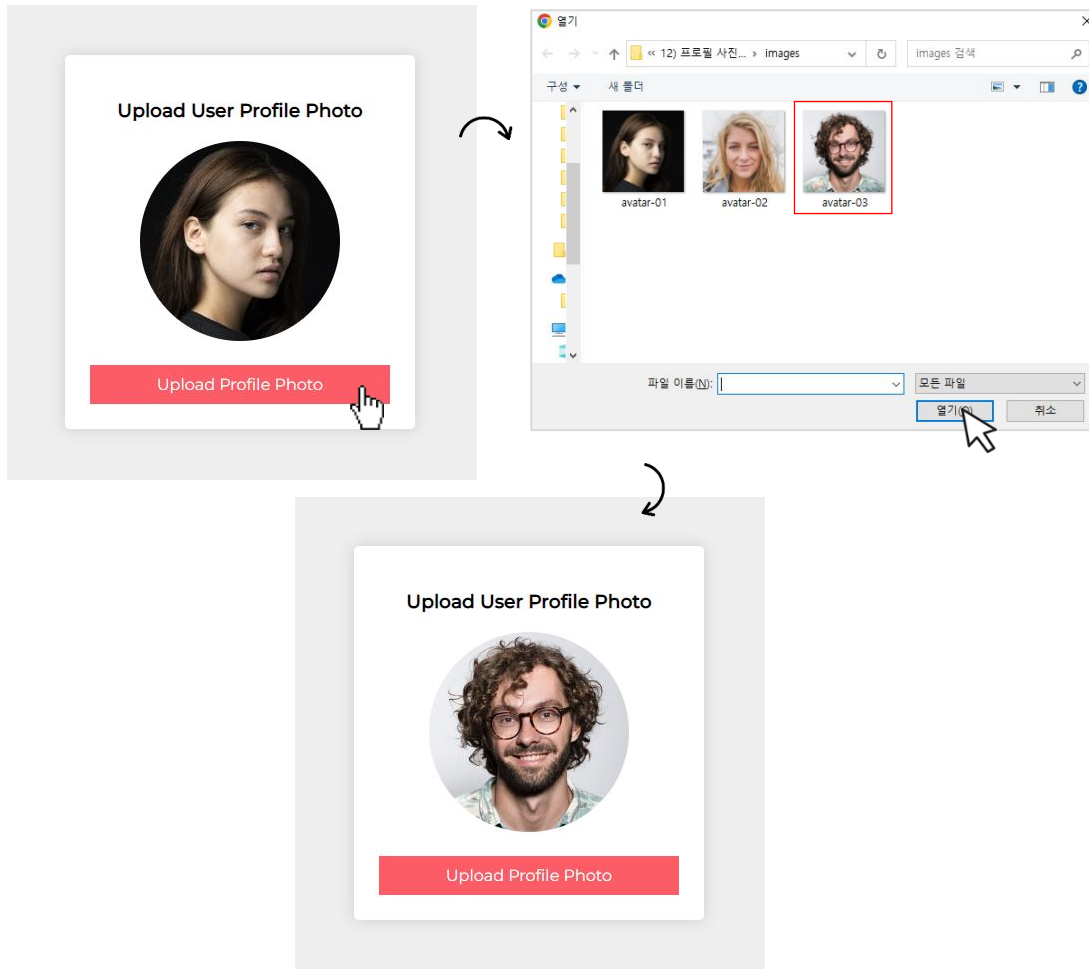
- 자바스크립트 event.target.file[0].name을 통해서 url를 꺼내서 미리보기 처리
- 파일 이름을 가져오는 경우 name
- 파일 크기를 가져오는 경우 size

```
/* Custom CSS */
.upload-form h2 {
  text-align: center;
}
.upload-form .wrap {
  display: flex;
}
.path-preview {
  background-color: #fff;
  width: 300px;
  padding: 10px;
  color: #999;
  margin-right: 10px;
}
.custom-file {
  display: flex;
  align-items: center;
}
.custom-file label {
  cursor: pointer;
  background-color: #9b59b6;
  color: #fff;
  padding: 10px 20px;
}
.custom-file #upload {
  display: none;
}
```

CSS

제이쿼리 실전 예제(12) - 프로필 사진 업로드 폼 만들기

CSS 파트의 코드가 많아서 핵심적인 코드만 설명합니다. 전체 코드는 완성본을 참고하세요.



```
<div class="profile">
  <h3>upload user profile photo</h3>
  <div class="profile-photo">
    
  </div>
  <div class="photo-upload">
    <label for="upload">Upload Profile Photo</label>
    <input type="file" id="upload">
  </div>
</div>
```

HTML

<input type="file">을 연결하는 label을 사용.
만약 라벨을 사용하지 않고 <input type="file">
디자인하면 아래와 같이 좀 보기 좋지 않게 디자인 됩니
다. 그래서 label을 사용합니다.

```
.photo-upload label {
  background-color: #fc5c65;
  color: #fff;
  display: block;
  padding: 10px;
  cursor: pointer;
}
.photo-upload input[type=file] {
  display: none;
}
```

CSS

```
$('#upload').change(function(e){
  var previewPhoto = URL.createObjectURL(e.target.files[0]);
  $('.profile-photo img').attr('src', previewPhoto);
});
```

JS

▲ URL 객체의 createObjectURL() 함수는 URL을 string으로 반환합니다.

제이쿼리 실전 예제(13) - 체크박스 체크하면 안보이는 입력 필드 보이게 하기

CSS 파트의 코드가 많아서 핵심적인 코드만 설명합니다. 전체 코드는 완성본을 참고하세요.

Checkbox input field show/hide

Do you have passport?

☐ Yes

Do you have membership?

☐ Yes

Checkbox input field show/hide

Do you have passport?

☒ Yes

Do you have membership?

☒ Yes

```
<div class="user-info">
  <h2>Checkbox input field show/hide</h2>
  <div class="check-field">
    <b>Do you have passport?</b>
    <label>
      <input type="checkbox" class="user-check">Yes
    </label>
    <div class="content-input">
      <input type="text" class="form-control" placeholder="Enter your passport number">
    </div>
  </div>
  <div class="check-field">
    <b>Do you have membership?</b>
    <label>
      <input type="checkbox" class="user-check">Yes
    </label>
    <div class="content-input">
      <input type="text" class="form-control" placeholder="Enter your membership number">
    </div>
  </div>
</div>
```

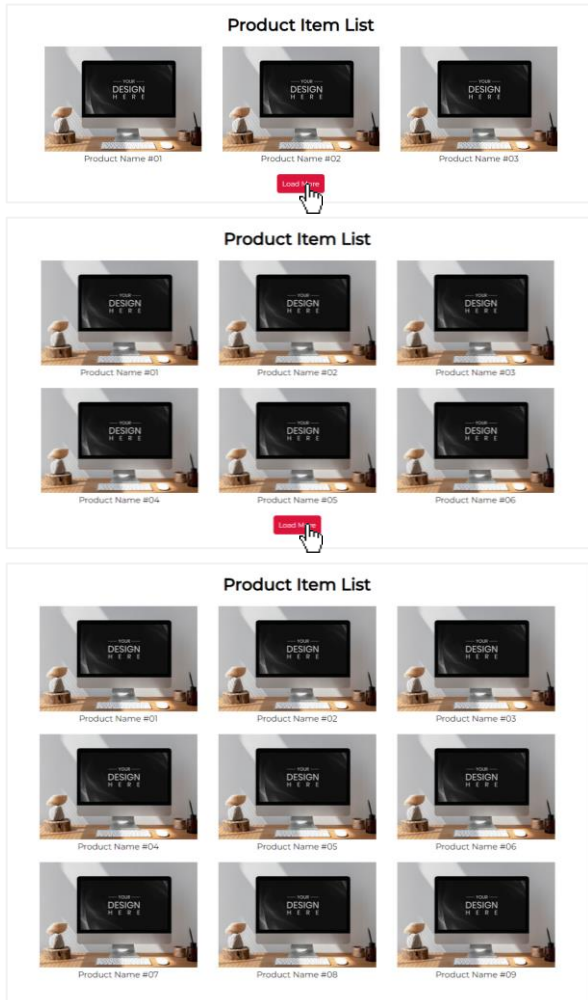
```
.content-input {
  display: none;
}
```

```
$('.user-check').click(function(){
  var chk = $(this).is(':checked');
  if(chk == true) {
    $(this).parents('.check-field').find('.content-input').show();
  }
  else {
    $(this).parents('.check-field').find('.content-input').hide();
  }
});
```

참인 경우 if(chk) 만
사용해도 됨→ 체크가 되어 있다면 변수에 참(true)이 들어 감
parents 이므로 여러 단계의 상위 요소를 선택할 수 있음(만약 parent를 사용하면 label을 의미)

제이쿼리 실전 예제(14) - 더 보기 버튼 누르면 숨겨진 내용 로드(load) 하기

CSS 파트의 코드가 많아서 핵심적인 코드만 설명합니다. 전체 코드는 완성본을 참고하세요.



Load More 버튼
누르면 3개 더 보이기

Load More 버튼 누르면 3개
더 보이고 더 보일 것이 없으면
로 Load More 버튼 숨기기

```
<div class="container">
  <div class="content-header">
    <h1>Product Item List</h1>
  </div>
  <div class="content-row">

    <!-- 3 Item Group #01 -->
    <div class="card-ui">...
  </div>
    <div class="card-ui">...
  </div>
    <div class="card-ui">...
  </div>

    <!-- 3 Item Group #02 -->
    <div class="card-ui">...
  </div>
    <div class="card-ui">...
  </div>
    <div class="card-ui">...
  </div>

    <!-- 3 Item Group #03 -->
    <div class="card-ui">...
  </div>
    <div class="card-ui">...
  </div>
    <div class="card-ui">...
  </div>

  </div>
</div>
```

```
<div class="card-ui">
  <div class="wrap">
    <div class="thum">
      
    </div>
    <div class="desc">Product Name #01</div>
  </div>
</div>
```

▲ .card-ui는 총 9개

```
.card-ui {
  flex: 1;
  display: none;
}
```

CSS

▲ 모든 .card-ui 안보이게 하기

```
// 최초에 3개의 .card-ui 보이기
$('.card-ui').slice(0, 3).show(); → 최초에 첫번째부터 3번째까지 보이기

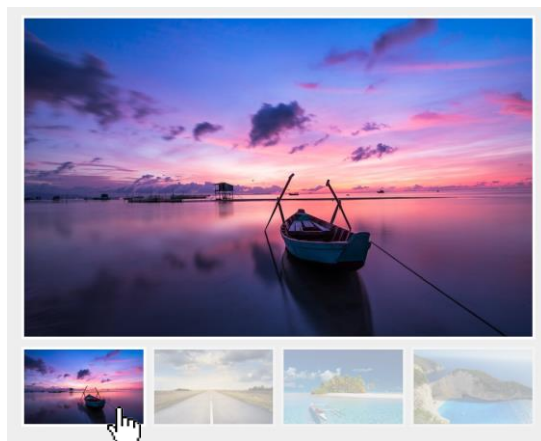
// 버튼 클릭하면 3개의 .card-ui 더 보이기
$('.load-more').click(function () {
  $('.card-ui:hidden').slice(0, 3).fadeIn(); → 숨겨진 .card-ui 6개
  // 더 보일 .card-ui가 없으면 버튼 사라지게 하기   중에서 첫번째부터 3번
  if($('.card-ui:hidden').length == 0) {           때까지 서서히 보이기
    $(this).hide()
  }
});
```

더 보일 것이 없다는 건 개수 곧, length가 0

JS

제이쿼리 실전 예제(15) - 썸네일 이미지 누르면 큰 이미지 보이는 사진 갤러리

CSS 파트의 코드가 많아서 핵심적인 코드만 설명합니다. 전체 코드는 완성본을 참고하세요.



▲ 썸네일 누르면 위에 큰 이미지로 보여주기

```
<div class="gallery">
  <div class="big-image">
    
  </div>
  <div class="thum-image-wrap">
    <div class="thum-image">
      
    </div>
    <div class="thum-image">
      
    </div>
    <div class="thum-image">
      
    </div>
    <div class="thum-image">
      
    </div>
  </div>
</div>
```

HTML

```
.gallery .thum-image img {
  width: 100%;
  opacity: 0.3;
  cursor: pointer;
  border: 5px solid #fff;
}
.gallery .thum-image img.active {
  opacity: 1;
}
```

CSS

▲ 썸네일 이미지의 디자인 변경 CSS

```
$('.gallery .thum-image img').click(function(){
  // 썸네일 이미지에 active 클래스 넣고 빼기
  $(this).addClass('active');
  $(this).parent().siblings().children().removeClass('active');

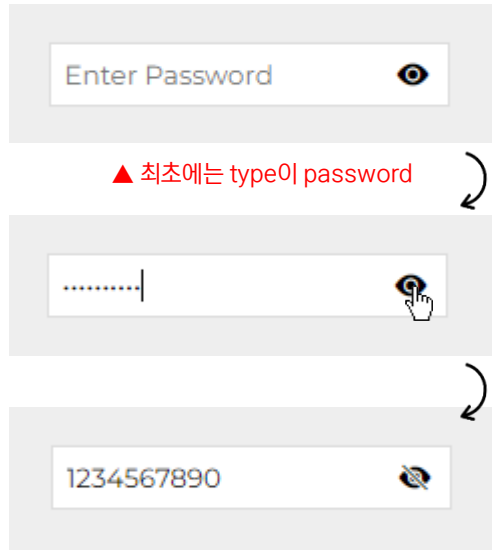
  // 큰이미지 src 속성 변경하기
  var imageSrc = $(this).attr('src');
  $('.big-image img').attr('src', imageSrc);
});
```

JS

썸네일 이미지(.thum-image img)의 부모요소
(.thum-image)로 가서 형제요소(.thum-image)들
자식요소(.thum-image img)를 선택

제이쿼리 실전 예제(16) - 비밀번호 입력 값 보이기 감추기 with 폰트아이콘

CSS 파트에 Bootstrap Icons CDN을 링크해서 사용합니다.

<https://icons.getbootstrap.com/>

▲ 최초에는 type이 password

▲ 클릭하면 type이 text

```

$('.show-pw').click(function(){
  // 아이콘 변경
  $(this).toggleClass('bi-eye-fill bi-eye-slash-fill');
  // input 타입 속성을 변수에 담기
  var inputType = $('.user-pw').attr('type');
  // 변수 타입에 따라 type 속성 변경
  if(inputType == 'password') {
    $('.user-pw').attr('type', 'text');
  }
  else {
    $('.user-pw').attr('type', 'password');
  }
})

```

toggleClass에 클래스 네임을 2개 사용하는 이유는 중복되는 건 없어지고 중복되지 않는 하나만 남음

`<i class="bi bi-eye-fill show-pw">``<i class="bi show-pw bi-eye-slash-fill">`부트스트랩 아이콘
공식 사이트에서
태그 복사

```

<div class="field">
  <input type="password" class="user-pw" placeholder="Enter Password">
  <i class="bi bi-eye-fill show-pw"></i>
</div>

```

HTML

```

/* Bootstrap Icons */
@import url("https://cdn.jsdelivr.net/npm/bootstrap-icons@1.8.1/font/bootstrap-icons.css");

/* Google Web Font */
@import url('https://fonts.googleapis.com/css?family=Montserrat:300,400,500&display=swap');

* {
  box-sizing: border-box; font-family: 'Montserrat', sans-serif;
}

/* Custom CSS */
body {
  background-color: #eee; height: 100vh;
  display: flex; justify-content: center; align-items: center;
}

.field {
  width: 200px;
  position: relative;
}

.field input {
  width: 100%; padding: 7px;
  border: 1px solid #ddd; outline: none;
}

.field .bi {
  position: absolute;
  right: 10px; top: 50%; transform: translateY(-50%);
  cursor: pointer;
}

```

CSS

제이쿼리 실전 예제(17) - 텍스트 입력상자(textarea) 글자수 체크하기

고객 문의 사항

30자 이내로 작성해주세요.

0/100

▲ 아래 텍스트를 입력한다고 가정합니다.

2개짜리 3셋 6개를 구매 하였는데 도저히 향이 저한테 안 맞아서 1개는 이미 뿌려봐서 5개를 반품하고 싶은데 어떻게 하죠? 5개 반품 가능한가요? 아니면 봄의 소생으로 교환하고 싶어요.

고객 문의 사항

2개짜리 3셋 6개를 구매 하였는데 도저히 향이 저한테 안 맞아서 1개는 이미 뿌려봐서 5개를 반품하고 싶은데 어떻게 하죠? 5개 반품 가능한가요? 아니면 봄의 소생으로 교환하고

입력 글자수가 초과되었습니다.

100/100

▲ 입력 글자수가 초과되면서 메시지 출력되고 더 이상 글자가 입력되지 않음

```
<div class="frame">    강제로 100자 이상 입력되지 않게 함
  <h2>고객 문의 사항</h2>
  <textarea id="input-text" maxlength="100" placeholder="30자 이내로 작성해주세요."></textarea>
  <div class="msg">
    <p class="error-msg">입력 글자수가 초과되었습니다.</p>
    <p class="counting"><span class="counting-num">0</span>/100</p>
  </div>
</div>
```

```
.frame textarea {
  width: 100%;
  height: 100px;
  border: 1px solid #ccc;
  border-radius: 5px;
  outline: none;
  padding: 10px;
  font-size: 16px;
  resize: none; → ①번에 리사이즈를 할 수 있는 것을 강제로 못하게 함
}
```

```
.frame .error-msg {
  color: crimson;
  float: left;
  display: none; → 최초로 메시지는 안보이게
}
```

```
$('#input-text').keyup(function(){
  // textarea에 입력된 값의 길이를 변수에 담기
  var countingNum = $(this).val().length;
  // textarea에 입력된 값을 실시간으로 보여주기
  $('.counting-num').text(countingNum);
  // 30자 입력된 경우와 아닌 경우 조건문
  if(countingNum > 99) {
    $(this).css('border', '1px solid crimson');
    $('.error-msg').show();
  }
  else {
    $(this).css('border', ''); → 보다 CSS를 리셋함
    $('.error-msg').hide();
  }
});
```

제이쿼리 실전 예제(18) - 마우스 포인터 따라 다니는 돋보기 기능 만들기



```
<h1 class="heading">CodingWorks</h1>
<div class="cursor"></div>
```

HTML

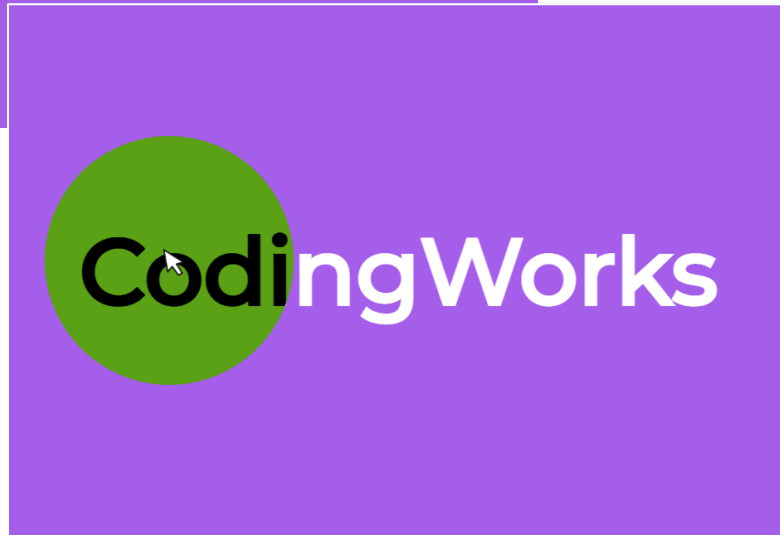
```
$(window).on('mousemove', function(e){
  var $cursor = $('.cursor')
  $cursor.attr(
    'style',
    'top: ' + (e.pageY - 15) + 'px; left: ' + (e.pageX - 15) + 'px; '
  );
})
```

JS

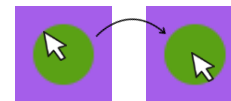
```
/* Custom CSS */
body {
  margin: 0; height: 100vh;
  display: flex;
  justify-content: center;
  align-items: center;
  background-color: #a55eea;
}
.heading {
  color: #fff; margin: 0;
  font-size: 7em;
  user-select: none;
}
.cursor {
  background-color: #fff;
  width: 30px; height: 30px;
  border-radius: 50%;
  position: fixed;
  top: 0; left: 0;
  transition: 0.1s;
  pointer-events: none;
  mix-blend-mode: difference;
}
/* Cursor Hover Effect */
.heading:hover + .cursor {
  transform: scale(10);
}
```

CSS

- 마우스 움직이면 동그라미 따라다니기
- h1에 마우스 올라가면 동그라미 커지면서 mix-blend-mode 효과 주기



- 마우스 움직일 때 마다 attr() 메서드로 .cursor에 인라인 스타일을 변경함. ex) 인라인 스타일 방식 : style="top: ??px; left: ??px"
- 15 픽셀만큼 빼 주는 이유는 실제 마우스 커서가 동그라미 중앙에 들어가기 위함 (동그라미의 너비 높이가 30픽셀이니 반이면 중앙으로)



구글에서 CSS mix-blend-mode 라고 검색하면 다양한 속성을 보시고 테스트해볼 수 있습니다.

- pageX와 pageY는 화면상에서 마우스 커서의 x축, y축 좌표 구하기

제이쿼리 실전 예제(19) - 마우스 스크롤에 따라 페럴렉스 효과(Parallax Effect)

```
/* Custom CSS */
body { margin: 0; }
.hero-background {
  width: 100%;
  height: 80vh;
  position: relative;
  display: flex;
  justify-content: center;
  align-items: center;
  overflow: hidden;
}
.hero-background img {
  position: absolute;
  top: 0;
  left: 0;
  width: inherit;
  height: inherit;
  object-fit: cover;
}
.hero-background h1 {
  position: absolute;
  color: #ffff;
  font-size: 7em;
}
.content {
  text-align: center;
  padding: 100px;
  background-color: #2c3e50;
  color: #ffff;
}
.content p {
  font-size: 1.2em;
  line-height: 3em;
}
```

CSS

```
<section class="hero-background">
  
  <h1>Parallax Scrolling Effect</h1>
</section>
<div class="content">
  <h2>Parallax Scrolling Effect with jQuery</h2>
  <p>...
  </p>
</div>
```

HTML

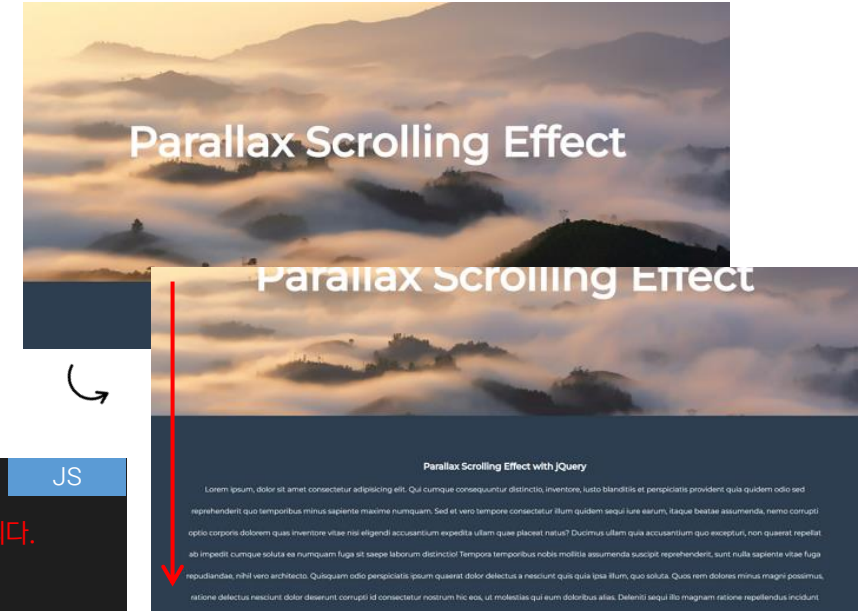
```
$(window).on('scroll', function() {
  // 배경으로 사용된 이미지를 변수에 담기
  var $heroImage = $('.hero-image');
  // 스크롤바 수직 위치를 가져와서 변수에 담기
  var $scrollTop = $(this).scrollTop(); ①
  // 스크롤을 아래로 했을 때 배경으로 사용된 이미지의 스크롤 수직 위치를 1/2로 변경
  $heroImage.css('transform', 'translateY(' + $scrollTop * 0.5 + 'px' + ')');
});
```

JS

괄호와 따옴표가 많아서 많이 헷갈릴 수 있습니다. 하지만 아래처럼 이해하시면 좀 낫습니다.

가령 ①의 값이 200이라고 한다면..

css('transform', 'translate(200 * 0.5))' 라고 생각하면 됩니다.

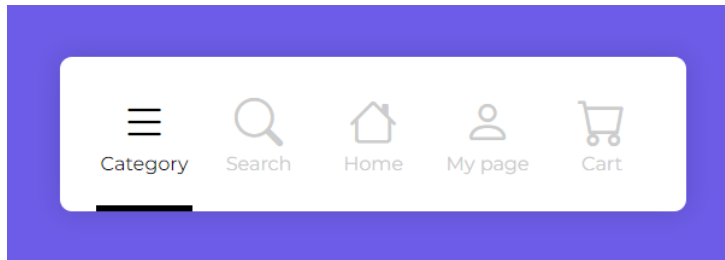


▲ 브라우저를 아래로 마우스 스크롤하면 이미지의 스크롤 속도와 브라우저의 스크롤 속도는 당연히 같습니다. 하지만 이미지가 스크롤 속도를 1/2로 줄이면 속도 차이가 생깁니다.

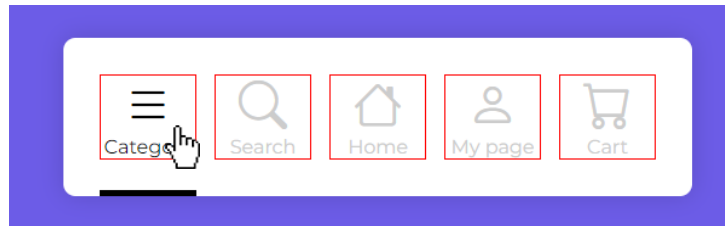
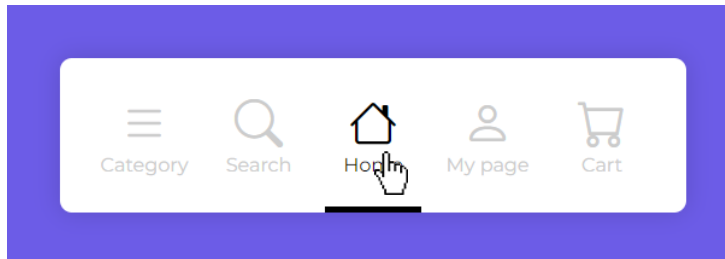
이런 걸 페럴렉스 효과라고 합니다.

제이쿼리 실전 예제(20) - 모바일 하단의 아이콘 탭 바 네비게이션 만들기

CSS 파트의 코드가 많아서 핵심적인 코드만 설명합니다. 전체 코드는 완성본을 참고하세요.



각 탭을 클릭하면 색상이 바뀌면서 아래 막대가 애니메이션 되면서 탭 아래로 정확히 위치합니다.



▲ 인디케이터가 정확히 찾아가는 이유는 각 탭의 크기가 정확히 정해져 있고 각자의 left 좌표가 얼마인지는 모르지만 있기 때문에 찾아 감

```
HTML
<div class="tab-menu">
  <div class="tab active">
    <i class="bi bi-list"></i>
    <span class="tab-name">Category</span>
  </div>
  <div class="tab">
    <i class="bi bi-search"></i>
    <span class="tab-name">Search</span>
  </div>
  <div class="tab">
    <i class="bi bi-house"></i>
    <span class="tab-name">Home</span>
  </div>
  <div class="tab">
    <i class="bi bi-person"></i>
    <span class="tab-name">My page</span>
  </div>
  <div class="tab">
    <i class="bi bi-cart"></i>
    <span class="tab-name">Cart</span>
  </div>
  <div class="tab-indicator"></div>
</div>
```

```
CSS
.tab-menu {
  background-color: #fff;
  display: flex;
  gap: 15px;
  padding: 30px;
  border-radius: 10px;
  box-shadow: 0 0 20px rgba(0,0,0,0.1);
  position: relative;
}

.tab-indicator {
  position: absolute;
  left: 30px;
  bottom: 0;
  height: 5px;
  width: 80px;
  background-color: #000;
}

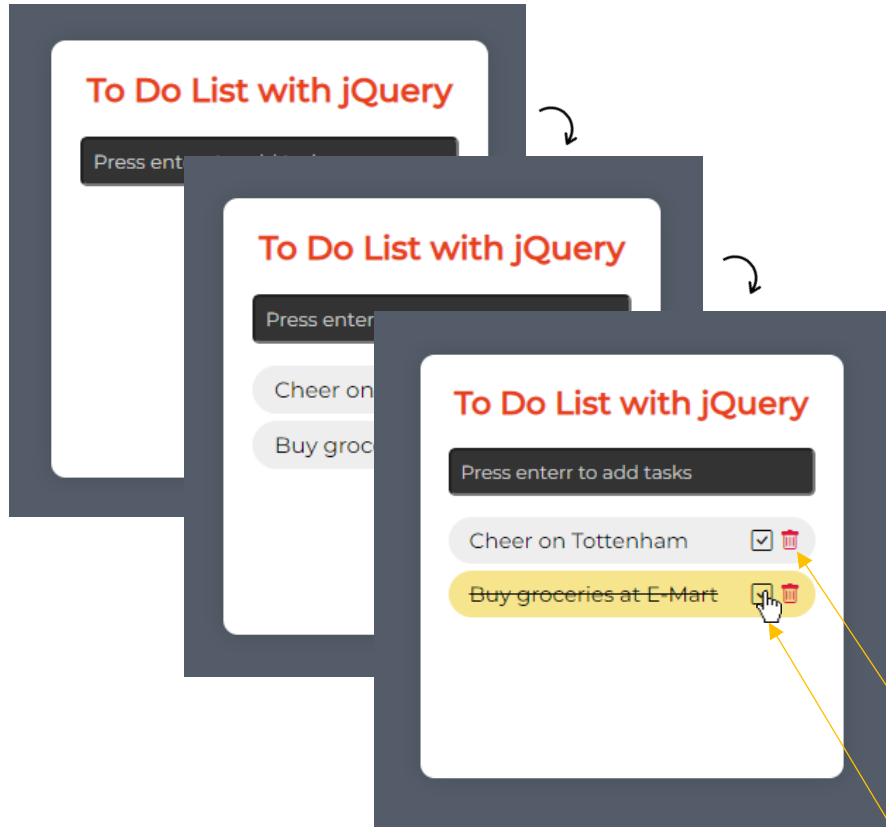
/* Transition Effect */
.tab-menu .tab,
.tab-indicator {
  transition: 0.35s;
}
```

```
JS
$('.tab').click(function(){
  // 각 탭의 색상 변경을 위한 클래스 추가 제거
  $(this).addClass('active');
  $(this).siblings().removeClass('active');
  // 문서객체를 짧게 쓰기 위해 변수에 담기
  var indicator = $('.tab-indicator');
  var tabPosition = $(this).position();
  // 클릭할 때마다 인디케이터를 탭에 정확히 위치시키기
  indicator.css('left', tabPosition.left + 'px');
});

• $('.tab-indicator').position().left 와 동일
• 상단 좌표가 필요하면 position().top 을 사용
```

제이쿼리 실전 예제(21) - 할 일(To Do List) 만들기

CSS 파트의 코드가 많아서 핵심적인 코드만 설명합니다. 전체 코드는 완성본을 참고하세요.



```

<div class="todo-frame">
  <h2>To Do List with jQuery</h2>
  <input type="text" id="input" placeholder="Press enter to add tasks">
  <ul class="todo-list">
    <!--
      <li>todo item sample<i class="bi bi-check-square"></i><i class="bi bi-trash"></i></li>
      <li>todo item sample<i class="bi bi-check-square"></i><i class="bi bi-trash"></i></li>
    -->
  </ul>
</div>

```

동적으로 html이 만들어진 것의 CSS를 확인하는 것은 매우 어렵습니다. 동적으로 생성되기 전에 임시로 html 만들고 CSS 디자인을 해야 정교한 디자인을 하기 좋습니다.

```

// 입력하면 할 일 목록(ul)에 할 일 아이템 추가
$('#input').change(function(){
  var inputValue = $(this).val(); —▶ 할 일을 입력하는 input에 엔터를 치면...
  $('#todo-list').append('<li>' + inputValue + '<i class="bi bi-check-square"></i><i class="bi bi-trash"></i></li>');
  // 엔터 후 input 초기화(비우기)
  $(this).val('');
  append() 메서드를 사용해서 동적으로 ul 안에 마지막 li로 새로운 아이템을 추가
});

// 휴지통 아이콘 누르면 생성된 할 일 아이템 삭제
$('#todo-list').on('click', '.bi-trash', function(){
  $(this).parent().fadeOut(200)
});

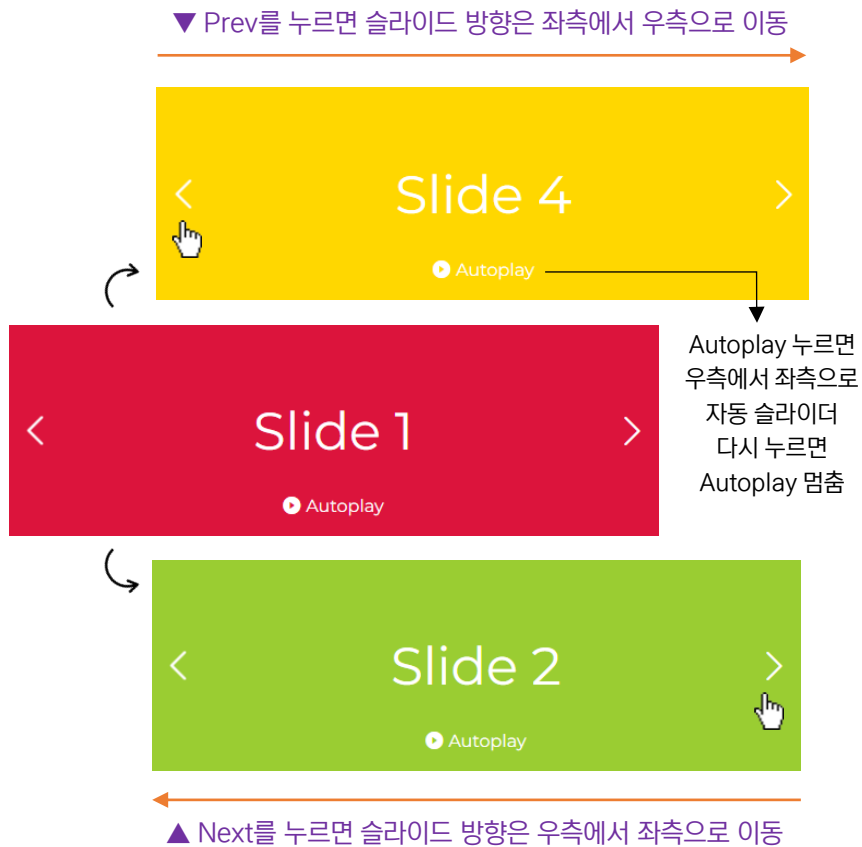
// 체크 아이콘 누르면 li에 .done 클래스 추가/제거 하기
$('#todo-list').on('click', '.bi-check-square', function(){
  $(this).parent().toggleClass('done'); —▶ 체크 아이콘을 누르면 그 부모요소 곧, li에 .done 클래스를 토글(추가/제거)
});

```

▼ \$('선택자').on('click')과 \$('선택자').click()의 차이점

- \$('선택자').on('click') : 동적으로 생성된 태그에 클릭을 가능하게 이벤트 바인딩
- \$('선택자').click() : 최초에 선언된 태그에만 동작(동적 생성 태그에 작동 안함)

제이쿼리 실전 예제(22) - 가로 슬라이더(이전 다음 버튼, Autoplay 제어 버튼) #01



```
<div class="slider">

  <!-- Next Prev Button -->
  <span class="control prev">
    <i class="bi bi-chevron-right"></i>
  </span>
  <span class="control next">
    <i class="bi bi-chevron-right"></i>
  </span>

  <!-- Autoplay control --> 체크박스의 on/off 특성을 이용
  <label for="toggle">
    <input type="checkbox" id="toggle"><em></em>Autoplay
  </label>

  <!-- Slider Content -->
  <ul class="slide-items">
    <li class="slide-item crimson">Slide 1 </li>
    <li class="slide-item yellowgreen">Slide 2</li>
    <li class="slide-item royalblue">Slide 3</li>
    <li class="slide-item gold">Slide 4</li>
  </ul>

</div>
```

Bootstrap Icons
사용(다른 것 사용해도
상관없음)

슬라이더 아이템은 꼭 ul li로 만들어야 하는 것은 아닙니
다. 그냥 div로 만들어도 전혀 상관없습니다.

```
/* Slider Layout */
.slider {
  position: relative;
  width: 600px;
  height: 200px;
  overflow: hidden;
}

.slider ul {
  list-style: none;
  margin: 0;
  padding: 0;
  position: absolute;
  top: 0;
  left: 0;
}

.slider ul li {
  background-color: blue;
  float: left;
  color: #fff;
  font-size: 3em;
  display: flex;
  justify-content: center;
  align-items: center;
}

.slider ul li.crimson {
  background-color: crimson;
}

.slider ul li.yellowgreen {
  background-color: yellowgreen;
}

.slider ul li.royalblue {
  background-color: royalblue;
}

.slider ul li.gold {
  background-color: gold;
}

/* Next Prev Button */
.control {
  position: absolute; z-index: 1000;
  color: #fff;
  text-decoration: none;
  padding: 10px; top: 50%;
  transform: translateY(-50%);
  cursor: pointer; font-size: 2em;
}

.prev {
  left: 0;
  transform: translateY(-50%) rotateY(-180deg);
}

.next {
  right: 0;
}

/* Autoplay Control */
label[for="toggle"] {
  position: absolute;
  z-index: 100;
  left: 50%;
  transform: translateX(-50%);
  bottom: 20px;
  cursor: pointer;
  color: #fff;
}

input[id="toggle"] { display: none; }
input[id="toggle"] + em {
  display: inline-block; font-size: 1em;
  margin-right: 5px; vertical-align: middle;
}

input[id="toggle"] + em:before {
  content: '\F4F2';
  font-family: bootstrap-icons;
  font-style: normal;
}

input[id="toggle"]:checked + em:before {
  content: '\F4C1';
}
```

슬라이드 전체 너비를 600픽
셀을 기준으로 설정(변경되는
경우 자동으로 JS도 변경됨)

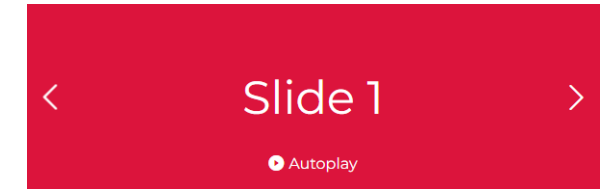
CSS 부분이 좀 까다로운 편입니다.
CSS로 배치를 먼저 잘 해야 만들 수 있습니다.

제이쿼리 실전 예제(22) - 가로 슬라이더(이전 다음 버튼, Autoplay 제어 버튼) #02

```
// 슬라이더 너비 높이, 슬라이드 개수, 전체 슬라이드 너비
var slideWidth = $('.slider').width();
var slideHeight = $('.slider').height();
var slideCount = $('.slide-item').length;
var slideItemsWidth = slideWidth * slideCount;

// 슬라이드의 기본 위치
$('.slide-item').css({'width': slideWidth, 'height': slideHeight});
$('.slide-items').css({'width': slideItemsWidth, 'height': slideHeight});
$('.slide-item:last-child').prependTo($('.slide-items'));
$('.slide-items').css({'margin-left': -slideWidth});
```

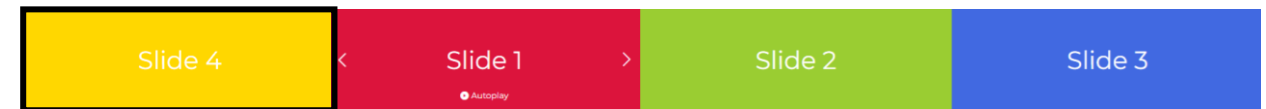
- `$('.slider').width() ==> 600px`
- CSS에서 `.slider`에 너비를 600픽셀 준 상태입니다. 이 너비에 맞춰 모든 것들이 유동적으로 바뀝니다.
- 필요에 따라 CSS에서 `.slider`의 너비를 변경하면 모든 것들이 자동으로 변경됩니다.



▲ 최종적으로 `.slider`에 `overflow: hidden`을 줌



▲ 검은색 박스(`.slide-items`)를 기준으로 첫번째 `.slide-item`가 맨 왼쪽에 위치합니다. Next를 눌러서 좌측으로 이동하는 경우는 문제가 없지만, Prev를 누르는 경우 왼쪽에 준비되어 있는 `.slide-item`이 없어서 어색합니다. 이걸 방지하기 위해서 마지막 `.slide-items`을 맨 앞에 위치합니다.(`prependTo`)



▲ 하지만 시작이 마지막 슬라이드 아이템이 먼저 나오는 게 어색하므로 첫번째 슬라이드 아이템이 나오도록 합니다.



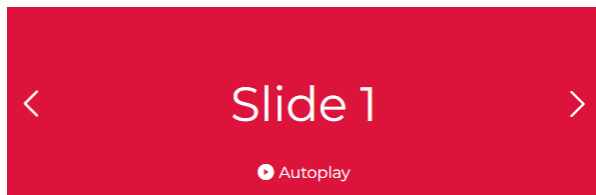
▲ 전체적으로 `.slide-items`를 하나의 `.slide-item`의 너비 만큼 왼쪽으로 이동 시킵니다.

제이쿼리 실전 예제(22) - 가로 슬라이더(이전 다음 버튼, Autoplay 제어 버튼) #03

```
// Next Function
function moveRight(){
    $('.slide-items').stop().animate({'left': -slideWidth}, 500, function(){
        $('.slide-items').css({'left': 0}); ①
        $('.slide-item:first-child').appendTo('.slide-items'); ②
    });
}

// Prev Function
function moveLeft(){
    $('.slide-items').stop().animate({'left': slideWidth}, 500, function(){
        $('.slide-items').css({'left': 0});
        $('.slide-item:last-child').prependTo('.slide-items');
    });
}
```

- left 대신 `margin-left`를 사용해도 결과는 같습니다.
- 순식간에 실행되기 때문에 ① ② 순서는 바뀌도 상관없습니다.
- 첫번째 슬라이드 아이템을 `appendTo`로 `.slide-items` 맨 뒤에 붙입니다.



▲ 재생 멈춘 아이콘 변경은 CSS에
서 `:before`로 변경 시킴

- left 대신 `margin-left`를 사용해도 결과는 같습니다.
- 순식간에 실행되기 때문에 ① ② 순서는 바뀌도 상관없습니다.
- 첫번째 슬라이드 아이템을 `appendTo`로 `.slide-items` 맨 뒤에 붙입니다.

```
// Next Prev Button
$('.next').click(function(e){
    moveRight();
    e.preventDefault();
});

$('.prev').click(function(e){
    moveLeft();
    e.preventDefault();
});

/* Autoplay Control */
$('#toggle').click(function(){
    if($(this).is(':checked')) {
        autoplay = setInterval(moveRight, 5000);
    }
    else {
        clearInterval(autoplay);
    }
});
```

▲ `is()` 메서드 : 특정요소가 선택요소의 현재의 상태 확인 후 참(true) 거짓(false) 반환

제이쿼리 실전 예제(23) - 가로 슬라이더(자동 Autoplay, 마우스 오버 멈추고 빠지면 다시 시작)

▼ 자동 슬라이드로 방향은 좌측에서 우측으로 이동



CSS 파트의 코드가 많아서 핵심적인 코드만 설명합니다. 전체 코드는 완성본을 참고하세요.

```
<div class="slider">
  <!-- Slider Content -->
  <ul class="slide-items">
    <li class="slide-item">
      
    </li>
    <li class="slide-item">
      
    </li>
    <li class="slide-item">
      
    </li>
    <li class="slide-item">
      
    </li>
  </ul>
</div>
```

HTML

▲ 자동 슬라이드이기 때문에 Next Prev 버튼이 없고, Autoplay 제어 버튼도 필요 없음

```
// 슬라이더 너비 높이, 슬라이드 개수, 전체 슬라이드 너비
var slideWidth = $('.slider').width();
var slideHeight = $('.slider').height();
var slideCount = $('.slide-item').length;
var slideItemsWidth = slideWidth * slideCount;

// 슬라이드의 기본 위치
$('.slide-item').css({'width': slideWidth, 'height': slideHeight});
$('.slide-items').css({'width': slideItemsWidth, 'height': slideHeight});
// $('.slide-item:last-child').prependTo($('.slide-items'));
// $('.slide-items').css({'margin-left': -slideWidth});

// Autoplay Function
function mainSlider(){
  autoplay = setInterval(function(){
    $('.slide-items').stop().animate({'left': -slideWidth}, 300, function(){
      $('.slide-items').css({'left': 0});
      $('.slide-item:first-child').appendTo('.slide-items');
    });
  }, 2000)
}
```

검은색 박스(.slide-items)를 기준으로 첫번째 .slide-item가 맨 왼쪽에 위치할 필요가 없으므로 2줄 코드는 필요없음

mainSlider라는 이름의 함수 만들고 setInterval을 편의상 autoplay 변수에 담습니다.

JS

// Start Autoplay
mainSlider(); → 슬라이더 자동 실행// Pause Autoplay
\$('.slider').mouseenter(function(){
 clearInterval(autoplay);
});
\$('.slider').mouseleave(function(){
 mainSlider();
});

- 마우스 올리면 autoplay 실행 중지
- 마우스 빠지면 mainSlider() 재실행

제이쿼리 실전 예제(24) - 폼 입력 필드 클릭하면 텍스트 올라가는 애니메이션

CSS 파트의 코드가 많아서 핵심적인 코드만 설명합니다. 전체 코드는 완성본을 참고하세요.

```

<form>
  <div class="login-form">
    <div class="field">
      <span class="label-text">email</span>
      <input type="email">
    </div>
    <div class="field">
      <span class="label-text">password</span>
      <input type="password">
    </div>
    <div class="field">
      <span class="label-text">password confirm</span>
      <input type="password">
    </div>
    <button>Login</button>
  </div>
</form>

```

input을 클릭해도
라벨 텍스트가 위에
있어서 클릭되지 않
는 것을 방지하기 위
해 필요함

```

.login-form .label-text {
  position: absolute;
  text-transform: capitalize;
  color: #999;
  font-size: 15px;
  left: 0;
  top: 10px;
  transition: 0.35s;
  pointer-events: none;
}
.login-form .label-text.active {
  top: -12px; color: #44bd32;
}

```

JS

```

// 선택자가 긴 경우 변수에 담아서 사용
var $inputField = $('.login-form input');

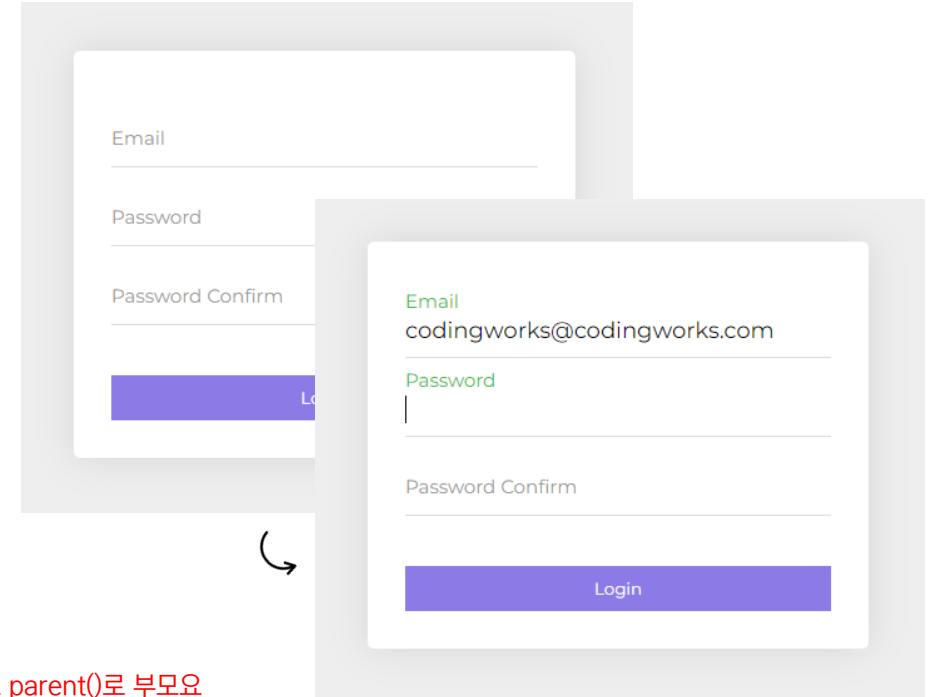
// input에 클릭했을 때 라벨 텍스트 위로 올리기
$inputField.focus(function(){
  $(this).parent().find('.label-text').addClass('active');
});

// input을 벗어났을 때 입력된 내용이 없으면 .active 제거하기
$inputField.focusout(function(){
  if($(this).val() == '') {
    $(this).parent().find('.label-text').removeClass('active');
  }
});

```

input에 입력 값이
없다면 클래스 제거

val() == "" 는 input에 어떤 입력 값도 없다는 의미



- input을 클릭할 경우 라벨 텍스트에 .active 클래스가 추가되면서 위로 올라감
- input 입력 값(value)이 있으면 라벨 텍스트는 내려오지 않음
- input 입력 값(value)가 없으면 라벨 텍스트는 내려옴

제이쿼리 실전 예제(25) - 호버 이펙트를 제이쿼리 animate로 만들기 #01

이런 효과는 퍼블리셔에게 CSS가 편하지만 개발자의 경우 자바스크립트 또는 제이쿼리가 편할 수도 있습니다.

```
<div class="image-gallery">
  <h2>jQuery Image Gallery</h2>
  <div class="image-items">
    <a class="image-item" href="#none">
      <span class="caption">Image Gallery #01</span>
    </a>
    <a class="image-item" href="#none">
      <span class="caption">Image Gallery #02</span>
    </a>
    <a class="image-item" href="#none">
      <span class="caption">Image Gallery #03</span>
    </a>
  </div>
</div>
```

/* Custom CSS */

```
.image-gallery {
  width: 800px;
}
```

```
.image-items {
  display: flex;
  gap: 15px;
}
```

```
.image-items .image-item {
  text-decoration: none;
  color: #fff;
  position: relative;
  width: 100%;
  overflow: hidden;
}
```

.caption 가리기와 이미지 확대될 때 가리기

```
.image-items .image-item img {
  width: 100%;
  display: block;
}
.image-items .image-item .caption {
  background-color: rgba(0, 0, 0, 0.5);
  position: absolute;
  bottom: -40px;
  left: 0;
  width: 100%;
  height: 40px;
  display: flex;
  justify-content: center;
  align-items: center;
  font-size: 14px;
}
```

→ 이미지 아래에 약간의 마진 제거

→ 안보이게 하기 위한 최초 위치(마우스가 올라가면 bottom: 0가 되면서 보여짐)

CSS

jQuery Image Gallery



jQuery Image Gallery



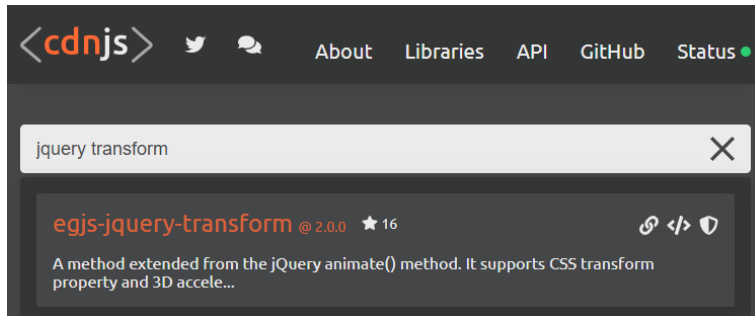
jQuery Image Gallery



▲ 마지막 .image-item은 마우스가 숨어있던 .caption 보여주고 이미지도 확대

제이쿼리 실전 예제(25) - 호버 이펙트를 제이쿼리 animate로 만들기 #02

제이쿼리 animate() 메서드에서 rotate, scale 등 transform 속성을 지원하지 않지만
별도 플러그인을 사용해서 해결할 수 있습니다. (동일한 기능의 여러가지 종류의 플러그인 있음)



```
<!-- jQuery CDN -->
<script src="https://code.jquery.com/jquery-3.5.1.min.js"></script>
<!-- Support transform Plugin CDN -->
<script src="https://cdnjs.cloudflare.com/ajax/libs/egjs-jquery-transform/2.0.0/transform.min.js"></script>
```

animate 메서드에서 transform 속성을 사용할 수 있게 하는 별도의 플러그인 CDN



▲ 마지막 .image-item은 마우스가 숨어있던 .caption 보여주고 이미지도 확대

```
// 편의상 문서객체를 변수로 선언
var $duration = 350,
    $item = $('.image-items .image-item'),
    $itemCaption = $('.image-items .image-item .caption'),
    $itemImg = $('.image-items .image-item img');

// 공통적인 이미지 갤러리 호버 이펙트
$item.mouseover(function(){
    $(this).find($itemCaption).stop().animate({bottom: 0}, $duration);
});
$item.mouseout(function(){
    $(this).find($itemCaption).stop().animate({bottom: '-40px'}, $duration);
});

// 개별적으로 다른 이펙트를 준다면..
$item.filter(':last-child').mouseenter(function(){
    $(this).find($itemImg).animate({'transform': 'scale(1.2)'}, $duration);
})
.mouseleave(function(){
    $(this).find($itemImg).animate({'transform': 'scale(1)'}, $duration);
});
```

→ 꼭 변수를 선언할 필요는 없습니다. 그냥 \$('.image-item')으로 해도 전혀 상관없습니다.

→ mouseover, mouseout 을 써도 되고 mouseenter, mouseleave 를 써도 됨

→ CSS에 클래스네임을 주지 않고 filter() 를 사용해서 마지막에 있는 .image-item을 손쉽게 선택

→ transform을 작동시키는 제이쿼리 플러그인이 없으면 작동하지 않음

제이쿼리 실전 예제(26) - 사이드바 네비게이션으로 숨겨진 콘텐츠 열기

CSS 파트의 코드가 많아서 핵심적인 코드만 설명합니다. 전체 코드는 완성본을 참고하세요.



```
<div class="sidebar-content">
  <h2>Side Content</h2>
  <p>
    Lorem ipsum dolor sit amet, consectetur adipiscing
    elit. Deserunt perspicatis nostrum ab? Earum quia e
    optio reiciendis amet quae cumque soluta ex iusto,
    labore aliquid consectetur nisi? Sequi, odio vitae.
  </p>
  <div class="btn-sidebar">
    <i class="bi bi-chevron-right"></i> <span>open</span>
  </div>
</div>
```

HTML

/* Active CSS */

CSS

```
.sidebar-content.active {
  left: 0;
}
.sidebar-content.active .btn-sidebar {
  background-color: crimson;
}
```

```
// 편의상 문서객체를 변수로 선언
var $sidebarContent = $('.sidebar-content'),
    $btnSidebar = $('.btn-sidebar'),
    $btnSidebarIcon = $('.btn-sidebar .bi'),
    $btnSidebarText = $('.btn-sidebar span');

// 클릭 이벤트
$btnSidebar.click(function(){
  $sidebarContent.toggleClass('active'); // → 사이드바 콘텐츠를 보이고 감추는 기능
  $btnSidebarIcon.toggleClass('bi-chevron-left bi-chevron-right');
  // 텍스트 변경 open을 close로, close를 open으로
  if($btnSidebarText.text() == 'open') {
    $btnSidebarText.text('close')
  }
  else {
    $btnSidebarText.text('open')
  }
});
```

아이콘을 변경하는 기능

→ 최초에 close라는 단어가 없기 때문에
\$btnSidebarText.text() == 'close' 로 하면
안됩니다.

JS

제이쿼리 실전 예제(27) - 사진 갤러리 라이트박스(Lightbox) 만들기 #01



```
<div class="gallery-content">
  <h1>Lightbox Image Gallery</h1>
  <div class="gallery">
    <a href="images/goods-01.jpg" data-caption="Food Image Gallery #01">
      
    </a>
    <a href="images/goods-02.jpg" data-caption="Food Image Gallery #02">
      
    </a>
    <a href="images/goods-03.jpg" data-caption="Food Image Gallery #03">
      
    </a>
    <a href="images/goods-04.jpg" data-caption="Food Image Gallery #04">
      
    </a>
    <a href="images/goods-05.jpg" data-caption="Food Image Gallery #05">
      
    </a>
    <a href="images/goods-06.jpg" data-caption="Food Image Gallery #06">
      
    </a>
  </div>
</div>

<div class="lightbox-overlay">
  <div class="lightbox-overlay-content">
    <img src="" alt="Goods Image" title="Close the image">
    <span></span>
  </div>
</div>
```

제이쿼리 실전 예제(27) - 사진 갤러리 라이트박스(Lightbox) 만들기 #02

CSS 파트의 코드가 많아서 핵심적인 코드만 설명합니다. 전체 코드는 완성본을 참고하세요.



```

.lightbox-overlay {
  position: fixed;
  background-color: #000;
  width: 100%;
  height: 100%;
  display: flex;
  justify-content: center;
  align-items: center;
  opacity: 0;
  visibility: hidden;
  transition: 0.35s;
}

.lightbox-overlay span {
  color: #fff;
  display: block;
  text-align: center;
  padding: 20px;
}

.lightbox-overlay.active {
  opacity: 1;
  visibility: visible;
}

```

▲ 오버레이 기능을 하는 가장 중요한 CSS

- 썸네일 이미지를 CSS로 배치하는 부분도 JS만큼 중요합니다. 완성본을 보기 전에 꼭 CSS 배치를 혼자서 해보세요.

```

// 편의상 문서객체 변수선언
var $zoomSrc = $('.gallery a'),
    $overlay = $('.lightbox-overlay'),
    $overlayImage = $overlay.find('img'),
    $overlayCaption = $overlay.find('span');

// 라이트박스 열기
$zoomSrc.click(function(e){
  var zoomSrc = $(this).attr('href');
  $overlay.addClass('active');
  $overlayImage.attr('src', zoomSrc);
  $overlayCaption.text($(this).attr('data-caption'));

  // href 속성 값 때문에 url이 이미지 경로로 연결되는거 방지
  e.preventDefault();
});

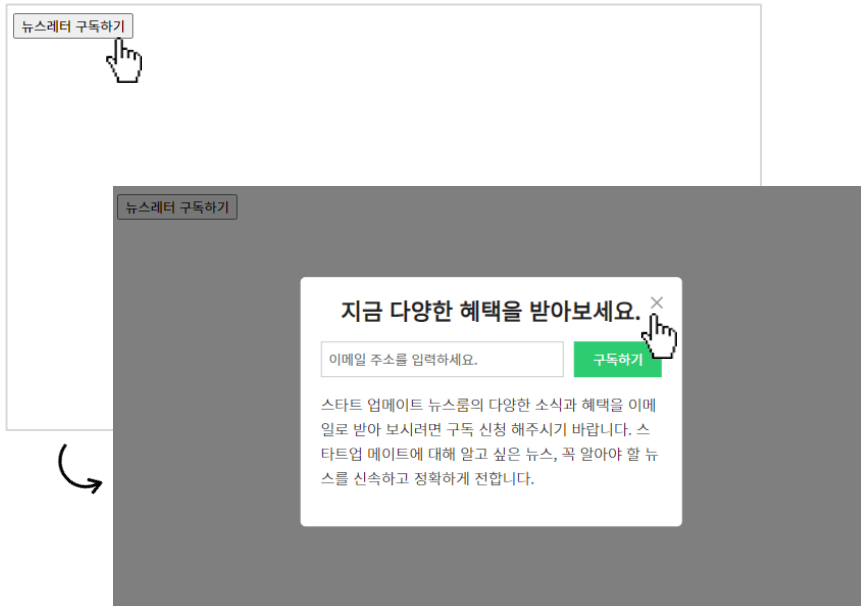
// 라이트박스 닫기
$overlay.click(function(){
  $(this).removeClass('active');
});

```

반드시 변수로 사용할 필요는 없지만 문서 객체가 길어지는 것 보다는 변수 사용이 좋습니다.

a태그의 data-caption 속성의 값 곧, 텍스트를 span에 찍어줍니다.

제이쿼리 실전 예제(28) - 반응형 애니메이션 모달 띄우기 #01



▲ 구독하기 버튼 누르면 모달 보이기 / 닫기 버튼 누르면 모달 숨기기

- visibility: hidden을 해주지 않으면 body 내 콘텐츠가 .overlay 위에 있어서 마우스 이벤트 곧, 클릭 되지 않습니다.
- visibility: hidden 대신에 pointer-events: none을 사용해도 됨
- .active 클래스가 추가되면 visibility: visible로 보이게 합니다.

```
<button class="btn-open">뉴스레터 구독하기</button>

<div class="modal">
  <div class="modal-content">
    <div class="modal-header">
      <h2>지금 다양한 혜택을 받아보세요.</h2>
      <button class="btn-close">&times;</button>
    </div>
    <div class="newsletter">
      <div class="newsletter-apply">
        <input type="email" placeholder="이메일 주소">
        <button>구독하기</button>
      </div>
      <p>...</p>
    </div>
  </div>
</div>
<div class="overlay"></div>
```

HTML

CSS

```
/* Custom CSS */
.modal {
  position: absolute;
  top: 50%;
  left: 50%;
  transform: translate(-50%, -50%);
  border-radius: 5px;
  padding: 20px;
  z-index: 100;
  background-color: #fff;
  opacity: 0;
}

.modal-header h2 {
  text-align: center;
  margin-top: 0;
}

.modal-header .btn-close {
  position: absolute;
  right: 10px;
  top: 10px;
  background-color: transparent;
  border: none;
  cursor: pointer;
  font-size: 1.2em;
  color: #999;
}

.modal-header .btn-close:hover {
  color: #000;
}
```

```
.overlay {
  background-color: rgba(0, 0, 0, 0.5);
  position: fixed;
  top: 0;
  left: 0;
  width: 100%;
  height: 100vh;
  border-radius: 5px;
  opacity: 0;
}

.newsletter-apply {
  display: flex;
  gap: 10px;
}

.newsletter-apply input[type=email] {
  padding: 7px;
  flex: 3;
  border: 1px solid #ccc;
  outline: none;
}

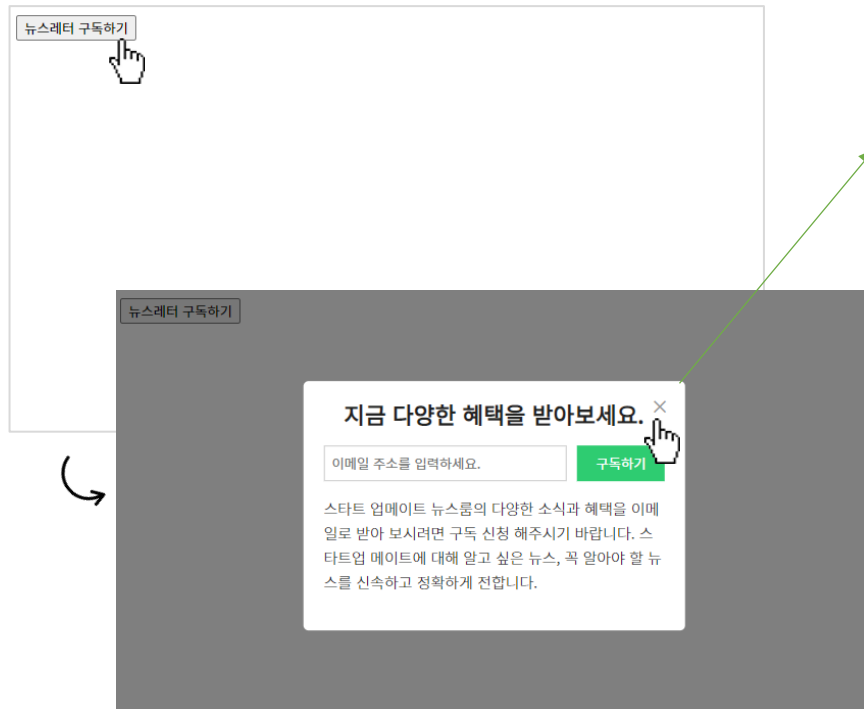
.newsletter-apply button {
  flex: 1;
  border: none;
  background-color: #2ecc71;
  color: #fff;
  cursor: pointer;
}
```

```
/* Transition & Visibility */
.modal,
.overlay {
  transition: 0.5s;
  visibility: hidden;
}

/* Active Class */
.modal.active,
.overlay.active {
  opacity: 1;
  visibility: visible;
}
```

제이쿼리 실전 예제(28) - 반응형 애니메이션 모달 띄우기 #02(다양한 애니메이션으로 모달 보이기)

모달이 보여지는 방식은 CSS만 변화를 주면 다양한 애니메이션으로 보여줄 수 있습니다.



▲ 구독하기 버튼 누르면 모달 보이기 / 닫기 버튼 누르면 모달 숨기기

- ① 위에서 아래로 내려오면서 보여지는 모달 CSS
- ② 밑에서 위로 올라가면서 보여지는 모달 CSS
- ③ 우측에서 좌측으로 보여지는 모달 CSS
- ④ 좌측에서 우측으로 보여지는 모달 CSS
- ⑤ 크기가 커지면서 보여지는 모달 CSS

- ① .modal에 **transform: translate(-50%, -70%)** 주고
.modal.active에 **transform: translate(-50%, -50%)**
- ② .modal에 **transform: translate(-50%, -30%)** 주고
.modal.active에 **transform: translate(-50%, -50%)**
- ③ .modal에 **transform: translate(-30%, -50%)** 주고
.modal.active에 **transform: translate(-50%, -50%)**
- ④ .modal에 **transform: translate(-70%, -30%)** 주고
.modal.active에 **transform: translate(-50%, -50%)**
- ⑤ .modal에 **transform: translate(-50%, -50%) scale(0.7)** 주고
.modal.active에 **transform: translate(-50%, -50%) scale(1)**

```
// 페이지 로드될 때 바로 모달을 띄울 때
$('.modal, .overlay').addClass('active')

$('.btn-open').click(function(){
  $('.modal, .overlay').addClass('active')
});
$('.btn-close, .overlay').click(function(){
  $('.modal, .overlay').removeClass('active')
});
```

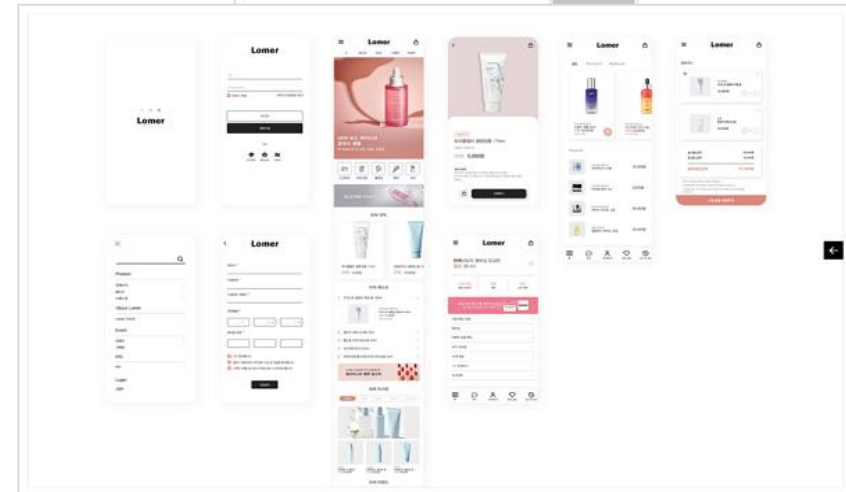
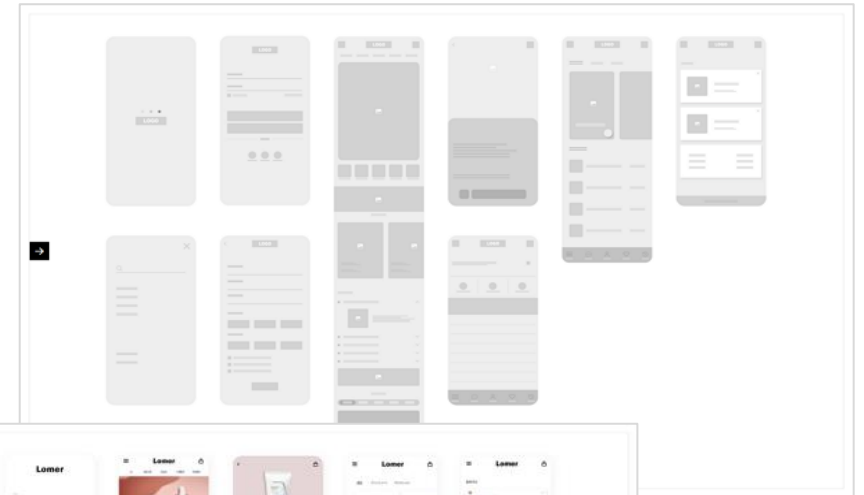
JS

- transform: translate(x축, y축);
- transform 속성에서 translate와 scale을 동시에 사용할 때 반드시 하나로 사용해야 합니다. 절대 transform 속성을 2번 사용해서 사용하면 안됩니다.

제이쿼리 실전 예제(29) - Front Back 트랜지션 화면전환 #01



화면전환을 하는 제이쿼리 예제입니다. 단순하지만 우측에 있는 것처럼 개인 포트폴리오 홈페이지에 모바일 웹 와이어 프레임과 UI 디자인을 전환해서 보여주는 방식으로 활용하면 좋습니다.



제이쿼리 실전 예제(29) - Front Back 트랜지션 화면전환 #02

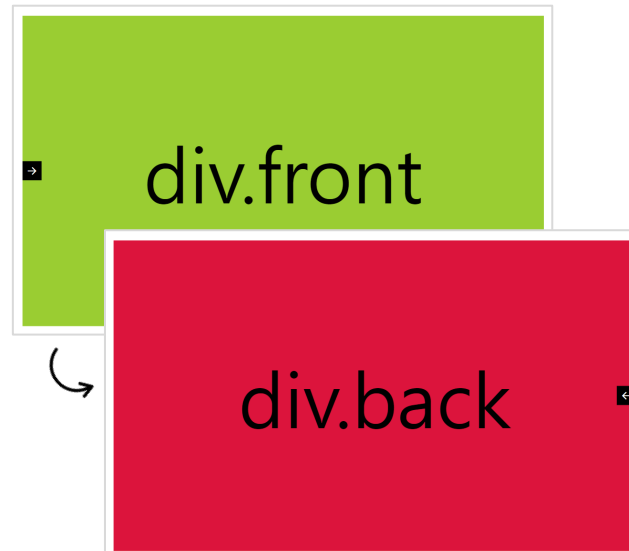
- div 안에 img를 활용해서 트랜지션 효과를 주면 포토 폴리오 작업물로 활용하면 좋습니다.
- 예시로 인테리어 전후 또는 성형 전후 사진을 전환해서 보여주는 용도도 좋습니다.

```
<div class="container">
  <div class="open-navi">
    <i class="xi-arrow-right"></i>
  </div>
  <div class="close-navi">
    <i class="xi-arrow-left"></i>
  </div>
  <div class="front">div.front</div>
  <div class="back">div.back</div>
</div>
```

HTML

```
$('.open-navi').click(function(){
  $(this).addClass('active');
  $('.close-navi').addClass('active');
  $('.front').fadeOut();
  $('.back').fadeIn();
})
$('.close-navi').click(function(){
  $(this).removeClass('active');
  $('.open-navi').removeClass('active');
  $('.back').fadeOut();
  $('.front').fadeIn();
})
```

JS



```
.container {
  border: 1px solid #ddd;
  position: absolute;
  width: calc(100% - 40px);
  height: calc(100vh - 40px);
  top: 50%;
  left: 50%;
  transform: translate(-50%, -50%);
  overflow: hidden;
}
.front,
.back {
  position: absolute;
  top: 0;
  left: 0;
  width: 100%;
  height: 100%;
  display: flex;
  justify-content: center;
  align-items: center;
  font-size: 10em;
}
.front {
  background-color: yellowgreen;
  z-index: 1;
}
.back {
  background-color: crimson;
  z-index: 0;
}
```

```
/* Icon */
.open-navi,
.close-navi {
  background-color: #000;
  color: #fff;
  position: absolute;
  z-index: 2;
  top: 50%;
  transform: translateY(-50%);
  width: 40px;
  height: 40px;
  text-align: center;
  line-height: 40px;
  font-size: 1.5em;
  cursor: pointer;
  transition: 0.5s;
}
.open-navi {
  left: 0;
}
.open-navi.active {
  left: -40px;
}
.close-navi {
  right: -40px;
}
.close-navi.active {
  right: 0;
}
```

CSS



JQUERY 플러그인

05. 제이쿼리 플러그인

- 제이쿼리 슬라이더 플러그인 Slick Slider 사용법 - 슬라이더 플러그인은 하나쯤 완벽하게 다룰 수 있어야 합니다!
- 제이쿼리 타이핑 플러그인 Typed.js 사용법 - 웹 사이트의 인트로 페이지에 적당한 타이핑 애니메이션은 추천!
- 제이쿼리 모달 & 라이트박스 플러그인 featherlight.js 사용법 - 모달과 라이트박스를 효율적으로 멋지게 띄우기
- 제이쿼리 카운트다운 타이머 플러그인 The Final Countdown - 하나의 플러그인으로 다양한 카운트다운 종류 사용



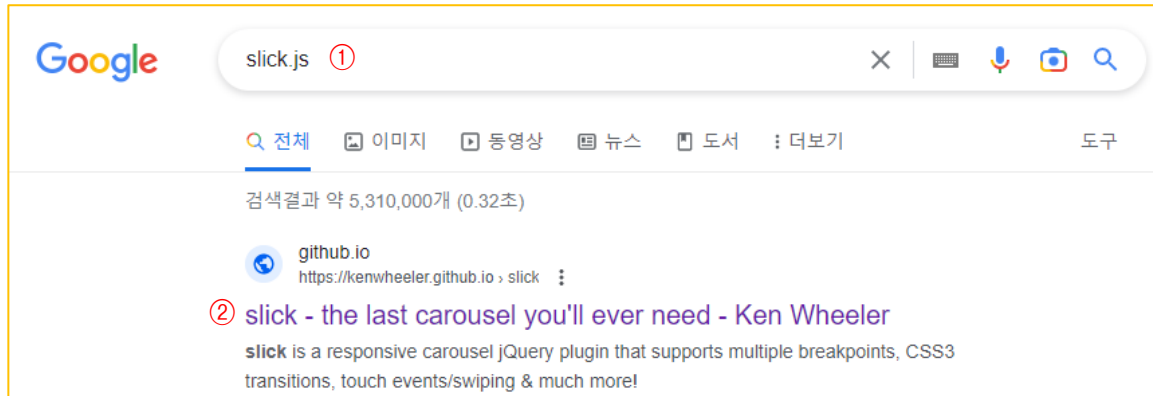
제이쿼리 플러그인을 사용할 때 기능성 플러그인이 아닌 경우 가급적 사용을 자제해야 합니다. 기능성 플러그인이란 슬라이더, 카운트다운 타이머 등 제작자가 만들기도 어렵고 만든다고 효율적이지 못한 것들을 말합니다. 그리고 제이쿼리 플러그인을 선택할 때는 반드시 사용하기 편하고 옵션 조절이 쉬운 플러그인을 선택해야 합니다.

제이쿼리 슬라이더 플러그인 Slick Slider 사용법 - 슬라이더 플러그인은 하나쯤 완벽하게 다룰 수 있어야 합니다!

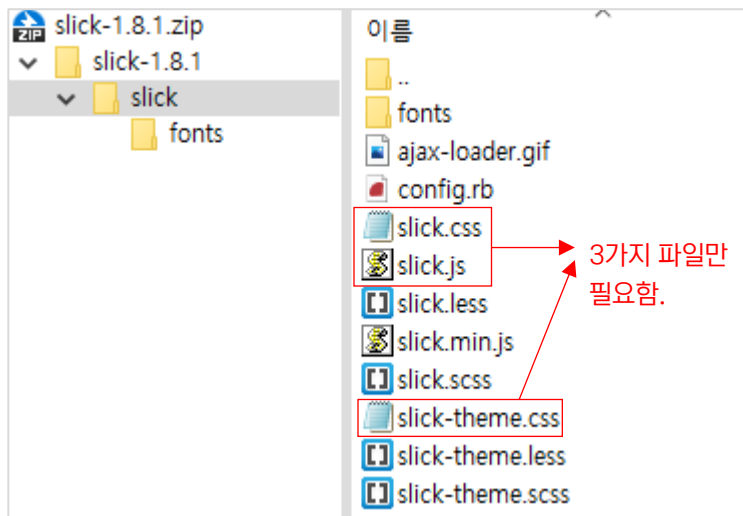
1

제이쿼리 플러그인은 필수적인 것만 사용하는 것이 바람직합니다. 제이쿼리 플러그인 중에 슬라이더 플러그인은 반드시 잘 사용할 줄 알아야 합니다. 대표적인 제이쿼리 슬라이더 플러그인으로 **Slick Slider**, **Swiper Slider** 입니다. 대부분의 프로젝트에서 2가지 중에 하나를 사용하는 것이 일반적입니다.



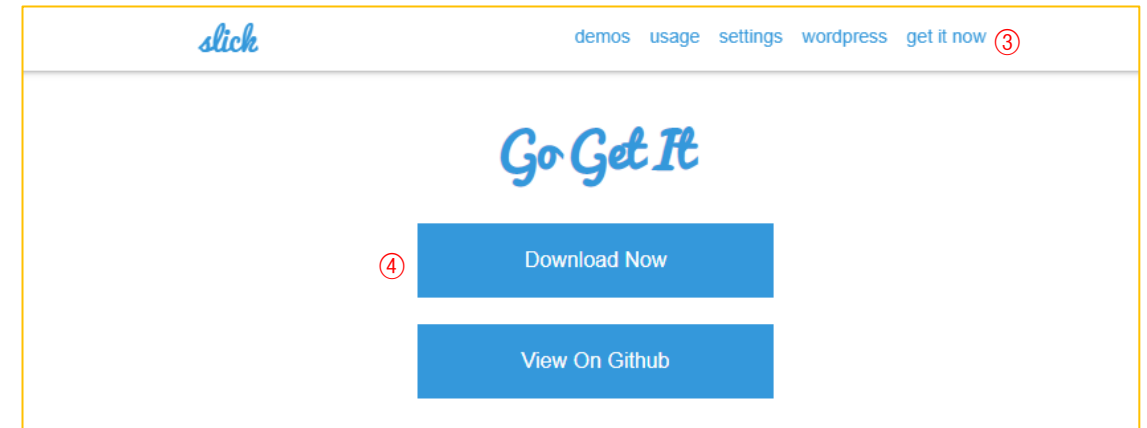
제이쿼리 슬라이더 플러그인 Slick Slider 사용법(1) : 슬릭(Slick) 슬라이더 공식 사이트 & 다운로드 

- ① 구글에서 slick.js 검색
- ② Slick Slider 공식 사이트 클릭
- ③ get it now 클릭
- ④ Download Now 클릭
- 다운로드 대신에 CDN을 사용해도 됩니다. <http://www.cdnjs.com>



- ⑤ 3가지 파일을 slick이라는 폴더 안에 드래그해서 준비합니다.

- slick : slick
- slick-theme : slick-theme
- slick.css : 슬라이더 상세 CSS
- slick-theme.css : 슬라이더 기본 레이아웃 CSS
- slick.js : 슬라이더를 작동하는 JS

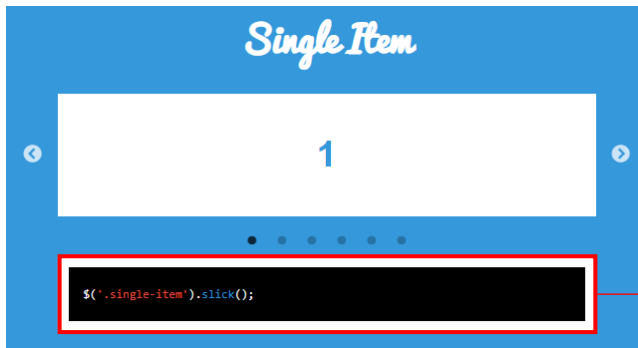
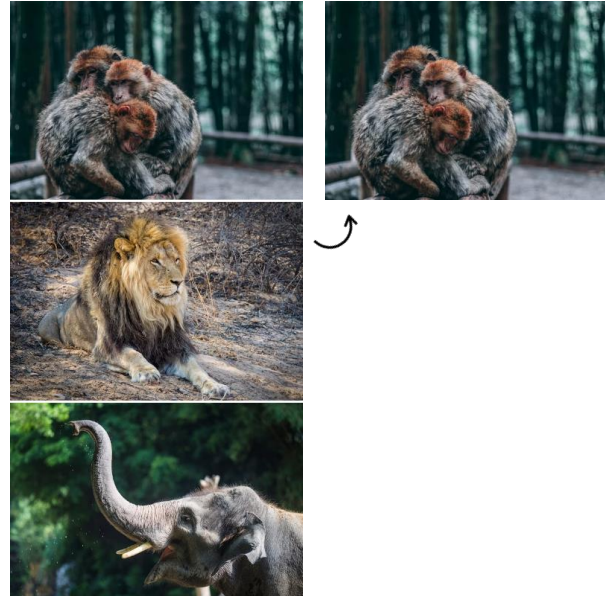


▲ 슬릭 슬라이더 공식 사이트 : <https://kenwheeler.github.io/slick/>

제이쿼리 슬라이더 플러그인 Slick Slider 사용법(2-1) : 파일 링크로 사용

- ✓ 제이쿼리 슬라이더 플러그인 Slick Slider 사용법
 - ✓ images
 -  slide-01.jpg
 -  slide-02.jpg
 -  slide-03.jpg
 - ✓ slick
 - # slick-theme.css
 - # slick.css
 - JS slick.js
 - 1) 파일 링크로 사용.html

▲ 슬릭 슬라이더 연습을 위한 폴더구조



▲ 공식 사이트에서 붉은색 박스 부분 코드 복사

```
$('.single-item').slick();
```

.single-item은 예시 클래스네임 입니다.
html 만든 클래스네임을 사용합니다.

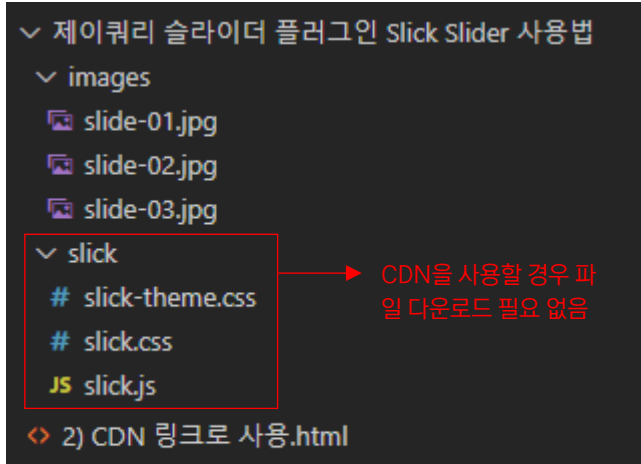
```
<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <title>링크하고 기본적인 사용법 알아보기</title>
  <!-- jQuery CDN -->
  <script src="https://code.jquery.com/jquery-3.5.1.min.js"></script>
  <!-- Slick Slider -->
  <script src="slick/slick.js"></script>
  <link rel="stylesheet" href="slick/slick-theme.css">
  <link rel="stylesheet" href="slick/slick.css">
</head>
<body>
  <div class="slide-items">
    
    
    
  </div>

  <script>
    <!-- jQuery -->
    <!-- Slick Slider -->
    <script>
      $('.slide-items').slick();
    </script>
  </script>
</body>
</html>
```

- head 사이에 여러가지 CSS와 Script 파일이 링크되므로 항상 주석 처리 잘 해야 합니다.
- 슥릭 슬라이더는 제이쿼리 플러그인이므로 제이쿼리 라이브러리는 반드시 위에 있어야 합니다.
- 슥릭 슬라이더는 js와 css 순서는 관계 없음

스릭 슬라이더 옵션들

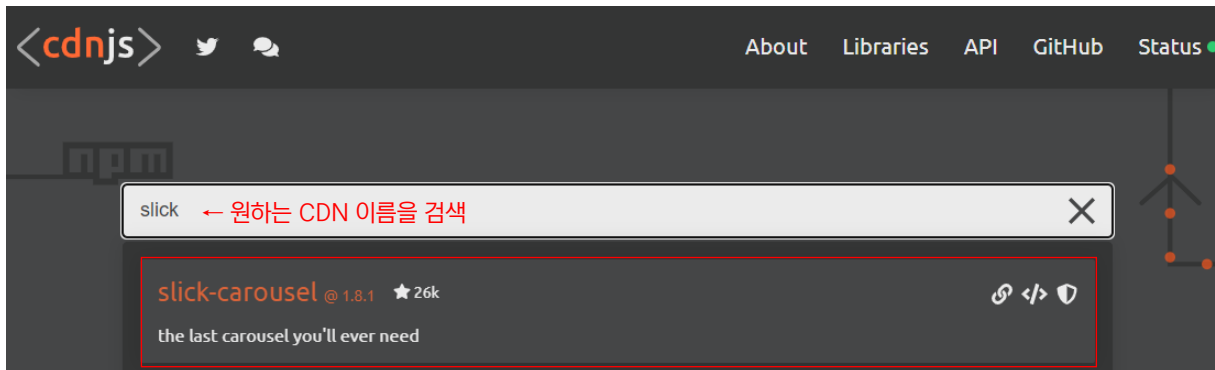
제이쿼리 슬라이더 플러그인 Slick Slider 사용법(2-2) : CDN 링크로 사용



▲ 슬라이더 연습을 위한 폴더구조

```
<head>
  <meta charset="UTF-8">
  <title>링크하고 기본적인 사용법 알아보기</title>
  <!-- jQuery CDN -->
  <script src="https://code.jquery.com/jquery-3.5.1.min.js"></script>
  <!-- Slick Slider CDN -->
  <script src="https://cdnjs.cloudflare.com/ajax/libs/slick-carousel/1.8.1/slick.min.js"></script>
  <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/slick-carousel/1.8.1/slick-theme.css">
  <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/slick-carousel/1.8.1/slick.css">
</head>
```

슬릭 슬라이더 CDN 사용



▲ cdnjs 사용법 : <https://www.infllearn.com/blogs/848>



슬릭 슬라이더 CSS 레이아웃은 Flex를 기반으로 하고 있습니다.

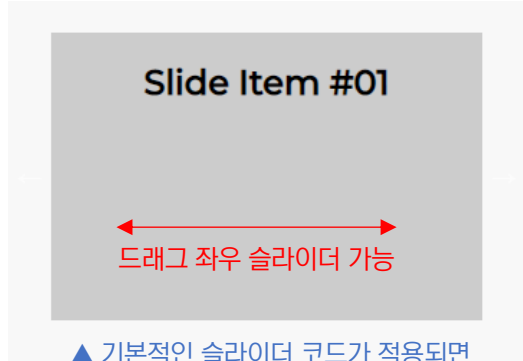
제이쿼리 슬라이더 플러그인 Slick Slider 사용법(2-3) : 슬릭 슬라이더 옵션 설정

모든 슬라이드 옵션을 알아야 할 필요는 없습니다. 필요한 경우 슬릭 슬라이더 공식 사이트에서 체크하고 사용하면 됩니다. 하지만 해당 교재에서 설명하는 필수적인 것들만 사용하시면 충분합니다.

Slide Item #01
Slide Item #02
Slide Item #03
Slide Item #04
Slide Item #05
Slide Item #06

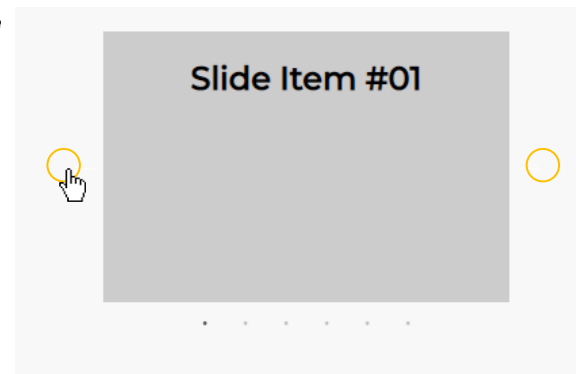
▲ 슬릭 슬라이더 가 적용되지 않은 상태

```
HTML
<div class="slide-items">
  <div class="item">
    <h2>Slide Item #01</h2>
  </div>
  <div class="item">
    <h2>Slide Item #02</h2>
  </div>
  <div class="item">
    <h2>Slide Item #03</h2>
  </div>
  <div class="item">
    <h2>Slide Item #04</h2>
  </div>
  <div class="item">
    <h2>Slide Item #05</h2>
  </div>
  <div class="item">
    <h2>Slide Item #06</h2>
  </div>
</div>
```



▲ 기본적인 슬라이더 코드가 적용되면

```
$('.slide-items').slick();
```



```
$('.slide-items').slick({
  infinite: true,
  arrows: true,
  dots: true,
  speed: 500,
  autoplay: true,
  autoplaySpeed: 2000,
});
```

- **infinite** : 슬라이더 무한 반복 여부, 기본값 true이므로 넣지 않아도 됨. 만약 마지막 슬라이드 아이템에서 반복하지 않으려면 false를 넣음.
- **arrows** : 다음 이전 버튼. 투명한 상태라서 보이지 않지만 실제로는 있음(CSS로 디자인 변경해야 함)
- **dots** : 슬라이드 아이템 밑에 동그라미 버튼. 사용하지 않으려면 false (CSS로 디자인 변경해야 함)
- **speed** : 슬라이드 아이템끼리 지나가는 시간
- **autoplay** : 자동 플레이 여부(기본값 false)
- **autoplaySpeed** : 슬라이드 아이템이 머무는 시간

```
/* Custom CSS */
body{
  margin: 50px;
  background-color: #f8f8f8;
}
.slide-items {
  width: 300px; 슬라이더 전체 크기
}
.slide-items .item {
  background-color: #ccc;
  height: 200px;
  text-align: center;
}
// 개별적인 슬라이드 아이템 높이
```

제이쿼리 슬라이더 플러그인 Slick Slider 사용법(2-4) : 슬릭 슬라이더 옵션 설정(슬라이드 개수 및 여백 조절)

```
<div class="slide-items">
  <div class="item"><h2>Slide Item #01</h2></div>
  <div class="item"><h2>Slide Item #02</h2></div>
  <div class="item"><h2>Slide Item #03</h2></div>
  <div class="item"><h2>Slide Item #04</h2></div>
  <div class="item"><h2>Slide Item #05</h2></div>
  <div class="item"><h2>Slide Item #06</h2></div>
</div>
```

HTML

```
$('.slide-items').slick({
  infinite: true,
  arrows: true,
  dots: true,
  speed: 500,
  slidesToShow: 3,
  slidesToScroll: 3
});
```

JS

- fade: 슬라이더 아이템을 크로스페이드로 전환함(기본값은 false) ex) fade: true;

Slide Item #01 Slide Item #02 Slide Item #03

- slidesToShow: 슬라이드 아이템 개수를 설정(기본값은 1)
- slidesToScroll: 슬라이드 할 경우 한번에 몇 개씩 슬라이드 시킬지 설정 (기본값은 1)

슬라이드 아이템에 마진을 주면 마진을 준 만큼 아이템의 너비가 늘기 때문에 아이템을 감싸고 있는 요소에 마진을 그만큼 줄여줘야 슬라이드 아이템의 정상 너비가 됩니다.

```
/* slick slider custom CSS */
.slide-items .slide-list {
  margin: 0 -10px;
}
.slide-items .slick-slide {
  margin: 0 10px;
}
```

CSS

Slide Item #01 Slide Item #02 Slide Item #03

- 슬릭 슬라이더에서 슬라이드 아이템 사이의 간격을 조정하는 옵션은 없습니다. CSS로 간격 조절해야 합니다.

▲ .slide-items .item이라고 생각해서 여기에 마진을 주면 간격이 생길 것 같지만 실제 html 구조는 아래와 같습니다. 곧, .item을 감싸고 있는 부모 요소에 마진을 주어야 합니다.
.slide-items .slide-list .slick-slide div .item {...}

제이쿼리 슬라이더 플러그인 Slick Slider 사용법(2-5) : 슬릭 슬라이더 옵션 설정(디바이스 별 반응형 설정하기)

```

<div class="content">
  <div class="slide-items">
    <div class="item"></div>
    <div class="item"></div>
    <div class="item"></div>
    <div class="item"></div>
    <div class="item"></div>
    <div class="item"></div>
  </div>
  <div class="slide-items">
    <div class="item"></div>
    <div class="item"></div>
    <div class="item"></div>
    <div class="item"></div>
    <div class="item"></div>
    <div class="item"></div>
  </div>
</div>

```

HTML

```

.slide-items { margin: 50px 0; }
.slide-items .item {}
.slide-items .item img {
  width: 100%;
  height: inherit;
  object-fit: cover;
  aspect-ratio: 16 / 9; —> 이미지 너비 기준으로 비율에 맞게
                        자동으로 높이 설정
}
/* Slick Slider Custom CSS */
.slide-items .slick-list {
  margin: 0 -5px;
}
.slide-items .slick-slide {
  margin: 0 5px;
}

```

CSS



```

$('.slide-items').slick({
  dots: true,
  infinite: false,
  speed: 300,
  slidesToShow: 4,
  slidesToScroll: 4,
  responsive: [
    {
      breakpoint: 1024, // ex) 1280
      settings: {
        slidesToShow: 3,
        slidesToScroll: 3,
        infinite: true,
        dots: true
      }
    },
    {
      breakpoint: 600,
      settings: {
        slidesToShow: 2,
        slidesToScroll: 2
      }
    },
    {
      breakpoint: 480,
      settings: {
        slidesToShow: 1,
        slidesToScroll: 1
      }
    }
  ]
});

```

JS

PC 레이아웃 옵션

필요한 경우 브레이크 포인트를 변경할 수 있습니다

Tablet 레이아웃 옵션

Mobile 레이아웃 옵션

작은 사이즈 Mobile 레이아웃 옵션

(참고로 작은 사이즈 모바일 디바이스 부분은 사용하지 않아도 됩니다. 하지만 있는 대로 사용을 추천)

슬릭 슬라이더 반응형 JS 코드는 공식 사이트에서 복사해서 사용

제이쿼리 슬라이더 플러그인 Slick Slider 사용법(2-6) : 슬릭 슬라이더 디자인 설정(커스텀 CSS)

- 제이쿼리 플러그인을 사용하는건 크게 어려운 일이 아닙니다. 플러그인 주소 링크하고 공식 사이트의 사용법으로 스크립트에 복사해서 넣어주면 됩니다. 하지만 CSS 디자인은 제작자가 직접 해야 합니다. 개발자 도구를 활용해서 어떤 선택자를 어떻게 바꿀야 하는지 노력해서 직접 찾아야 합니다. 곧, 제이쿼리 플러그인을 Custom CSS로 보기 좋게 만드느냐 못하느냐가 바로 퍼블리싱 실력이 좌우합니다.
- 대부분 개발자 도구에서 선택자를 찾고 그 선택자에 CSS 속성을 주면 정상적으로 변경됩니다. 가끔 정확한 선택자인데 CSS가 변경이 안되는 경우는 원본 CSS에 선택자 우선 순위 때문에 그렇습니다. 이런 경우 **!important**를 사용해주면 우선 적용할 수 있습니다.



▲ Custom CSS 적용 전



▲ Custom CSS 적용 후

Next Button
Previous Button

Dots Button

```
<div class="front-slider">
  <a href="#none"></a>
  <a href="#none"></a>
  <a href="#none"></a>
</div>
```

HTML

```
/* Front Slider */
$('.front-slider').slick({
  slidesToShow: 1, /* 화면에 출력할 슬라이드 개수 */
  dots: true, /* 하단의 도트 네비게이션 출력 */
  arrows: true, /* 좌우 화살표 네비게이션 출력 */
  autoplay: true, /* 자동재생 */
  autoplaySpeed: 3000, /* 자동재생 속도 */
  slidesToScroll: 1
});
```

JS

제이쿼리 슬라이더 플러그인 Slick Slider 사용법(2-7) : 슬릭 슬라이더 디자인 설정(커스텀 CSS)



```
<div class="front-slider slick-initialized slick-slider slick-dotted">
  <button class="slick-prev slick-arrow" aria-label="Previous" type="button" style>
    ::before → 이전 버튼 CSS 변경할 선택자
    "Previous"
  </button>
  <div class="slick-list draggable"> ... </div>
  <button class="slick-next slick-arrow" aria-label="Next" type="button" style>
    ::before → 다음 버튼 CSS 변경할 선택자
    "Next"
  </button>
  <ul class="slick-dots" style role="tablist"> ... </ul>
</div>
```

도트 버튼들 전체 위치 조정

```
/* Custom CSS */
.front-slider {
  width: 450px; → 슬라이드 너비 설정
  aspect-ratio: 16 / 9; → 슬라이드 너비를 기
  }                               준으로 비율에 맞게
                                  자동으로 높이 설정
.front-slider a img {
  width: 100%; → 슬라이드 아이템 내에 이미지
  }                               의 너비를 100% 설정
```

```
/* Next Prev Arrow */
.front-slider .slick-arrow {
  z-index: 10;
}
.front-slider .slick-prev {
  left: 15px;
  transform: rotateY(-180deg)
}
.front-slider .slick-next {
  right: 20px;
}
.front-slider .slick-prev:before,
.front-slider .slick-next:before {
  color: #000;
  content: '\F285';
  font-family: bootstrap-icons;
  zoom: 1.5;
  display: inline-block;
}
```

▲ Next Button & Previous Button

동그라미 Dot 버튼 간격을 변경할 선택자

```
<ul class="slick-dots" style role="tablist">
  <li class="slick-active" role="presentation"> == $0
    <button type="button" role="tab" id="slick-slide-co
      e00" aria-label="1 of 3" tabindex="0" aria-selected
        ::before → 동그라미 Dot 버튼 모양을 변경할 선택자
        "1"
      </button>
    </li>
  <li role="presentation" class>... </li>
  <li role="presentation">... </li>
</ul>
```

- dot 네비게이션 전체를 이동 시킬 경우는 .slick-dots 선택자를 사용함
- 슬라이드 아이템들을 가지고 있는 전체 슬라이드에 position: relative를 주고 .slick-dots에 position: absolute를 주고 bottom을 이용해서 위치를 잡음 ex) bottom: 0 또는 bottom: 10px

```
/* Dots */
.front-slider .slick-dots li {
  margin: 0;
}
.front-slider .slick-dots li button:before {
  font-size: 40px;
}
.front-slider .slick-dots li.slick-active button:before {
  color: crimson;
}
```

▲ Dots Button

제이쿼리 타이핑 플러그인 Typed.js 사용법(1) - 웹 사이트의 인트로 페이지에 적당한 타이핑 애니메이션은 추천!

2

Typed.js|

공식 사이트 : <https://github.com/mattboldt/typed.js/>

[Typed.js 기본 사용법]

- ① Typed.js CDN 링크를 head 사이에 링크
- ② 타이핑 효과를 줄 html 요소 삽입
- ③ Typed.js를 작동시키는 스크립트 작성

[Typed.js 필수 옵션]

- typeSpeed : 타이핑 속도
- loop : 반복(기본값 반복 안함)

- Typed.js 공식 사이트에서 다양한 옵션을 사용할 수 있지만 기본적인 옵션인 속도와 반복만 사용해도 해당 플러그인을 사용하는 목적에 부합합니다.
- Typed.js 더 많은 옵션 보기 : <https://mattboldt.github.io/typed.js/>

```
<script src="https://unpkg.com/typed.js@2.0.16/dist/typed.umd.js"></script> ①
```

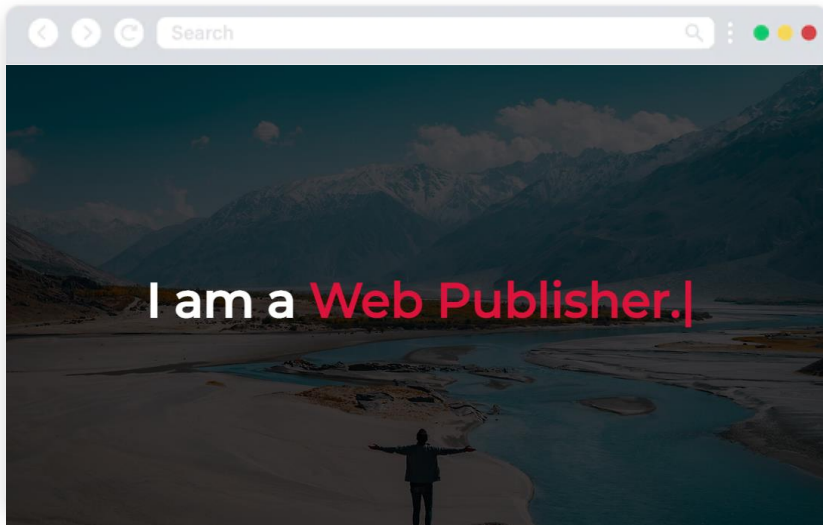
```
<!-- Element to contain animated typing -->
<span id="element"></span> ②

<!-- Setup and start animation! -->
③ <script>
  var typed = new Typed('#element', {
    strings: ['<i>First</i> sentence.', '&amp; a second sentence.'],
    typeSpeed: 50,
  });
</script>
</body>
```

string 내 배열의 개수만큼이 타이핑이 개수

▲ string: ['문자열1', '문자열2', '문자열3']

제이쿼리 타이핑 플러그인 Typed.js 사용법(2) - 웹 사이트 타이핑 인트로 만들기



```
<section>
  <h1 class="heading">I am a <span id="typing"></span></h1>
</section>
```

HTML

```
var typed = new Typed('#typing', {
  strings: ['Free man.', 'Designer.', 'Web Publisher.'],
  typeSpeed: 50, —> 글자 하나당 속도(50 => 0.05초)
  loop: true —> 반복하지 않으려면 false 또는 loop 옵션 넣지 않기
});
```

JS

```
<!-- jQuery CDN -->
<script src="https://code.jquery.com/jquery-3.5.1.min.js"></script>
<!-- Typed.js CDN -->
<script src="https://unpkg.com/typed.js@2.0.16/dist/typed.umd.js"></script>
```

▲ head 사이에 제이쿼리 라이브러리와 Typed.js CDN 링크

```
.heading {
  font-size: 4em;
  position: relative;
  z-index: 100;
  color: #fff;
}
.heading span {
  color: crimson;
}
```

CSS

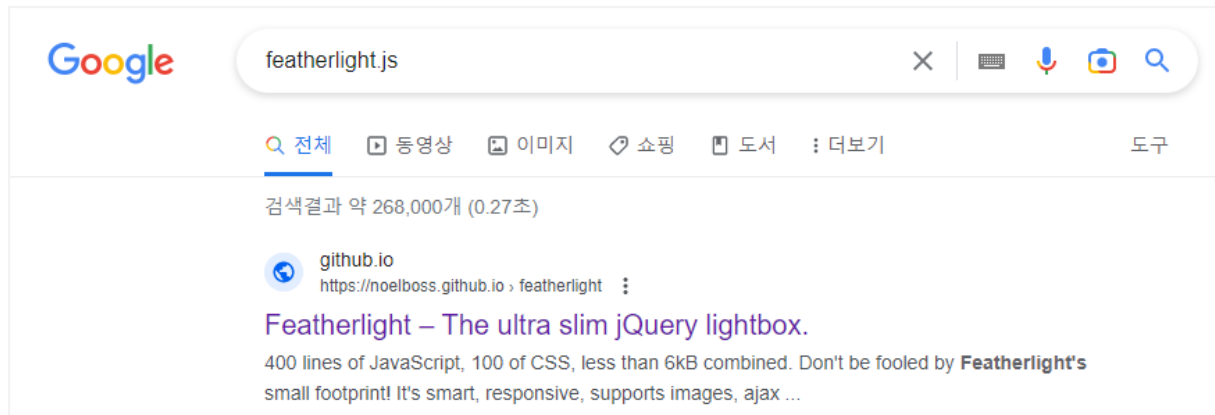
```
/* Custom CSS */
section {
  height: 100vh;
  position: relative;
  display: flex;
  justify-content: center;
  align-items: center;
  background: url('background.jpg') no-repeat;
  background-size: cover;
}
section:before {
  content: '';
  position: absolute;
  width: 100%;
  height: 100%;
  background-color: rgba(0, 0, 0, 0.7);
}
```

- :before로 배경 이미지에 약간 어두운 느낌의 오버레이
- 어두운 오버레이가 있어야 타이핑 텍스트가 강조될 수 있습니다.

제이쿼리 라이트박스 플러그인 featherlight.js 사용법(1) - 모달과 라이트박스를 효율적으로 멋지게 띄우기

3

- 개인 포트폴리오 홈페이지 제작 또는 실무에서 모달과 라이트박스를 띄울일이 많습니다. 한 두개 의 모달이나 라이트박스라면 직접 코딩해서 만들면 됩니다. 하지만 여러 개 또는 반복된다면 직접 코딩해서 만드는 건 쉬운 일이 아닙니다.
- 반복되는 여러 개의 모달과 라이트박스를 아주 효율적으로 멋지게 띄워주는 플러그인 입니다.



▲ 구글에서 featherlight.js 검색

Featherlight.js

공식 사이트 : <https://noelboss.github.io/featherlight/>

제이쿼리 라이트박스 플러그인 featherlight.js 사용법(2) - 모달과 라이트박스를 효율적으로 멋지게 띄우기

Featherlight.js The ultra slim

lightbox.

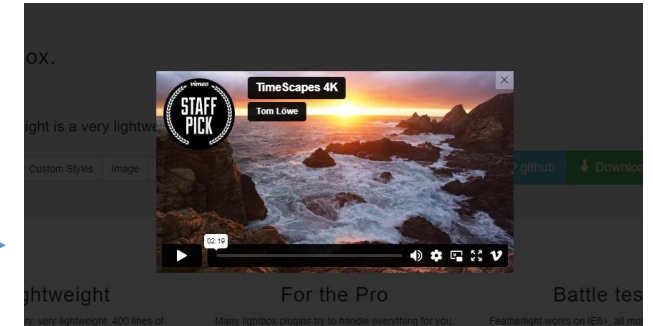
Featherlight is a very lightweight jQuery lightbox.

Default Custom Styles Image iFrame Ajax

자세한 사용법을 볼 수 있는 공식 깃허브

github Download (1.7.13)

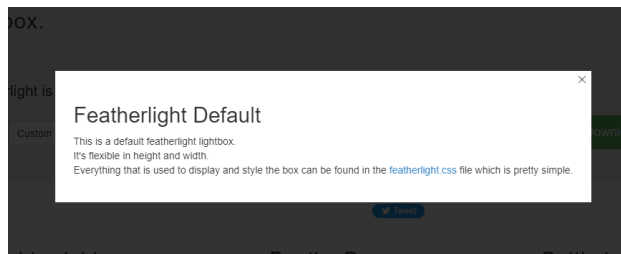
featherlight 자바스크립트 파일과 CSS 파일 다운로드 ▲



▲ iframe 모달 띄우기

(URL, 파일 등 링크를 iframe에 넣어서 달 띄우기)

디자이너, 웹 퍼블리셔 개인 포트폴리오 홈페이지에서
결과물을 보여줄 때 매우 효율적입니다.

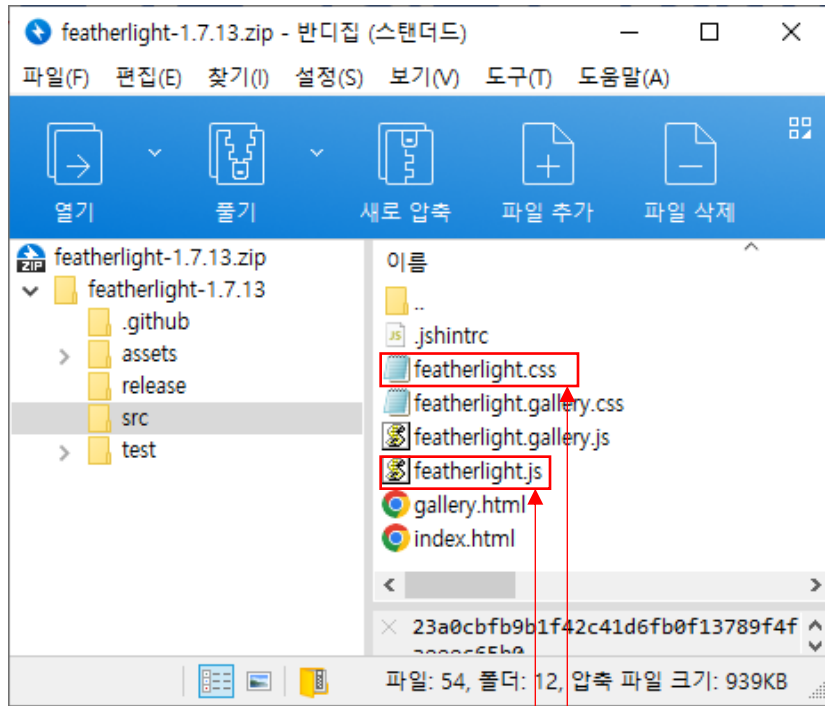


▲ 기본적인 모달 띄우기

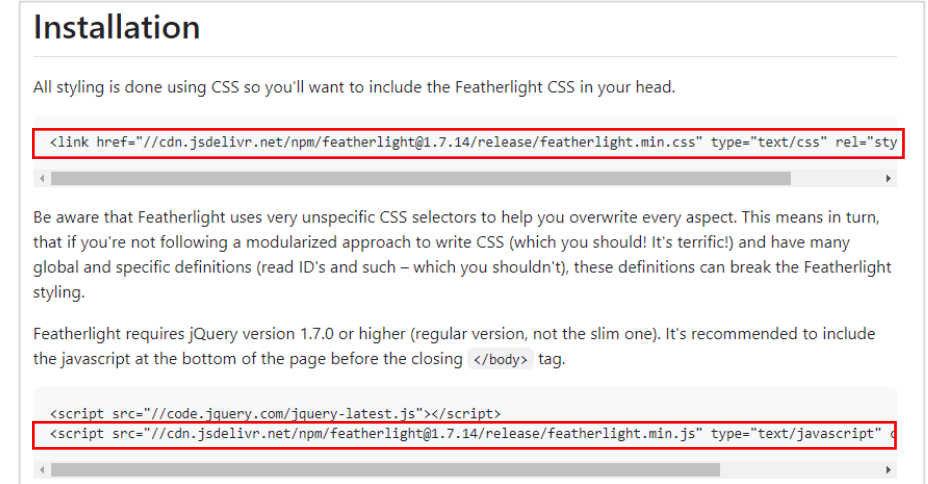


▲ 이미지 모달 띄우기

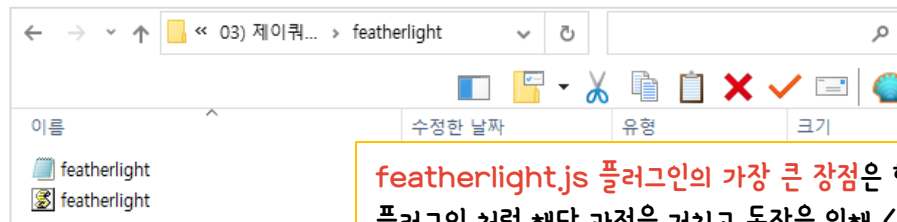
제이쿼리 라이트박스 플러그인 featherlight.js 사용법(3) - 모달과 라이트박스를 효율적으로 멋지게 띄우기



플러그인 다운로드 후 featherlight라는 폴더를 만들고 그 안에 꼭 필요한 파일만 넣고 링크해서 사용합니다. (물론 cdnjs.com 에서 CDN 주소를 링크해서 사용해도 됩니다)



▲ 페더라이트 공식 깃허브에 있는 CDN 사용해도 됩니다.



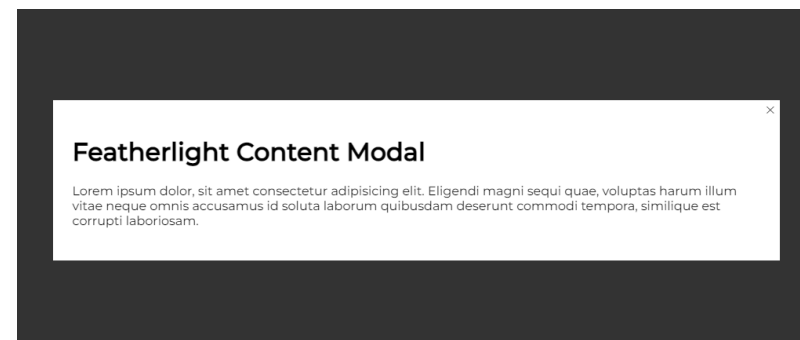
featherlight.js 플러그인의 가장 큰 장점은 현재 과정만 하면 모든게 완성됩니다. 대부분의 제이쿼리 플러그인처럼 해당 과정을 거치고 동작을 위해 <script>...</script>에 제이쿼리 또는 자바스크립트 구문을 반드시 적어야 합니다. 하지만 featherlight.js 플러그인은 특별한 옵션을 사용하지 않으면 <script>...</script>에 제이쿼리 또는 자바스크립트 구문을 작성할 필요가 없습니다.

제이쿼리 라이트박스 플러그인 featherlight.js 실습하기(1-1) - 콘텐츠 모달 띄우기(일반적인 모달 띄우기)

```
<a href="#" data-featherlight="#myModal">Open element in lightbox</a>
<div id="myModal" class="hidden">
  <h1>Featherlight Content Modal</h1>
  <p>
    Lorem ipsum dolor, sit amet consectetur adipisicing elit. Eligendi
    magni sequi quae, voluptas harum illum vitae neque omnis accusamus
    id soluta laborum quibusdam deserunt commodi tempora, similique est
    corrupti laboriosam.
  </p>
</div>
```

HTML

a태그에 data-featherlight 속성 값과
모달의 아이디 값을 일치시킵니다.

[Open element in lightbox](#)


▲ 현재 상태는 featherlight 기본 CSS로 만들어진 모달이므로 제작자가 직접 작성한 CSS 곧, Custom CSS로 예쁘게 만들도록 합니다.

```
.hidden {
  display: none;
}
```

→ 모달은 최초에 안보여야 하니까 클래스를 만들고 안보이게 합니다. (단, id에 display: none을 주면 안됩니다.)

CSS

```
<div class="featherlight" style="display: block;"> == $el
  ::before
  <div class="featherlight-content">
    <button class="featherlight-close-icon featherlight-close" aria-label="Close">X</button>
    <div id="myModal" class="hidden featherlight-inner">
      ...
    </div>
  </div>
```

→ 모달 전체 영역 디자인 CSS 선택자

모달 닫기 버튼 디자인 CSS 선택자

▲ 개발자 도구로 선택자를 알아내고 제작자가 원하는 CSS로 변경합니다.

```
.featherlight .featherlight-content {
  position: relative;
  text-align: left;
  vertical-align: middle;
  display: inline-block;
  overflow: auto;
  padding: 25px 25px 0;
  border-bottom: 25px solid transparent;
  margin-left: 5%;
  margin-right: 5%;
  max-height: 95%;
  background: #fff;
  cursor: auto;
  white-space: normal;
}
```

```
.featherlight .featherlight-close-icon {
  position: absolute;
  z-index: 9999;
  top: 0;
  right: 0;
  line-height: 25px;
  width: 25px;
  cursor: pointer;
  text-align: center;
  font-family: Arial, sans-serif;
  background: #fff;
  background-color: rgba(255, 255, 255, 0.3);
  color: #000;
  border: none;
  padding: 0;
}
```

[참고]

Custom CSS는 완성본 파일에서 보실 수 있습니다. 하지만 수강생 스스로 처음에 개발자 도구를 통해서 CSS 선택자를 알아내고 변경해보는 것이 중요합니다.

Custom CSS는 당연히 제작자에 따라 다르기 때문에 정답은 없습니다. 단, 보기 좋게 꾸미려고 노력하면 예쁘게 만들 수 있습니다.

제이쿼리 라이트박스 플러그인 featherlight.js 실습하기(1-2) - 콘텐츠 모달 띄우기(일반적인 모달 띄우기)

[참고] 제이쿼리 플러그인의 CSS를 수정할 때 반드시 제이쿼리 플러그인 CSS 파일 링크보다 style.css 파일 링크가 아래에 있어야 합니다.

```
<!-- Featherlight.js -->
<link rel="stylesheet" href="featherlight/featherlight.css">
<script src="featherlight/featherlight.js"></script>
<!-- Custom CSS -->
<link rel="stylesheet" href="style1.css">
```

Featherlight Content Modal

Lorem ipsum dolor, sit amet consectetur adipisicing elit. Eligendi magni sequi quae, voluptas harum illum vitae neque omnis accusamus id soluta laborum quibusdam deserunt commodi tempora, similique est corrupti laboriosam.

Featherlight Content Modal

Lorem ipsum dolor, sit amet consectetur adipisicing elit. Eligendi magni sequi quae, voluptas harum illum vitae neque omnis accusamus id soluta laborum quibusdam deserunt commodi tempora, similique est corrupti laboriosam.



오버레이 부분도 CSS 선택자를 찾아서 색상 또는 투명도를 변경할 수 있습니다.

```
/* Featherlight Custom CSS */
.featherlight .featherlight-content {
  width: 600px;
  border-radius: 10px;
  padding: 10px 30px !important;
}
.featherlight .featherlight-close-icon {
  font-size: 30px !important;
  right: 30px;
  top: 30px;
  color: #999;
}
.featherlight .featherlight-close-icon:hover {
  color: #000;
}
```

1. 정확히 알 수 없지만 플러그인 CSS 내에서 우선 순위를 높게 잡아 놓은 경우
2. 플러그인 CSS에 선언되어 있지 않은 속성을 Custom CSS에서 사용하는 경우

제이쿼리 라이트박스 플러그인 featherlight.js 실습하기(1-3) - 이미지 모달 띄우기(이미지 갤러리)

Goods Gallery with Featherlight.js



```
<div class="goods-gallery">
  <h1>Goods Gallery with Featherlight.js</h1>
  <div class="goods-items">
    <a href="images/goods-01.jpg" data-featherlight="image">
      
    </a>
    <a href="images/goods-02.jpg" data-featherlight="image">
      
    </a>
    <a href="images/goods-03.jpg" data-featherlight="image">
      
    </a>
    <a href="images/goods-01.jpg" data-featherlight="image">
      
    </a>
  </div>
</div>
```

HTML

→ a태그에 큰 이미지 경로를 href에 넣고 data-featherlight 속성에 image를 넣습니다.

```
/* Custom CSS */
.goods-gallery {
  width: 800px;
}
.goods-items {
  display: flex;
  gap: 10px;
}
.goods-items a img {
  width: 100%;
}
```

CSS

```
<div class="featherlight" style="display: block;">
  ::before
  <div class="featherlight-content">
    <button class="featherlight-close-icon featherlight-close" aria-label="Close">X</button>
    
  </div>
</div>
```

▲ 콘텐츠 모달과 동일한 HTML 구조와 CSS를 가지고 있습니다.

제이쿼리 라이트박스 플러그인 featherlight.js 실습하기(1-4) - iframe으로 풀스크린 모달 띄우기 ✕

앞서 학습한 콘텐츠 모달, 이미지 모달도 실무에서 활용도가 좋지만 'iframe으로 풀스크린 모달 띄우기'는 디자이너, 웹 퍼블리셔 개인 포트폴리오 홈페이지 제작에서 자신의 결과물을 세련되게 보여주는 최고의 방법이라고 생각합니다.

Open www.inflearn.com in an iframe



- iframe 태그로 구성되어 있기 때문에 iframe 부분의 너비와 높이를 가득 채우는 Custom CSS가 필요합니다.
- 닫기 버튼의 경우 눈에 띄게 디자인을 해서 보여주는 것이 좋습니다.

```
<a href="http://www.inflearn.com" data-featherlight="iframe">
  Open www.inflearn.com in an iframe
</a>
```

HTML

- a태그에 파일 경로를 href에 넣고 data-featherlight 속성에 iframe을 넣습니다.
- href 속성 값에는 어떤 파일도 링크할 수 있습니다. (URL, 문서파일, 이미지 파일, html 파일 등)

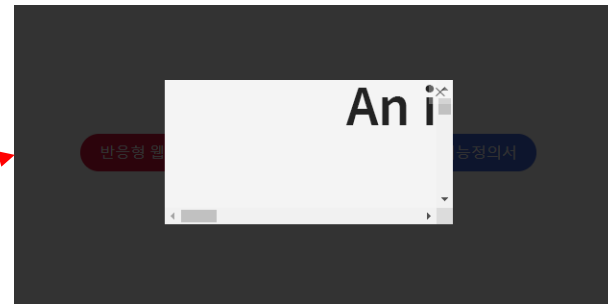
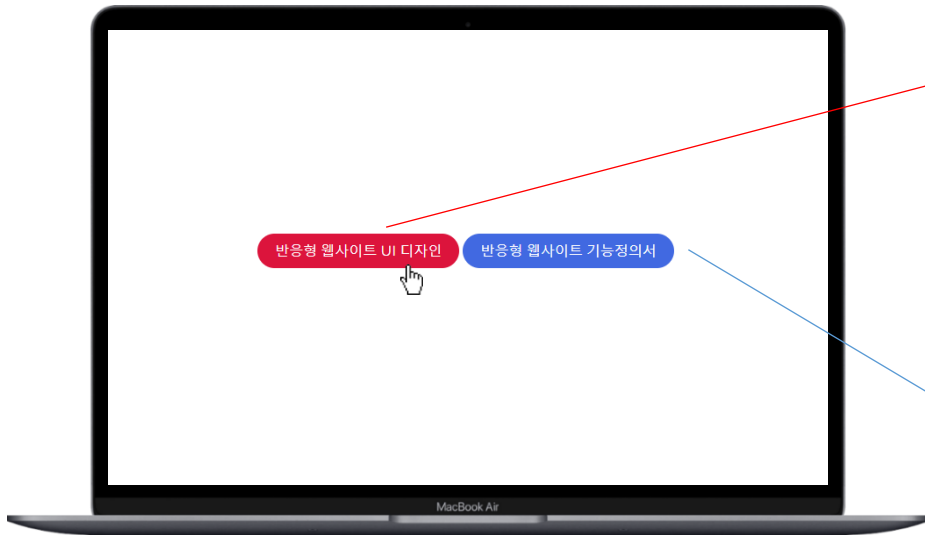
```
<div class="featherlight featherlight-iframe" style="display: block;
::before
<div class="featherlight-content">
  <button class="featherlight-close-icon featherlight-close"
    aria-label="Close">X</button>
  <iframe src="http://www.inflearn.com" class="featherlight-i
    nner" style>...</iframe>
</div>
</div>
```

← 왼쪽의 밑줄 친 부분
이 수정해야 하는
CSS 선택자

▲ iframe 모달 CSS 디자인 변경을 위한 CSS 선택자

제이쿼리 라이트박스 플러그인 featherlight.js 실습하기(1-5) - iframe으로 풀스크린 모달 띄우기 ✖

{ 커스텀 CSS를 적용하지 않은 경우 }



▲ CSS를 변경하지 않으면 위의 화면처럼 나옵니다.
그래서 반드시 커스텀 CSS로 좌우 꼭 차고 스크롤도
가능하도록 CSS를 변경해야 합니다.

```
<div class="portfolio-items">
  <a href="images/website-ui-design.png" data-featherlight="iframe">
    반응형 웹사이트 UI 디자인
  </a>
  <a href="docs/website-plan-function.pdf" data-featherlight="iframe">
    반응형 웹사이트 기능정의서
  </a>
</div>
```



- ① 웹 사이트 UI 디자인 이미지를 Full Size로 보여줍니다.
- ② 웹 사이트 HTML 와이어프레임과 기능정의서를 보여줍니다.

제이쿼리 라이트박스 플러그인 featherlight.js 실습하기(1-6) - iframe으로 풀스크린 모달 띄우기 ✕



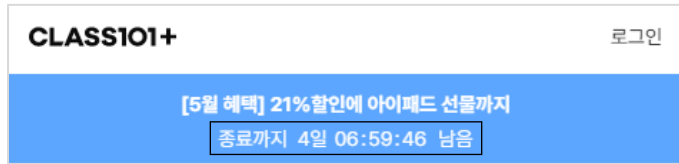
{ 커스텀 CSS를 적용한 경우 }

```
/* Featherlight Custom CSS */
.featherlight-iframe .featherlight-content {
  margin-left: 0;
  width: 100%;
  max-height: 100% !important;
  height: 100%;
}
.featherlight-iframe iframe {
  border: none;
  width: 100%;
  height: 100%;
}
.featherlight-iframe .featherlight-close-icon {
  left: 20px;
  top: 90%;
  width: 50px;
  height: 50px;
  font-size: 2em;
  color: #fff;
  background-color: tomato;
  border-radius: 50%;
  box-shadow: 0 0 10px rgba(0, 0, 0, 0.234);
  transition: 0.35s;
}
.featherlight-iframe .featherlight-close-icon:hover {
  transform: scale(1.1);
}
```

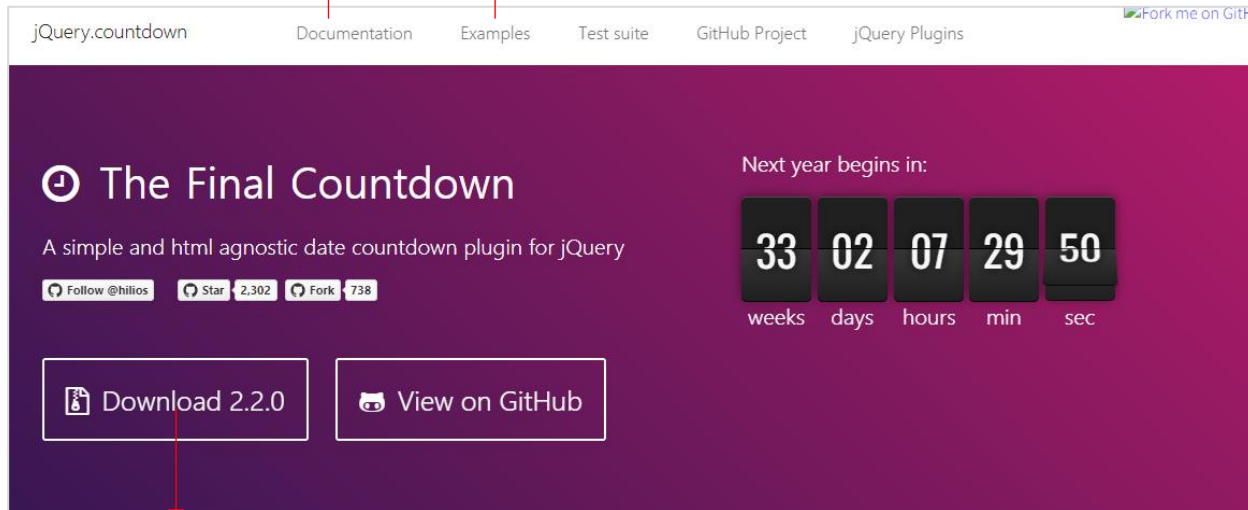
너비를 100%로 iframe 내 콘텐츠를
실 사이즈로 보여주면서 아래로 스크
롤 되게 CSS 디자인을 변경합니다.

제이쿼리 카운트다운 타이머 플러그인 - The Final Countdown 사용법(1) - 다양한 카운트다운 종류 사용!

4



▲ 클래스101 웹사이트 사용 예시

기본
사용법사용 예시(Demo)를 보면 많은 형태의 카운트 다운 타이머를 사용
할 수 있습니다. 하지만 교재에서는 필수 사용법만 다룹니다.

플러그인 JS 파일 다운로드

공식 사이트 : <http://hlios.github.io/jquery.countdown/>

- 카운트다운 타이머 플러그인은 실무에서 사용 빈도가 높은 편입니다.
- 하나의 플러그인으로 다양한 카운트다운 종류 사용가능한 것이 장점
- 제이쿼리 카운트다운 타이머 플러그인은 정말 많습니다. 많은 것들 중에 선택 기준은 예쁜 것이 아니라 제작자가 사용하기 편하고 설정하기 쉬워야 합니다.
- 이런 이유로 해당 플러그인은 여러가지 제이쿼리 카운트다운 타이머 플러그인을 검색하고 테스트해보고 결정했습니다. 제작자에 따라 원하는 플러그인을 사용해도 문제 없습니다. 단, 사용하기 쉽고 설정이 편해야 합니다.

[제이쿼리 플러그인 사용 범위]

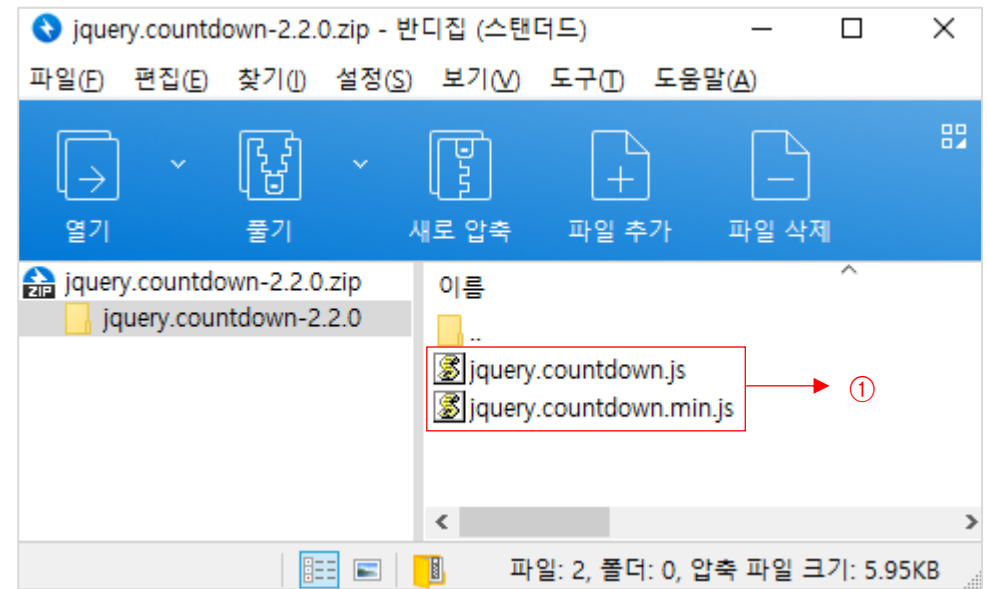
- 자바스크립트 또는 제이쿼리를 엄청 잘해서 카운트다운 타이머를 직접 만드는 것도 의미가 있지만 실무에서 일하는 거의 대부분의 퍼블리셔, 개발자는 잘 만들어진 플러그인을 사용합니다.
- 플러그인이 기능성이 아니라 단지 멋지게 보이는 용도로 사용되는 것은 지양해야 합니다. 하지만 슬라이더, 카운트다운 타이머와 같은 이런 기능성 플러그인은 잘 만들어진 것이기 때문에 잘 골라서 활용하는 것이 현명합니다.
- 물론 CSS로 제작자가 원하는 디자인으로 변경하는 것도 중요한 부분입니다.
- 해당 플러그인은 CSS를 따로 제공하지 않기 때문에 CSS를 잘 못하는 사람에게는 비추입니다. 하지만 CSS를 잘 하는 사람은 Pure한 상태에서 꾸밀 수 있어서 오히려 장점이 됩니다.

제이쿼리 카운트다운 타이머 플러그인 - The Final Countdown 사용법(2) - 다양한 카운트다운 종류 사용!

Documentation > Install

- ① 다운로드해서 JS 파일을 링크하거나 CDN을 복사합니다.
 - 해당 플러그인은 CSS 파일은 없고 JS 파일만 있음
 - 둘 중에 아무거나 사용해도 상관 없음
- ② 임의의 폴더를 만들고 해당 폴더에 압축파일에 있는 js 파일을 드래그해서 복사합니다.
- ③ 사용법 연습을 위해 index.html을 만들고 <head>...</head> 사이에 js 파일을 링크합니다.

```
<head>  
  <meta charset="UTF-8">  
  <title>The Final Countdown 사용법</title>  
  <!-- jQuery CDN -->  
  <script src="https://code.jquery.com/jquery-3.5.1.min.js"></script>  
  <!-- The Final Countdown -->  
  <script src="the-final-countdown/jquery.countdown.min.js"></script>  
</head>
```



```
04) 제이쿼리 카운트다운 타이머 플러그인 - THE FINAL COUNTDOWN  
  the-final-countdown  
    JS jquery.countdown.min.js  
    index.html
```

제이쿼리 카운트다운 타이머 플러그인 - The Final Countdown 사용법(3) - 다양한 카운트다운 종류 사용!

Documentation > Install

- ① 구현된 코드로 브라우저 미리보기 화면
- ② HTML body 안에 들어갈 html 코드
- id는 제작자가 원하는 이름으로 변경해도 상관없음 ex) #event-countdown
- ③ 카운트다운 타이머를 구동 시키는 제이쿼리 스크립트 구문

19 days 06:44:40 ①

```
<div id="early-bird-discount"></div> ②

<script type="text/javascript">
  /* The Final Countdown */
  $("#early-bird-discount")
    .countdown("2023/06/01", function(event) { ③
      $(this).text(
        event.strftime('%D days %H:%M:%S')
      );
    });
</script>
```

종료까지 19일 06:42:32 남음

```
<script type="text/javascript">
  /* The Final Countdown */
  $("#early-bird-discount")
    .countdown("2023/06/01", function(event) {
      $(this).text(
        event.strftime('종료까지 %D일 %H:%M:%S 남음')
      );
    });
</script>
```

↓ 제작자가 원하는 형식과 문구로 변경

19일 06시 36분 08초

```
<script type="text/javascript">
  /* The Final Countdown */
  $("#early-bird-discount")
    .countdown("2023/06/01", function(event) {
      $(this).text(
        event.strftime('%D일 %H시 %M분 %S초')
      );
    });
</script>
```

설정 날짜 입력
(날짜 형식은 2023-06-01으로 넣어도 정상 작동)

- 왼쪽부터 %부분을 순서대로
- 일, 시, 분, 초
- days는 문자열 이므로 변경 가능 ex) 일
- %D %H %M %S 이외 부분은 변경 가능
- 원하는 항목만 출력하고 싶으면 필요 없는 부분을 삭제하면 됩니다. 예를 들어 초를 표시하는 부분이 필요 없다면 %S를 지우면 됩니다.

제이쿼리 카운트다운 타이머 플러그인 - The Final Countdown 사용법(4) - Custom CSS #01

{ 커스텀 CSS를 적용하지 않은 경우 }

종료까지 19일 06:42:32 남음

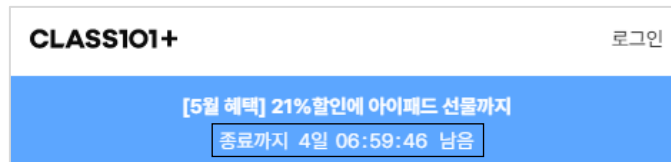
```
<script type="text/javascript">
  /* The Final Countdown */
  $("#early-bird-discount")
    .countdown("2023/06/01", function(event) {
      $(this).text(
        event.strftime('종료까지 %D일 %H:%M:%S 남음')
      );
    });
</script>
```

▲ CSS를 변경하지 않으면 위의 화면처럼 나옵니다. 하지만 해당 플러그인은 CSS를 제공하지 않기 때문에 개발자 도구로 봐도 변경할 CSS 선택자를 찾을 수 없습니다. 단지 div 하나 밖에 없습니다.

div 하나만 있으니까 세부적으로 선택해서 CSS 변경이 불가능 합니다. 하지만 플러그인 제작자가 이런 부분을 할 수 있도록 여지를 두었습니다.

```
<div id="early-bird-discount">종료까지 19일 06:30:56 남음</div>
```

{ 커스텀 CSS를 적용한 경우 }



▲ 위의 경우 같이 심플한 디자인의 경우 #early-bird-discount 라는 선택자에 글자크기와 색상을 주면 됩니다.

```
/* The Final Countdown Custom CSS */
#early-bird-discount { font-size: 26px; }
#early-bird-discount b {
  font-weight: normal; color: crimson;
}
#early-bird-discount em {
  font-style: normal; background-color: #000;
  color: #fff; padding: 0 5px; border-radius: 5px;
}
```

종료까지 19일 06 : 13 : 22 남음

```
/* The Final Countdown */
$("#early-bird-discount")
  .countdown("2023/06/01", function(event) {
    $(this).html(
      event.strftime('<b>종료까지</b> %D일 <em>%H</em> : <em>%M</em> : <em>%S</em> 남음')
    );
  });
```

▲ 개별 요소를 html로 마크업 했기 때문에 CSS 선택자로 자유롭게 디자인 할 수 있습니다. (단, html 태그가 들어가기 때문에 text() 메서드가 아니라 html() 메서드를 사용해야 합니다.)

제이쿼리 카운트다운 타이머 플러그인 - The Final Countdown 사용법(4) - Custom CSS #02

{ 더 다양한 커스텀 CSS를 적용한 경우 }

종료까지 19일 06 : 00 : 59 남음
hrs min sec

<div id="early-bird-discount"></div>

▲ 동일한 HTML 구조

```
<script type="text/javascript">
  /* The Final Countdown */
  $("#early-bird-discount")
    .countdown("2023/06/01", function(event) {
      $(this).html(
        event.strftime('<b>종료까지</b> %D일 <em>%H</em> : <em>%M</em> : <em>%S</em> 남음')
      );
    });
</script>
```

▲ 동일한 JS 코드

```
/* The Final Countdown Custom CSS */
#early-bird-discount {
  font-size: 26px;
}
#early-bird-discount b {
  font-weight: normal;
  color: crimson;
}
#early-bird-discount em {
  font-style: normal;
  background-color: #000;
  color: #fff;
  padding: 0 5px;
  border-radius: 5px;
  position: relative;
}
```

```
#early-bird-discount em:after {
  content: '';
  position: absolute;
  top: 100%;
  left: 0;
  color: #000;
  font-size: 10px;
  width: 100%;
  text-align: center;
}
#early-bird-discount em:nth-of-type(1):after {
  content: 'hrs';
}
#early-bird-discount em:nth-of-type(2):after {
  content: 'min';
}
#early-bird-discount em:nth-of-type(3):after {
  content: 'sec';
}
```

b태그가 있기 때문에 nth-child()를 사용하면 안됩니다. 반드시 nth-of-type()을 사용해야 합니다.